

O'REILLY®



**Early
Release**

**RAW &
UNEDITED**

Hands-On Generative AI with Transformers and Diffusion Models

Omar Sanseviero, Pedro Cuenca,
Apolinário Passos & Jonathan Whitaker

O'REILLY®



Hands-On Generative AI with Transformers and Diffusion Models

Omar Sanseviero, Pedro Cuenca,
Apolinário Passos & Jonathan Whitaker

O'REILLY®



Hands-On Generative AI with Transformers and Diffusion Models

Omar Sanseviero, Pedro Cuenca,
Apolinário Passos & Jonathan Whitaker

Hands-On Generative AI with Transformers and Diffusion Models

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Omar Sanseviero, Pedro Cuenca, Apolinario Passos, and
Jonathan Whitaker



Beijing • Boston • Farnham • Sebastopol • Tokyo

Hands-On Generative AI with Transformers and Diffusion Models

by Omar Sanseviero, Pedro Cuenca, Apolinario Passos, and Jonathan Whitaker

Copyright © 2025 Omar Sanseviero, Pedro Cuenca, Apolinario Passos, and Jonathan Whitaker. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<https://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Acquisitions Editor: Nicole Butterfield
- Development Editor: Jill Leonard
- Production Editor: Gregory Hyman
- Interior Designer: David Futato

- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea
- December 2024: First Edition

Revision History for the Early Release

- 2023-03-16: First Release
- 2024-01-19: Second Release
- 2024-03-20: Third Release
- 2024-05-20: Fourth Release
- 2024-07-16: Fifth Release
- 2024-09-06: Sixth Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098149246> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. Hands-On Generative AI with Transformers and Diffusion Models, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the

information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-14924-6

[LSI]

Preface

Generative AI is a revolutionary technology that has rapidly transitioned from lab demos to real-world applications, impacting billions. It can create new content—images, text, audio, videos, and more—by learning patterns from existing data, thereby enhancing creativity, augmenting data, or assisting in many tasks. For instance, a generative AI model trained on music can compose new melodies, while one trained on text can generate stories or even programming code.

This book isn't just for experts – it's for anyone who wants to learn about this fascinating new field. We won't focus on building models from scratch or diving straight into complicated mathematics. Instead, we'll leverage existing models to solve real-world problems, helping you to build a solid intuition around how these techniques work and providing the foundation for you to keep exploring.

This hands-on approach, we hope, will help you get up and running quickly and efficiently with generative AI. You'll learn how to use pre-trained models, adapt them for your needs, and generate new data with them. You'll also learn how to evaluate the quality of generated data and explore ethical and social

issues that may arise from using generative AI. This exposure will allow you to stay up to date with new models and help you identify areas that you may want to explore more deeply.

Who Should Read This Book

Given the impressive products and news you might have seen about generative AI, it's normal to be excited, or worried, about it! Whether you're curious about how programs can generate images, want to train a model to tweet in your style, or are looking to gain a deeper understanding of products like ChatGPT, this book is for you. With generative AI, we can do all of that and many other things, including:

- Write summaries of news articles
- Generate images based on a description
- Enhance the quality of an image
- Transcribe meetings
- Generate synthetic speech in your voice style
- Incorporate new subjects or styles into image generation models, like creating images of "your cat dressed as an astronaut".

No matter your reason, you've decided to learn about generative AI, and this book will guide you through it.

Prerequisites

This book assumes that you are comfortable programming in Python and have a foundational understanding of what Machine Learning is, including basic usage of frameworks like PyTorch or TensorFlow. Having practical experience with training models is not required, but it will be helpful to understand the content with more depth. The following resources provide a good foundation for the topics covered in this book:

- [Hands-On Machine Learning with Scikit-Learn and TensorFlow](#), by Aurélien Géron (O'Reilly)
- [Deep Learning for Coders with fastai and PyTorch](#), by Jeremy Howard and Sylvain Gugger (O'Reilly)

If you feel intimidated by it, don't worry! The book is designed to enhance your intuition and provide a hands-on approach to help you get started.

What You Will Learn

This book is divided into three parts:

- **In Part 1**, we'll introduce the fundamental building blocks of generative AI. You'll learn how to use pre-trained models to generate text and images. This part will help you understand the basics of the field and understand the big picture.
- **Part 2** is all about fine-tuning, showcasing ways to take existing models and adapt them to your needs. We'll walk you through how to *teach* a diffusion model a new concept, customize a transformer model to classify text and reply in conversations, and explore advanced techniques for working with large models on limited hardware. Don't worry if this is the first time you read about transformer or diffusion models, you'll learn about them soon.
- **In Part 3**, we'll extend the ideas from the previous parts, generating new modalities such as audio and getting creative with new applications. After you've read this book, you'll have a solid understanding of the methods and techniques on which generative applications are built.

How to Read This Book

We designed the book to be read in order, but we have kept the chapters as self-contained as possible so you can jump around to the parts that interest you most. Many of the ideas covered in

this book apply to multiple modalities, so even if you are only interested in one particular domain (such as image generation), you may still find it valuable to skim through the other chapters.

We've included exercises and code examples throughout the book, designed to help you get hands-on with the material. Try to complete these exercises as you go along, and where possible, see if you can adapt the examples to your use cases. Trying things out for yourself will help you build a much deeper understanding of the material.

Finally, each chapter includes some additional resources in their summary for further reading. We encourage you to explore these resources to deepen your understanding of the topics covered in the book. You don't need to do it before you progress to a new chapter; you can come back later whenever you are ready to go deeper into the subjects that interest you.

Software and Hardware Requirements

To get the most out of this book, we highly recommend running the code examples as you read along. Experimenting with the

code by making changes and exploring different scenarios will enhance your understanding. Working with transformers and diffusion models can be computationally intensive, so having access to a computer with an NVIDIA GPU is beneficial. While a GPU is not mandatory, it will significantly speed up training times.

There are multiple online options that you can use, such as [Google Colaboratory](#) and [Kaggle Notebooks](#). Follow these instructions to setup your environment and follow along:

- **Using Google Colab**

Most code should work on any [Google Colab](#) instance. We recommend you use GPU runtimes for chapters with training loops.

- **Running Code Locally**

To run the code in your computer, create a Python 3.10 virtual environment using your preferred method. As an example, you can do it with *conda* like this:

```
conda create -n genaibook python=3.10
conda activate genaibook
```

For optimal performance, we recommend using a CUDA-compatible GPU.¹ If you don't know what CUDA is, don't worry, we'll explain it in the book.

There are many support utilities and helper functions used throughout the book. To access them, please install the *genaibook* package:

```
pip install genaibook
```

This will in turn install the libraries required to run transformers and diffusion models, along with PyTorch, Matplotlib, NumPy, and other essentials.

All code examples and supplementary material can be found in the [book's GitHub repository](#). You can run all the examples interactively in Jupyter Notebooks, and the repository will be regularly updated with the latest resources.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

- O'Reilly Media, Inc.

- 1005 Gravenstein Highway North
- Sebastopol, CA 95472
- 800-998-9938 (in the United States or Canada)
- 707-829-0515 (international or local)
- 707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://learning.oreilly.com/library/view/hands-on-generative-ai/9781098149239/>.

State of the Art: A Moving Target

State of the Art (SOTA) is used to describe the highest level of performance currently achieved in a particular task or domain. In the field of generative AI, the SOTA is constantly changing as new models are developed and new techniques are discovered. This book will provide you with a solid grounding in the fundamentals of generative AI, but by the time you read it, new models will have been released that outperform the ones we discuss here.

Rather than trying to chase the ever-shifting *best*, we've tried to focus on general principles that will help you to understand

how the models work in a way that will be useful even as the field continues to evolve. New models rarely come out of nowhere and often build on the ideas of previous models. By understanding the fundamentals, you'll be better equipped to understand the latest developments as they happen.

- Rather than GPU, you can also use the MPS device, which might work on Macs with Apple Silicon, but we have not tested this configuration extensively.

Chapter 1. An Introduction to Generative Media

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the first chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

Generative models have become widely popular in recent years. If you’re reading this book, you’ve probably interacted with a generative model at some point. Maybe you’ve used ChatGPT to generate text, used style transfer in apps like Instagram, or seen the deepfake videos that have been making

headlines. These are all examples of generative models in action!

In this book, we'll explore the world of generative models, starting with the basics of two families of generative models, transformers and diffusion, and working our way up to more advanced topics. We'll cover the different types of generative models, how they work, and how to use them. In this chapter, we'll cover some of the history of how we got here and take a look at the capabilities offered by some of the models, which we'll explore in more depth throughout the book.

So, what exactly is generative modeling? At its core, it's about teaching a *model* to *generate* new data that resembles its training data. For example, if I train a model on a dataset of images of cats, I can then use that model to generate new images of cats that look like they could have come from the original dataset. This is a powerful idea, and it has a wide range of applications, from creating novel images and videos to generating text with a specific style.

Throughout this book, you'll discover popular tools that make using existing generative models straightforward. The world of Machine Learning offers numerous *open-access* models, trained on large datasets, available for anyone to use. Training these

models from scratch can be costly and time-consuming, but open-access models provide a practical and efficient alternative. These *pre-trained* models can generate new data, classify existing data, and be adapted for new applications. One of the most popular places to find open-access models is [Hugging Face](#), a platform with over two million models for many Machine Learning tasks, including image generation.

Generating Images

As an example of an *open-source* library, we'll kick off with *diffusers*. *diffusers* is a popular library that provides access to state-of-the-art diffusion models. It's a powerful, simple toolbox that allows us to quickly load and train diffusion models.

By going to the Hugging Face Hub and [filtering](#) for models that generate images based on a text prompt (`text-to-image`), we can find some of the most popular models, such as Stable Diffusion and SDXL. We'll use Lykon's Dreamshaper 8, a community-fine tune of Stable Diffusion version 1.5, a diffusion model capable of generating high-quality images. If you browse the model website, you can read the model card, an essential document for discoverability and reproducibility. There, you

can read about the model, how it was trained, intended use cases, and more.

Given we have a model (Stable Diffusion) and a tool to use the model (*diffusers*), we can now generate our first image! When we load models, we'll need to send them to a specific hardware device, such as CPU (`cpu`), GPU (`cuda` or `cuda:0`), or Mac hardware called Metal (`mps`). The `genaibook` library we mentioned in the Preface has a utility function to select an appropriate device depending on where you run the example code. For example, the following code will assign `cuda` to the `device` variable if you have a GPU.

```
from genaibook.core import get_device

device = get_device()
print(f"Using device: {device}")
```

```
Using device: cuda
```

Next, we'll load Stable Diffusion 1.5. *diffusers* offers a convenient, high-level wrapper called `StableDiffusionPipeline`, which is ideal for this use case.

Don't worry about all the parameters for now - the highlights are:

- There are many models with the Stable Diffusion architecture, so we need to specify the one we want to use, we are going to use `Lykon/dreamshaper-8`, a high quality fine-tune of `runwayml/stable-diffusion-v1-5`, in this case.
- We need to specify the *precision* we'll load the model with. Precision is something we'll learn more about later. At a high level, models are composed of many *parameters* (millions or billions of them). Each parameter is a number learned during training, and we can store these parameters with different levels of precision (in other words, we can use more bits to store the model). A larger precision allows the model to store more information, but it also requires more memory and computation. On the other hand, we can use a lower precision by setting `torch_dtype=float16` and use less memory than the default `float32`. When doing inference (a fancy way of saying "`executing`" the models), using `float16` is usually fine.¹

The first time you run this code, it can take a bit: the pipeline downloads a model of multiple gigabytes, after all! If you load the pipeline a second time, it will only re-download the model if

there has been a change in the remote repository² that hosts the model on Hugging Face. Hugging Face libraries store the model locally in a cache, making things much faster for subsequent loads.

```
import torch
from diffusers import StableDiffusionPipeline

pipe =
StableDiffusionPipeline.from_pretrained(
    "Lykon/dreamshaper-8",
    torch_dtype=torch.float16,
    variant="fp16",
).to(device)
```

Now that the model is loaded, we can define a *prompt*- the text input the model will receive. We can then pass the prompt through the model and generate our first image based on that text! Try inputting the following prompt:

```
prompt = "a photograph of an astronaut riding
a horse"
pipe(prompt).images[0]
```

```
0%|          | 0/50 [00:00<?, ?it/s]
```



Figure 1-1: Image generated in a few seconds with a Stable Diffusion text-to-image model.

Exciting! With a couple of lines of code, we generated a new image, shown in Figure 1-1. Play with the prompt and generate new images. You might notice two things. First, running the same code will generate different images each time. This is because the diffusion process is *stochastic* in nature, meaning it

uses randomness to generate images. We can control this randomness by setting a *seed*:

```
import torch
torch.manual_seed(0)
```

Second, the generated images are not perfect. They might have artifacts, be blurry, or not match the prompt at all. We'll explore these limitations and how to improve the quality of the generated images in later chapters. For instance:

- Chapters 4 and 5 dive into all the components behind diffusion models and how to get from text to new images. They rely on foundational methods, like AutoEncoders – introduced in Chapter 3-, that can learn efficient representations from input data and reduce the compute requirements to build diffusion and other generative models.
- In Chapter 7, we'll learn how to teach new concepts to Stable Diffusion. For example, we can teach Stable Diffusion the concept of "my dog" to generate images of the author's dog in novel scenarios, such as "my dog visiting the moon".
- Chapter 8 shows how diffusion models can be used for more than just image generation, such as editing images

with a prompt or filling empty parts of an image.

Generating Text

Just as *diffusers* is a very convenient library for diffusion models, the popular *transformers* library is extremely useful for running transformers-based models and adapting to new use cases. It provides a standardized interface for a wide range of tasks, such as generating text, detecting objects in images, and transcribing an audio file into text.

The *transformers* library provides different layers of abstractions. For example, if you don't care about all the internals, the easiest is to use `pipeline`, which abstracts all the processing required to get a prediction. We can instantiate a pipeline by calling the `pipeline()` function and specifying which task we want to solve, such as `text-classification`.

```
from transformers import pipeline

classifier = pipeline("text-classification",
device=device)
classifier("This movie is disgustingly good
!")
```



```
[{'label': 'POSITIVE', 'score':  
0.9998536109924316}]
```

The model correctly predicted that the sentiment in the input text was positive. By default, the text classification pipeline uses a sentiment analysis model under the hood, but we can also specify other transformers-based text classification models.

Similarly, we can switch the task to text generation (`text-generation`), with which we can generate new text based on an input prompt. By default, the pipeline uses the GPT-2 model. The transformer pipeline uses a default maximum number of words to generate, so don't be surprised if the output is truncated. We'll learn later how to change this.

```
from transformers import set_seed  
  
# Setting the seed ensures we get the same  
# results every time we run this code  
set_seed(10)
```

```
generator = pipeline("text-generation",  
device=device)
```

```
prompt = "It was a dark and stormy"  
generator(prompt)[0]["generated_text"]
```

```
It was a dark and stormy year, and my mind  
went blank," says the 27-ye  
ar-old, who has become obsessed with art,  
poetry and music since movin  
g to France. "I don't really know why, but  
there are things
```

Although GPT-2 is not a great model by today's standards, it gives us an initial example of transformers' generation capabilities while using a small model. The same concepts we learn about GPT-2 can be applied to models such as Llama or Mistral, some of the most powerful open-access models (at the time of writing). Throughout the book, we'll strike a balance between the quality and size of the models. Usually, larger models have higher-quality generations. At the same time, we want people with consumer computers or access to free services, such as Google Colab, to be able to create new generations by running code.

- Chapter 2 will teach you how transformer models work under the hood. We'll dive into different types of

transformer models and how to use them for generating text.

- Chapter 6 will teach you how to continue training transformer models with our data for different use cases. This will allow us to make conversational models like those you might have used with ChatGPT or Bard. We'll also discuss efficient training approaches so you can run the training on your computer.

Generating Sound Clips

Generative models are not limited to images and text. Models can generate videos, short songs, synthetic spoken speech, protein proposals, and more!

Chapter 9 deepens into audio-related tasks that can be solved with Machine Learning, such as transcribing meetings and generating sound effects. For now, we can limit ourselves to the now familiar *transformers* `pipeline` and use the small version of MusicGen, a model released by Meta to generate music conditioned on text.

```
pipe = pipeline("text-to-audio",  
model="facebook/musicgen-small",
```

```
device=device)
data = pipe("electric rock solo, very
intense")
```

```
print(data)
```

```
{'audio': array([[0.12342193, 0.11794732,
0.14775363, ..., 0.0265964 ,
0.02168683, 0.03067675]]),
dtype=float32), 'sampling_rate': 32000}
```

Later, we'll learn how audio data is represented and what these numbers are. Of course, there's no way for us to print the audio file directly in the book! The best alternative is to show a viewer in our notebook or save the audio to a file we can play with our favorite audio application. For example, we can use

`IPython.display()` for this!

```
import IPython.display as ipd

display(ipd.Audio(data["audio"][0],
rate=data["sampling_rate"]))
```

Ethical and Societal Implications

While generative models offer remarkable capabilities, their widespread adoption raises important considerations around ethics and societal impact. It's important to keep them in mind as we explore the capabilities of generative models. Here are a few key areas to consider:

- **Privacy and consent:** The ability of generative models to generate realistic images and videos based on very little data poses significant challenges to privacy. For example, creating synthetic images from a small set of real images from an individual raises questions about using personal data without consent. It also increases the risk of creating deepfakes, which can be used to spread misinformation or harm individuals.
- **Bias and fairness:** Generative models are trained on large datasets that contain biases. These biases can be inherited and amplified by the generative models, as we'll explore in Chapter 2. For example, biased datasets used to train image generation models may generate stereotypical or discriminatory images. It's important to consider mitigating these biases and ensure that generative models are used fairly and ethically.

- **Regulation:** Given the potential risks associated with generative models, there is a growing call for regulatory oversight and accountability mechanisms to ensure responsible development. This includes transparency requirements, ethical guidelines, and legal frameworks to address the misuse of generative models.

It's important to approach generative models with a thoughtful and ethical mindset. As we explore the capabilities of these models, we'll also consider the ethical implications and how to use them responsibly.

Where We've Been and Where Things Stand

Generative models began decades ago with efforts focused on rule-based systems. As computing power and data availability increased, generative models evolved to use statistical methods and Machine Learning. With the emergence of deep learning as a powerful paradigm in Machine Learning and breakthroughs in the fields of image and speech recognition, generative models have advanced significantly. Although invented decades ago, Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have become widely popular in the

last decade. CNNs revolutionized image processing tasks, and RNNs brought sequential data modeling capabilities, enabling tasks like translating text and text generation.

The introduction of Generative Adversarial Networks (GANs) by Ian Goodfellow in 2014, and variants such as DCGAN and conditional GANs, brought a new era of generative models. GANs have been used to generate high-quality images and applied to tasks like style transfer, enabling users to apply artistic styles to their images with astonishing realism. Although quite powerful, the quality of GANs has been surpassed by diffusion models in recent years.

Similarly, although RNNs were the to-go tool for language modeling, transformer models, including architectures like GPT, achieved state-of-the-art performance in Natural Language Processing (NLP). These models have demonstrated remarkable capabilities in tasks such as language understanding, text generation, and machine translation. GPT, in particular, became extremely popular due to its ability to generate coherent and contextually relevant text. Not long afterward, a huge wave of generative language models emerged.

The field of generative AI is more accessible than ever due to the rapid expansion of research, resources, and development in

recent years. A growing community interested in the area, a rich open-source ecosystem, and research facilitating deployment have led to a wide range of applications and use cases. Between 2023 and 2024, a new generation of models that can generate high-quality images, text, code, videos, and more has emerged; examples include ChatGPT, DALL·E, Imagen, Stable Diffusion, Llama, Mistral, and many others.

How Are Generative AI Models Created?

Typically, it comes down to big budgets or open-source.

Several of the most impressive generative models in the past couple of years were created by influential research labs in big, private companies. OpenAI developed ChatGPT, DALL·E, and Sora; Google built Imagen, Bard, and Gemini; and Meta created Llama and Code Llama. There's a varying degree of openness in the way these models are released. Some can be used via specific UIs, some have access through developer APIs, and some are just announced as research reports with no public access at all. In some cases, code and model weights are released as well: these are usually called *open-source* releases because those are the essential artifacts necessary to run the

model on your hardware. Frequently, however, they are kept hidden for strategic reasons.

At the same time, an ever-increasing, energetic, and enthusiastic community uses open-source models as the clay for their creativity. All types of practitioners, including researchers, engineers, tinkerers, and amateurs, build on top of each other's work and come up with novel solutions and clever ideas that push the field forward, one commit at a time. Some of these ideas make their way into the theoretical corpus of knowledge where researchers draw from, and new impressive models that use them come out after a while.

Big models, even when hidden, serve as inspiration for the community, whose work yields fruits that serve the field as a whole.

This cycle can only work because some of the models are open-sourced and can be used by the community. Companies that release open-source models don't do it for altruistic reasons but because they discover economic value in this strategy. By providing code and models that are adopted by the community, they receive public scrutiny with bug fixes, new ideas, derived model architectures, or even new datasets that work well with the models released. Because all these contributions are based

on the assets they published, these companies can quickly adopt them and thus move faster than they would on their own. When Meta released Llama, one of the most popular language models, a thriving ecosystem organically grew around it. Both established and new companies alike, including Meta, Stability AI (Stable Diffusion), or Mistral AI, have embraced varying degrees of open-source as part of their business strategy. This is as legitimate as the strategy of competing companies that prefer to keep their trade secrets behind closed doors (even if those companies can also draw from the open-source community).

At this point, we'd like to clarify that model releases are rarely truly open-source. Unlike in the software world, source code is not enough to fully understand a Machine Learning system. Model weights are not enough either: they are just the final output of the model training process. Being able to exactly replicate an existing model would require the source code used to train the model (not just the modeling code or the inference code), the training regime and parameters, and, crucially, all the data used for training. None of these, and particularly the data, are usually released. If there were access to these details, it would be possible for the community and the public to understand how the model works, explore the biases that may afflict it, and better assess its strengths and limitations. Access to the weights and model code provides an imperfect

estimation of all this knowledge, but the actual hard data would be much better. On top of it, even when the models are publicly released, they often come out with a special license that does not adhere to the Open Source Initiative's definition of open-source. This is not to say that the models are not useful or that the companies are not doing a good thing by releasing them, but it's an important context to keep in mind and one of the reasons we'll often say *open access* instead of *open-source*.

Be it as it may, there has never been a better time to build generative models or *with* generative models. You don't need to be an engineer in a top-notch research lab to come up with ideas to solve the problems that interest you or to contribute to the field. We hope you find these pages helpful in your journey!

Summary

Hopefully, after generating your first images, text, and audios, you'll be excited to learn how diffusion and transformers work under the hood, how to adapt them for new use cases, and how to use them for different creative applications. Although this chapter focused on high-level tools, we'll build solid foundations and intuition on how these models work as we

embark on our generative journey. Let's go ahead and learn about the principles of generative models!

- You might wonder about the `variant` parameter. For some models, you might find multiple checkpoints with different precision. When specifying `torch_dtype=float16`, we download the default model (`float32`) and convert it to `float16`. By also specifying the `fp16` variant, we're downloading a smaller checkpoint already stored in `float16` precision, which requires half the bandwidth and storage to download it. Check the model you want to use to find out if there are multiple precision variants.
- Hugging Face repositories are git-based repositories under the hood

Chapter 2. Transformers

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the second chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

Many trace the most recent wave of advances in generative AI to the introduction of a class of models called *transformers* in 2017. Their most well-known application is the powerful *Large Language Model (LLM)*, such as Llama and GPT-4, used by hundreds of millions daily. Transformers have become a backbone for modern AI applications, powering everything from chatbots and search systems to machine translation and

content summarization. They've even branched out beyond text, making waves in fields like computer vision, music generation, and protein folding. In this chapter, we'll explore the core ideas behind transformers and how they work, with a focus on one of the most common applications: language modeling.

Before we dive into the details of transformers, let's take a step back and understand what language modeling is. At its core, a language model (LM) is a probabilistic model that learns to predict the next word (or *token*) in a sequence based on the preceding or surrounding words. Doing so captures language's underlying structure and patterns, allowing it to generate realistic and coherent text. For example, given the sentence "I began my day eating ", a language model might predict the next word as "breakfast" with a high probability.

So, how do transformers fit into this picture? Transformers are designed to handle long-range dependencies and complex relationships between words efficiently and expressively. For example, imagine that you want to use an LM to summarize a news article, which might contain hundreds or even thousands of words. Traditional LMs, such as Recurrent Neural Networks, struggle with long contexts, so the summary might skip critical details from the beginning of the article. Transformer-based

LMs, however, show strong results in this task. Besides high-quality generations, transformers have other properties, such as efficient parallelization of training, scalability, and knowledge transfer, making them popular and well-suited for multiple tasks. At the heart of this innovation lies a mechanism called self-attention, which allows the model to weigh the importance of each word in the context of the entire sequence.

To help us build intuition about how language models work, we'll use code examples that interact with existing models, and we'll describe the relevant pieces as we find them. Let's get to it.

A Language Model in Action

In this section, we will load and interact with an existing small (pre-trained) transformer model to get a high-level understanding of how they work. In recent years, companies, research labs, and open communities have released thousands of open models that you can use. These models are composed from

We'll pick a small model you can run directly in your hardware, but consider that the same principles apply to the larger (over 100 times larger!) and more powerful models that have since been released. Some good examples of small models are:

- **GPT-2 (137M)**, which made headlines in 2019 for its (then) impressive text-generation capabilities. Although small and almost quaint by today's standards, GPT-2 illustrates how these language models work.
- **Qwen2 (494M)** is an Alibaba model and part of the Qwen family. The Qwen family has models with 500 million to over 100 billion parameters.
- **SmolLM (135M)** is a model by Hugging Face trained with very high-quality data. The authors released models with 135 million, 360 million, and 1.7 billion parameters.

Chapter 6 will provide more insights into picking the suitable model for your use case. For now, we suggest exploring one of the models above (or all of them!).

Tokenizing Text

Let's begin our journey to generate some text based on an initial input. For example, given the phrase "it was a dark and stormy", we want the model to generate some words to continue it. Models can't receive text directly as input; their input must be data represented as numbers. To feed text into a model, we must first find a way to turn sequences into numbers. This process is called tokenization, a crucial step in any Natural Language Processing (NLP) pipeline.

An easy option would be to split the text into individual characters and assign each a unique numerical ID. This scheme could be helpful for languages such as Chinese, where each character carries much information. In languages like English, this creates a very small token *vocabulary*, and there will be very few unknown tokens (characters not found during training) when running inference. However, this method requires many tokens to represent a string, which is bad for performance and erases some of the structure and meaning of the text – a downside for accuracy. Each character carries very little information, making it hard for the model to learn the underlying structure of the text.

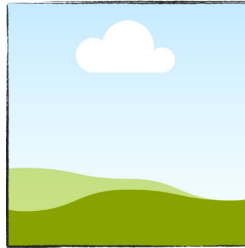


Figure 2-1. Figure 2-1. An example of character-level tokenization. Each letter has its own ID, with all instances of the same letter having the same ID. In this example, the IDs correspond to their position in the alphabet.

Another approach could be to split the text into individual words. While this lets us capture more meaning per token, it has the downsides that we need to deal with more unknown words (e.g., typos or slang), we need to deal with different forms of the same word (e.g., "run", "runs", and "running"), and we might end with a very large vocabulary, which could easily be over half a million words for languages such as English.

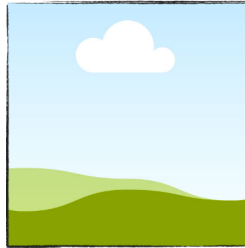


Figure 2-2. Figure 2-2. An example of word-level tokenization. The same word always has the same ID.

Modern tokenization strategies strike a balance between these two extremes, splitting the text into subwords that capture both the structure and meaning of the text while still being able to handle unknown words and different forms of the same word. Characters that are usually found together (like most frequent words) can be assigned a single token that represents the whole word or group. Long or complicated words, or words with many inflections, may be split into multiple tokens, where each one usually represents a meaningful section of the word. There is no single best tokenizer; each language model comes with its own one. The differences between tokenizers reside in the

number of tokens supported and the tokenization strategy. For example, the GPT-2 tokenizer averages 1.3 tokens per word.¹

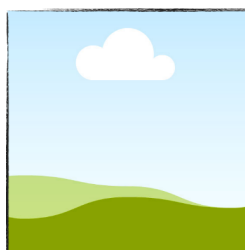


Figure 2-3. Figure 2-3. An example of sub-word tokenization. Llama is split in two tokens because it was likely not a common word in the data used to create the tokenizer.

Let's find out how the Qwen tokenizer handles a sentence. We'll first use the *transformers* library to load the tokenizer corresponding to Qwen. Then, we'll run the input text (also called *prompt*) through the tokenizer to encode the string into numbers representing the tokens. We'll use the `decode()` method to convert each ID back into its corresponding token for demonstration purposes.

```
from transformers import AutoTokenizer

# Use the id of the model you want to use
# GPT-2 "openai-community/gpt2"
# Qwen "Qwen/Qwen2-0.5B"
# SmolLM "HuggingFaceTB/SmolLM-135M"

prompt = "It was a dark and stormy"
tokenizer =
AutoTokenizer.from_pretrained("Qwen/Qwen2-
0.5B")
input_ids = tokenizer(prompt).input_ids
input_ids
```

```
[2132, 572, 264, 6319, 323, 13458, 88]
```

```
for t in input_ids:
    print(t, "\t:", tokenizer.decode(t))
```

```
2132      : It
572       :  was
264       :  a
6319      :  dark
323       :  and
```

```
13458 : storm
88 : y
```

As shown, the tokenizer splits the input string into a series of tokens and assigns a unique ID to each. Most words are represented by a single token, but when using Qwen and GPT-2 tokenizers, "stormy" is represented by two tokens: one for "storm" (including the space before the word) and one for the suffix "y". This allows the model to learn that "stormy" is related to "storm" and that the suffix "y" is often used to turn nouns into adjectives. On the other hand, the SmolLM tokenizer does not split any of the words in this particular sentence. Each model is usually paired with its own tokenizer, so always use the proper tokenizer when using a model. The three models of this section have vocabularies that go from 50,000 to 150,000 tokens, which allows them to represent almost any input text.

NOTE

Even though we usually talk about *training* tokenizers, this has nothing to do with training a model. Model training is stochastic (non-deterministic) by nature, whereas we train a tokenizer using a statistical process that identifies which subwords are the best to pick for a given dataset. How to choose the subwords is a design decision of the tokenization algorithm. Therefore, tokenization training is deterministic. We won't dive into different tokenization strategies, but some of the most popular subword approaches are Byte-level BPE, used in GPT-2, WordPiece, and SentencePiece.

Predicting Probabilities

GPT-2, Qwen, and SmolLM were trained as *causal language models* (also known as auto-regressive), which means they were trained to predict the next token in a sequence given the preceding tokens. The *transformers* library has high-level tools that enable us to use such a model to generate text or perform other tasks quickly. It is helpful to understand how the model makes its predictions by directly inspecting them on this language-modeling task. We begin by loading the model.

```
from transformers import AutoModelForCausalLM

model =
```

```
AutoModelForCausalLM.from_pretrained("Qwen/Qwen2-0.5B")
```

Note the use of `AutoTokenizer` and `AutoModelForCausalLM`. The *transformers* library supports hundreds of models and their corresponding tokenizers. Rather than having to learn the name of each tokenizer and model class, we will use `AutoTokenizer` and `AutoModelFor*`. For the automatic model class, we need to specify for which task we're using the model, such as classification (`AutoModelForSequenceClassification`) or object detection (`AutoModelForObjectDetection`). In the case of Qwen2, we'll use the class corresponding to the causal language modeling task. When using the automatic classes, *transformers* will pick an adequate default class based on the configuration of a model. For example, under the hood, *transformers* will use `Qwen2Tokenizer` and `Qwen2ForCausalLM`.

If we feed the tokenized sentence from the previous section through the model, we get a result with 151,936 values for each token in the input string:

```
# We tokenize again but specifying the  
tokenizer that we want it to  
# return a PyTorch tensor, which is what the
```



```
model expects,  
# rather than a list of integers  
input_ids = tokenizer(prompt,  
return_tensors="pt").input_ids  
  
outputs = model(input_ids)  
outputs.logits.shape # An output for each  
input token
```

```
torch.Size([1, 7, 151936])
```

The first dimension of the output is the number of batches (1 because we just ran a single sequence through the model). The second dimension is the sequence length, or the number of tokens in the input sequence (7 in this case). The third dimension is the vocabulary size. We get a list of ~150 thousand numbers for each token in the original sequence. These are the raw model outputs, or *logits*, that correspond to the tokens in the vocabulary. For every input token, the model predicts how likely each token in the vocabulary is to continue the sequence up to that point. With our example sentence, the model will predict logits for "It ", "It was ", "It was a ", and so on. Higher logits' values mean the model considers the corresponding token a more likely continuation of the

sequence. Table 2-1 shows the input sequences, the most likely token ID, and its corresponding token.

NOTE

Logits are the raw output of the model (a list of numbers such as [0.1, 0.2, 0.01, ...]). We can use the logits to select the most likely token to continue the sequence. However, we can also convert the logits into probabilities, as we'll do soon.

Input Sequence	ID of most likely next token	Corresponding token
It	374	is
It was	264	a
It was a	2244	great
It was a dark	323	and
It was a dark and	13458	storm
It was a dark and	88	y
It was a dark and stormy	3729	(let's figure this one)

Table 2-1. The most likely token to continue input sequences according to the Qwen2 model.

Let's focus on the logits for the entire prompt and learn how to predict the next word of the sequence. We can find the index of the token with the highest value using the `argmax()` method:

```
final_logits = model(input_ids).logits[0, -1]
# The last set of logits
final_logits.argmax() # The position of the
maximum
```

```
tensor(3729)
```

3729 corresponds to the ID of the token the model considers most likely to follow the input string "It was a dark and stormy". Decoding this token, we can find out that this model knows a few story tropes:

```
tokenizer.decode(final_logits.argmax())
```

```
' night'
```

So " night" is the most likely token. This makes sense considering the beginning of the sentence we provided as input. The model learns how to *pay attention* to other tokens using a mechanism called *self-attention*, which is the fundamental building block of transformers. Intuitively, self-attention allows the model to identify how much each token contributes to the meaning of the sequence.

NOTE

Transformer models contain many of these attention layers, each one specializing in some aspect of the input. Contrary to heuristics systems, these aspects or features are learned during training, instead of being specified beforehand.

Let's now find out which other tokens were potential candidates by selecting the top 10 values with `topk()` :

```
import torch

top10_logits = torch.topk(final_logits, 10)
for index in top10_logits.indices:
    print(tokenizer.decode(index))
```

```
night
evening
```

```
day
morning
winter
afternoon
Saturday
Sunday
Friday
October
```

We need to convert logits into probabilities to better understand how confident the model is about each prediction. We can do that by comparing each value with all the other predicted values and normalizing so all the numbers sum up to 1. That's precisely what the `softmax()` operation does. The following code uses `softmax()` to print out the top 10 most likely tokens and their associated probabilities according to the model:²

```
top10 =
torch.topk(final_logits.softmax(dim=0), 10)
for value, index in zip(top10.values,
top10.indices):
    print(f"{tokenizer.decode(index):<10}
{value.item():.2%}")
```

night	88.71%
evening	4.30%
day	2.19%
morning	0.49%
winter	0.45%
afternoon	0.27%
Saturday	0.25%
Sunday	0.19%
Friday	0.17%
October	0.16%

Before going further, we suggest you to experiment with the code above. Here are some ideas for you to try:

- **Change a few words:** Try changing the adjectives (e.g., "dark" and "stormy") in the input string and find out how the model's predictions change. Is the predicted word still "night"? How do the probabilities change?
- **Change the input string:** Try different input strings and analyze how the model's predictions change. Do you agree with the model's predictions?
- **Grammar:** What happens if you provide a string that is not a grammatically correct sequence? How does the model handle it? Look at the probabilities of the top predictions.

Generating Text

Once we know how to get the model's predictions for the next token in a sequence, it is easy to generate text by repeatedly feeding the model's predictions back into itself. We can call `model(ids)`, generate a new token ID, add it to the list, and call the function again. To make it more convenient to generate multiple words, *transformers* auto-regressive models have a `generate()` method ideal for this case. Let's explore an example.

```
output_ids = model.generate(input_ids,
                             max_new_tokens=20)
decoded_text =
tokenizer.decode(output_ids[0])

print("Input IDs", input_ids[0])
print("Output IDs", output_ids)
print(f"Generated text: {decoded_text}")
```

```
Input IDs tensor([ 2132,    572,    264,   6319,
                   323, 13458,     88])
Output IDs tensor([ 2132,    572,    264,
                   6319,    323, 13458,     88,   3729,     13,
                                576, 12884,    572,   6319,    323,
                   279,   9956,    572,   1246,
```

```
2718, 13, 576, 11174, 572,
50413, 1495, 323, 279])
Generated text: It was a dark and stormy
night. The sky was dark and t
he wind was howling. The rain was pouring
down and the
```

When we ran the `model()` forward method in the previous section, it returned a list of logits for each token in the vocabulary (151936). Then, we had to calculate the probabilities and pick the most likely token. `generate()` abstracts this logic away. It makes multiple forward passes, predicts the next token repeatedly, and appends it to the input sequence. `generate()` provides us with the token IDs of the final sequence, including both the input and new tokens. Then, with the tokenizer `decode()` method, we can convert the token IDs back to text.

There are many possible strategies to perform generation. The one we just did, picking the most likely token, is called *greedy decoding*. Although this approach is straightforward, it can sometimes lead to suboptimal outcomes, especially in generating longer text sequences. Greedy decoding can be problematic because it doesn't consider the overall probability of a sentence, focusing only on the immediate next word. For instance, given the starting word `Sky` and the choices `blue`

and *rockets* for the next word, greedy decoding might favor *Sky blue* since *blue* initially appears more likely following *Sky*. However, this approach might overlook a more coherent and probable overall sequence like *Sky rockets soar*. Therefore, greedy decoding can sometimes miss out on the most likely overall sequence, leading to less optimal text generation.

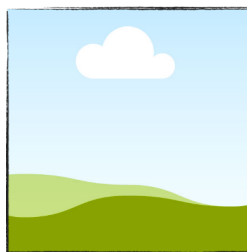


Figure 2-4. Figure 2-4. Another greedy decoding example. *The dog barks* is generated because *dog* is the most likely second token with a probability of 0.5, and *barks* is the most likely token following *The dog*

Rather than one token at a time, techniques such as *beam search* explore multiple possible continuations of the sequence and return the most likely *sequence* of continuations. It keeps

the most likely `num_beams` of hypotheses during generation and chooses the most likely one.

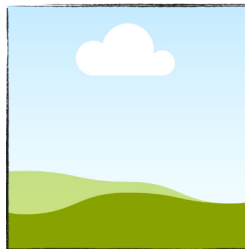


Figure 2-5. Figure 2-5. A beam search example which finds a more likely sequence. *The dog barks* has a total probability of $0.50.4=0.2$, while *The horse runs* has a probability of $0.40.9=0.36$

```
beam_output = model.generate(  
    input_ids,  
    num_beams=5,  
    max_new_tokens=30,  
)  
  
print(tokenizer.decode(beam_output[0]))
```

```
It was a dark and stormy night. The wind was  
howling, and the rain was  
pouring down. The sky was dark and gloomy,  
and the air was filled with the
```

Depending on the model, you might get somewhat repetitive results. Although not frequently used, there are multiple parameters one can control to perform less-repetitive generations. Let's consider two examples:

- `repetition_penalty` - how much to penalize already generated tokens, avoiding repetition. A good default value is 1.2.
- `bad_words_ids` - a list of tokens that should not be generated (e.g., to avoid generating offensive words).

Let's explore what we can achieve by penalizing repetition:

```
beam_output = model.generate(  
    input_ids,  
    num_beams=5,  
    repetition_penalty=2.0,  
    max_new_tokens=38,  
)
```

```
print(tokenizer.decode(beam_output[0]))
```

```
It was a dark and stormy night. The sky was  
filled with thunder and li  
ghtning, and the wind howled in the distance.  
It was raining cats and  
dogs, and the streets were covered in puddles  
of water.
```

Which generation strategy to use? As often is true in Machine Learning... it depends. Beam search works well when the desired length of the text is somewhat predictable. This is the case for tasks such as summarization or translation but not for open-ended generation, where the output length can vary greatly, leading to repetition. Although we can inhibit the model to avoid repeating itself, doing so can also lead to it performing worse. Also note that beam search will be slower than greedy search as it needs to run inference for multiple beams simultaneously, which can be an issue for large models.

When we generate with greedy search and beam search, we push the model to generate text with a distribution of high-probability next words, as can be seen in Figure 2-6³.

Interestingly, high-quality human language does not follow a

similar distribution. Human text tends to be more unpredictable. An excellent paper about this counter-intuitive observation is [The Curious Case of Neural Text Degeneration](#). The authors conjecture that human language disfavors predictable words - people optimize against stating the obvious. The paper proposes a method called nucleus *sampling*.

Before discussing nucleus sampling, let's discuss sampling. With sampling, we pick the next token by sampling from the probability distribution of the next tokens. This means that sampling is not a deterministic generation process. If the next possible tokens are night (60%), day (35%), and apple (5%), rather than choosing night (with greedy search), we will sample from the distribution. In other words, there will be a 5% chance of picking " `apple` " even if it's a low-probability token and leads to a nonsensical generation. Sampling avoids creating repetitive text, hence leading to more diverse generations. Sampling is done in *transformers* using the `do_sample` parameter.

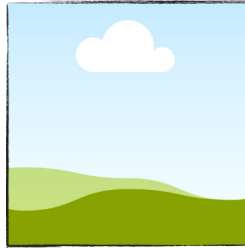


Figure 2-6. Greedy generation will always pick the most likely next token, while sampling will pick the next token by sampling from the probability distribution.

```
from transformers import set_seed

# Setting the seed ensures we get the same
results every time we run this code
set_seed(70)

sampling_output = model.generate(
    input_ids,
    do_sample=True,
    max_new_tokens=34,
    top_k=0, # We'll come back to this
    parameter
)
```

```
print(tokenizer.decode(sampling_output[0]))
```

```
It was a dark and stormy night. Kevin said he  
was going to stay up all  
    night, staring at the cloudless stars,  
wondering, what if I lost my d  
ream.He'd been teasing her about
```

We can manipulate the probability distribution before we sample from it, making it sharper or flatter using a `temperature` parameter. A temperature higher than one will increase the randomness of the distribution, which we can use to encourage generation of less probable tokens. A temperature between 0 and 1 will reduce the randomness, increasing the probability of the more likely tokens and avoiding predictions that might be too unexpected. A temperature of 0 will move all the probability to the most likely next token, which is equivalent to greedy decoding, as can be seen in Figure 2-7. Compare the effect of this temperature parameter on the generated text in the following example.

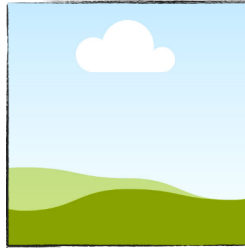


Figure 2-7. Figure 2-7. Effect of temperature on the token probability distribution.

```
sampling_output = model.generate(  
    input_ids,  
    do_sample=True,  
    temperature=0.4,  
    max_new_tokens=40,  
    top_k=0,  
)  
  
print(tokenizer.decode(sampling_output[0]))
```

It was a dark and stormy night in 1878. The
only light was the moon, a
nd the only sound was the distant roar of the

thunder. The only thing
that could be heard was the sound of the
storm

```
sampling_output = model.generate(  
    input_ids,  
    do_sample=True,  
    temperature=0.001,  
    max_new_tokens=40,  
    top_k=0,  
)  
  
print(tokenizer.decode(sampling_output[0]))
```

It was a dark and stormy night. The sky was
dark and the wind was howl
ing. The rain was pouring down and the
lightning was flashing. The sky
was dark and the wind was howling. The rain
was pouring down

```
sampling_output = model.generate(  
    input_ids,  
    do_sample=True,  
    temperature=3.0,  
    max_new_tokens=40,
```

```

        top_k=0,
    )

    print(tokenizer.decode(sampling_output[0]))

```

```

It was a dark and stormy清晨一步
        女人们都BL咻任何时候  تمس بن
attendees*sinruitmentlanuresindi ambassa
dors eventData原来是exalENCE hemisphere
worldsw.Anyar久? Sous dapat HVışıs
día.inventory emptiedfuncpping {\Sex

```

Well, the first test is much more coherent than the second one. The second, which uses a very low temperature, is repetitive (similar to greedy decoding). Finally, the third sample, with an extremely high temperature, gives gibberish text.

One parameter you likely noticed is `top_k`. What is it? *Top-K sampling* is a simple sampling approach in which only the K most likely next tokens are considered. For example, using `top_k=5`, the generation method will first filter the most likely five tokens and redistribute the probabilities so they add to one.

```

sampling_output = model.generate(
    input_ids,

```

```
do_sample=True,  
max_new_tokens=40,  
top_k=10,  
)  
  
print(tokenizer.decode(sampling_output[0]))
```

```
It was a dark and stormy night in New York.  
The city was on the brink  
of a violent storm. The sky above was painted  
with a mix of bright red  
and orange. It was a sign, but the storm had  
arrived
```

Hmm...this could be better. An issue with Top-K Sampling is that the number of relevant candidates in practice could vary greatly. If we define `top_k=5`, some distributions will still include tokens with very low probability, while others will consist of only high-probability tokens.

The final generation strategy we'll visit is *Top-p sampling* (also known as nucleus sampling). Rather than sampling the K words with the highest probability, we will use all the most likely words whose cumulative probability exceeds a given value. If we use a `top_p=0.94`, we'll first filter only to keep the most

likely words that cumulatively have a probability of 0.94 or higher, as can be observed in Figure 2-8. We then redistribute the probability and do regular sampling. Let's check this out in action.

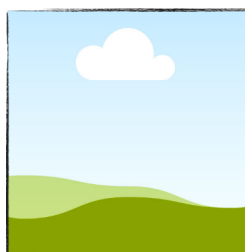


Figure 2-8. Figure 2-8. Effect of top_k and top_p on the token probability distribution.

```
sampling_output = model.generate(  
    input_ids,  
    do_sample=True,  
    max_new_tokens=40,  
    top_p=0.94,  
    top_k=0,  
)
```

```
print(tokenizer.decode(sampling_output[0]))
```

```
It was a dark and stormy night in the skies  
of Morrowind, and a partic  
ularly ruthless fighter had decided that  
these careless tourists at Ca  
rrabine should be dealt with with maximum  
cruelty. The chief of this i  
mportant operation was appointed by
```

Both Top-K and Top-p are commonly used in practice. They can even be combined to filter out low-probability words but have more generation control. The issue with the stochastic generation methods is that the generated text doesn't necessarily contain coherence.

We've explored three different generation methods: greedy search, beam-search decoding, and sampling (with temperature, Top-K, and Top-p providing further control). Those are lots of approaches! If you feel underwhelmed by the model generations, consider that it's a model with few hundred million parameters - the fact that it can generate coherent text is already impressive! New models spawn billions or even hundreds of billions of parameters and are trained with higher-

quality data, so switching to a more modern or larger model will lead to better results. In the public resources of the book, you can find an [online interactive demo](#) to visualize how `temperature`, `top_p`, and `top_k` impact a generation distribution.

If you want to experiment with generation further, here are some suggestions:

- Experiment with different parameter values. How does increasing the number of beams impact the quality of your generation? What happens if you reduce or increase your `top_p` value?
- One approach to reduce repetition in Beam Search is introducing penalties for n-grams (word sequence of n words). This can be configured using `no_repeat_ngram_size`, which avoids repeating the same n-gram. For example, if you use `no_repeat_ngram_size=4`, the generation will never contain the exact four consecutive words.
- Top-K can lead to discarding high-quality tokens and Top-p can lead to including some low-probability tokens. For a more dynamic approach, you can use Min-P (`min_p`), which multiplies `min_p` times the top token's probability, and then only include tokens above that percentage. In other

words, Min-P defines a dynamic threshold based on the top token's probability

- A newer method, contrastive search, can generate long, coherent output while avoiding repetition. This is achieved by considering both the probabilities predicted by the model and the similarity with the context. This can be controlled via `penalty_alpha` and `top_k`.⁴

If all of this sounds too empirical, it's because it is. Generation is an active area of research, with new papers coming up with different proposals, such as more sophisticated filtering. We'll briefly discuss these in the final chapter. No single rule works for all models, so it's always important to experiment with different techniques.

Zero-Shot Generalization

Generating language is a fun and exciting application of transformers, but writing fake articles about unicorns⁵ is not the reason why they are so popular. To predict the next token well, these models must *learn* a fair amount about the world. We can take advantage of this to perform various tasks. For example, instead of training a model dedicated to translation, we can prompt a sufficiently powerful language model with an input like:

```
Translate the following sentence from English  
to French:
```

```
Input: The cat sat on the mat.
```

```
Translation:
```

I typed this example with GitHub Copilot active, and it helpfully suggested "`Le chat était assis sur le tapis`" as a continuation of the above prompt - a perfect illustration of how a language model can perform tasks not explicitly trained for. The more powerful the model, the more tasks it can perform without additional training. This flexibility makes transformers quite powerful and has made them so popular in recent years.

To check this in action for ourselves, let's use Qwen as a classification model. Specifically, we'll classify movie reviews as positive or negative - a classic benchmark task in the NLP field. We'll use a *zero-shot* approach to make things interesting, which means we won't provide the model with any labeled data. Instead, we'll prompt the model with the text of a review and ask it to predict the sentiment.

There are multiple ways we can use a generative model as a classifier. Usually, we begin by inserting the movie review into a prompt template that provides context for the model. This prompt template could instruct the model to simply return the

sentiment of the review (and ask to limit as `positive` or `negative`). An alternative trick, specially needed with a small model like GPT-2 or the small Qwen, is to look at its prediction for the next token and find out which possible token is assigned a higher probability: `positive` or `negative`? Let's go with this approach and find the IDs corresponding to those tokens.

```
# Check the token IDs for the words '
positive' and ' negative'
# (note the space before the words)
tokenizer.encode(" positive"),
tokenizer.encode(" negative")
```

```
([6785], [8225])
```

Once we have the IDs, we can now run inference with the model and generate a label based on the probabilities:

```
def score(review):
    """Predict whether it is positive or
    negative

    This function predicts whether a review
    is positive or negative
    using a bit of clever prompting. It looks
```

```

at the logits for the
    tokens ' positive' and ' negative', and
returns the label
    with the highest score.
    """
    prompt = f"""Question: Is the following
review positive or
negative about the movie?
Review: {review} Answer: """

    input_ids = tokenizer(prompt,
return_tensors="pt").input_ids ❶
    final_logits = model(input_ids).logits[0,
-1] ❷
    if final_logits[6785] >
final_logits[8225]: ❸
        print("Positive")
    else:
        print("Negative")

```

❶ Tokenize the prompt

❷ Get the logits for each token in the vocabulary. Note that we're using `model()` rather than `model.generate()`, as `model()` returns the logits for each token in the vocabulary, while `model.generate()` returns only the chosen token.

- 3 Check if the logit for the `positive` token is higher than the logit for the `negative` token.

We can try out this zero-shot classifier on a few fake reviews to evaluate how it does:

```
score("This movie was terrible!")
```

Negative

```
score("That movie was great!")
```

Positive

```
score("A complex yet wonderful film about the  
depravity of man") # A mistake
```

Negative

In the [GitHub repo for this book](#), you'll find a dataset of labeled reviews and code to assess the accuracy of this zero-shot approach. Can you tweak the prompt template to improve the

model's performance? Can you think of other tasks that could be performed using a similar approach?

The zero-shot capabilities of recent models have been a game-changer. As the models improve, they can perform more tasks out-of-the-box, making them more accessible and easier to use and reducing the need for specialized models for each task.

Few-Shot Generalization

Despite the release of ChatGPT and the quest for the perfect prompts, zero-shot generalization (or prompt tuning) is not the only way to bend powerful language models to perform arbitrary tasks.

Zero-shot is the extreme application of a technique called *few-shot generalization*, in which we provide the language model a few examples about the task we want it to perform and then ask it to provide similar answers for us. Instead of training the model, we show some examples to influence generation by increasing the probability that the continuation text follows the same structure and pattern as our prompt. Let's try an example. Apart from providing examples, providing a short description of what the model should do, e.g., "Translate English to French", will help with higher-quality generations.

```
prompt = """\n
Translate English to Spanish:

English: I do not speak Spanish.
Spanish: No hablo español.

English: See you later!
Spanish: ¡Hasta luego!

English: Where is a good restaurant?
Spanish: ¿Dónde hay un buen restaurante?

English: What rooms do you have available?
Spanish: ¿Qué habitaciones tiene disponibles?

English: I like soccer
Spanish:"""
inputs = tokenizer(prompt,
return_tensors="pt").input_ids
output = model.generate(
    inputs,
    max_new_tokens=10,
)

print(tokenizer.decode(output[0]))
```

Translate English to Spanish:

English: I do not speak Spanish.

Spanish: No hablo español.

English: See you later!

Spanish: ¡Hasta luego!

English: Where is a good restaurant?

Spanish: ¿Dónde hay un buen restaurante?

English: What rooms do you have available?

Spanish: ¿Qué habitaciones tiene disponibles?

English: I like soccer

Spanish: Me gusta el fútbol

English:

We state the task we want to achieve and provide four examples to set the context for the model. Hence, this is a 4-shot generalization task. Then, we ask the model to generate more text to follow the pattern and provide the requested translation. Some ideas to explore:

- Would this work with fewer examples?
- Would it work without the task description?

- How about other tasks?
- How does GPT-2 and SmolLM score in this setting?

NOTE

GPT-2, given its size and training process, is not very good at few-shot tasks, and it's even worse at zero-shot generalization. How is it possible you could have used it for sentiment classification as in the previous section? We cheated a bit: we didn't look at the text generated by the model, but just checked whether the probability for "positive" was larger than the "negative" probability. Understanding how models work under the hood can unlock powerful applications even with small models. Remember to think about your problem; don't be afraid to explore.

GPT-2 and Qwen 0.5B are examples of *base models*. Some base models in the style of Qwen have zero-shot and few-shot capabilities that we can use at *inference* time. Another approach is to **fine-tune** a model: we take the base model and keep training it a bit longer on domain or task-specific data. We don't always need the extreme generalization capabilities showcased by the most powerful models in the world; if you only want to solve a particular task, it will usually be cheaper and better to fine-tune and deploy a smaller model specialized on a single task.

It's also important to note that **base models are not conversational**; although you can write a very nice prompt

that will help make a chatbot with a base model, it's often more convenient to fine-tune the base model with conversational data, hence improving the conversational capabilities of the model. That's precisely what we'll do in Chapter 6. Recent LLM releases tend to include both a base model and an official model with conversational capabilities. In the case of the small Qwen model, this would be [Qwen2-0.5B-Instruct](#).

A Transformer Block

After our brief experiments using language models, we are ready to introduce an architecture diagram for transformer-based language generation models shown in Figure 2-9.

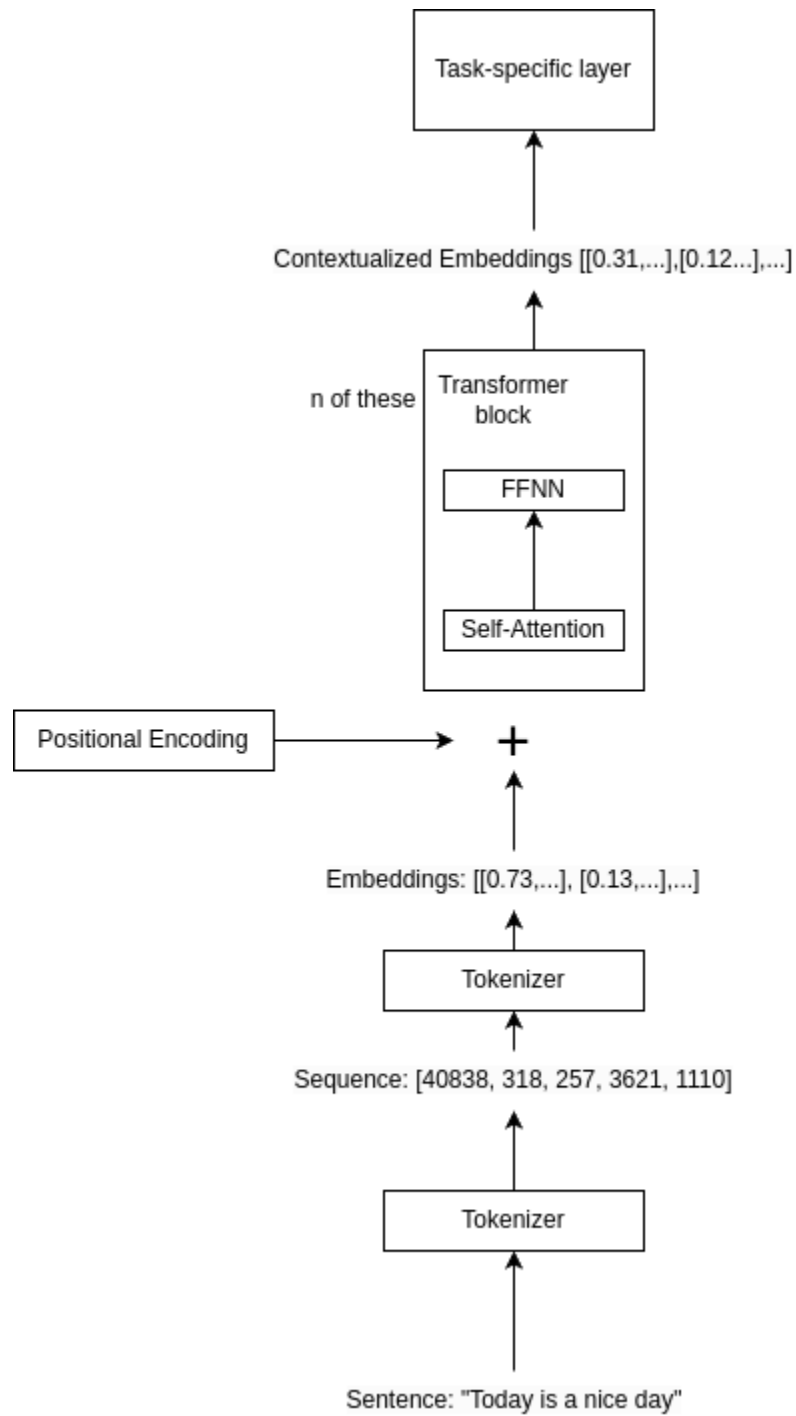


Figure 2-9. Architecture of a transformer-based language model.

The high-level pieces involved include:

- **Tokenization.** The input text is broken down into individual tokens. Each token has a corresponding ID used to index the token embeddings.
- **Input Token Embedding.** The tokens are represented as vectors called *embeddings*. These embeddings serve as numerical representations that capture basic information of each token. You can think of vectors as (a long) list of numbers, where each number corresponds to a particular aspect of the token's meaning. During training, a model learns how to map each token to its corresponding embedding. The token embedding will always be the same for each token, regardless of its position in the input sequence.
- **Positional Encoding.** The transformer model has no notion of order, so we need to enrich the token embeddings with positional information. This is done by adding a positional encoding to the token embeddings. The positional encoding is a set of vectors that encode the position of each token in the input sequence. This allows the model to differentiate between tokens based on their position in the sequence, which can be useful as the same token appearing in different places can have different meanings.
- **Transformer blocks:** The core of the transformer model is the transformer block. The power of transformers comes

from stacking multiple blocks, allowing the model to learn increasingly complex and abstract relationships between the input tokens. It consists of two main components:

- **Self-Attention Mechanism.** This mechanism allows the model to weigh the importance of each token in the context of the entire sequence. It helps the model understand the relationships between different tokens in the input. The self-attention mechanism is the key to the transformer's ability to handle long-range dependencies and complex relationships between words, and it helps generate coherent and contextually appropriate text.
- **Feed-Forward Neural Network.** The self-attention output is passed through a feed-forward neural network, which further refines the representation of the input sequence.
- **Contextual Embeddings.** The output of the transformer block is a set of contextual embeddings that capture the relationships between tokens in the input sequence. Unlike the input embeddings, which are fixed for each token, the contextual embeddings are updated at each layer of the transformer model based on the relationships between tokens. The embeddings capture rich and complex

semantic information about the token in the context in which it appears

- **Prediction.** An additional layer processes the final representation into a task-dependent final output. In the case of text generation, this involves having a linear layer that maps the contextual embeddings to the vocabulary space, followed by a softmax operation to predict the next token in the sequence.

Of course, this is a simplification of the transformer architecture. Diving into the internals of how self-attention works or the internals of the transformer block is beyond the scope of this book. However, understanding the high-level architecture of a transformer model can be helpful to grasp how these models work and how they can be applied to various tasks. This architecture has enabled transformers to achieve unprecedented performance in various tasks and domains, and you'll find them cropping up again and again –not only in the rest of this book, but also in the discipline as a whole.

Transformer Models Genealogy

At the beginning of the chapter, we experimented with Qwen to auto-regressively generate text. Qwen, an example of a decoder-

based transformer, has a single stack of transformer blocks that process an input sequence. This is a popular approach today, but there are other architectures that have been developed over the years. This section will provide a brief overview of the genealogy of transformer models.

Sequence-to-Sequence Tasks

The original transformer paper, *Attention is all you need*,⁶ shown in Figure 2-10, used a seemingly more complicated architecture called the encoder-decoder architecture. Although the encoder-decoder architecture was very popular until 2023, they have been superseded by decoders in most research labs

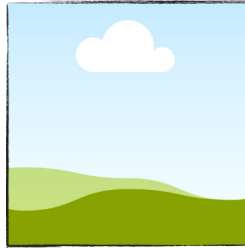


Figure 2-10. Figure 2-10. A diagram of the encoder-decoder transformer (inspired on <https://arxiv.org/abs/1706.03762>)

The transformer paper focused on machine translation as the example sequence-to-sequence task. The best results in machine translation at the time were achieved by RNNs, such as LSTM and GRU (don't worry if you're unfamiliar with them). The paper demonstrated better results by focusing solely on the *attention* method and showed that scalability and training were much easier. These factors –excellent performance, stable training, and easy scalability– are why transformers took off and were adapted to multiple tasks, as the next section explores in more depth.

In encoder-decoder models, like the original transformer model described in the paper, one stack of transformer blocks, called *encoder*, processes an input sequence into a set of rich representations, which are then fed into another stack of transformer blocks, called decoder, that decodes them into an output sequence. This approach to convert one sequence into a different one is called sequence-to-sequence or seq2seq and is naturally well suited for tasks such as translation, summarization, or question-answering.

For example, you feed an English sentence through the encoder of a translation model, which generates a rich embedding that captures the meaning of the input. Then, the decoder generates the corresponding French sentence using this embedding. The generation happens in the decoder one token at a time, as we saw when generating sequences earlier in the chapter. However, the predictions for each successive token are informed not just by the previous tokens in the sequence being generated but also by the output from the encoder.

The mechanism by which the output from the encoder side is incorporated into the decoder stack is called *cross-attention*. It resembles self-attention, except that each token in the input (the sequence being processed by the decoder) attends to the context from the encoder rather than other tokens in its

sequence. The cross-attention layers are interleaved with self-attention, allowing the decoder to use both contexts within its sequence and the information from the encoder.

After the transformer paper, existing sequence-to-sequence models, such as Marian NMT, incorporated these techniques as a central part of their architecture. New models were developed using these ideas. A notable one is BART (short for Bidirectional and Auto-Regressive Transformers⁷). During pre-training, BART corrupts input sequences and attempts to reconstruct them in the decoder output. Afterward, BART is fine-tuned for other generation tasks, such as translation or summarization, leveraging the rich sequence representations achieved during pre-training. Input corruption, by the way, is one of the key ideas behind diffusion models, as we'll discuss in Chapter 4.

Encoder-only Models

As we've discussed, the original transformer model was based on an encoder-decoder architecture that has been further explored in models such as BART or T5. In addition, the encoder or the decoder can be trained and used independently, giving rise to distinct transformer families. The first sections of this chapter explored decoder-only, or autoregressive models. These models are specialized in text generation using the techniques

we described and have shown impressive performance, as demonstrated by ChatGPT, Claude, Llama, or Gemma.

Encoder models, on the other hand, are specialized in obtaining rich representations from text sequences and can be used for tasks such as classification or to prepare semantic *embeddings* for a multitude of documents that can be used in retrieval systems. The best-known transformer encoder model is probably BERT⁸, which introduced the masked language model objective that was later picked up and further explored by BART.

Causal language modeling predicts the next token given the previous ones - it's what we did with Qwen. The model can only attend to the context on the left of a given token. A different approach used in encoder models is called *masked language modeling* (MLM). Masked language modeling, proposed in the famous BERT paper, pre-trains a model to learn to "fill in the blanks". Given an input text, we randomly mask some tokens, and the model must predict the hidden tokens, as shown in Figure 2-11. Unlike causal language modeling, MLM uses both the sequence at the masked token's left and right, hence the B of bidirectional in BERT's name. This helps create strong representations of the given text. Under the hood, these models use the encoder part of the transformer's architecture.

We just discussed encoder-decoder and decoder-only architectures. A common question is why one might need an encoder-decoder model for tasks like translation if decoder-only models like Qwen and Llama can show good results. Encoder-decoder models are designed to translate an entire input sequence to an output sequence, making them well-suited for translation. In contrast, decoder-only models focus on predicting the next token in a sequence. Initially, decoder-only models like GPT-2 were less capable in zero-shot learning scenarios than more recent models like GPT-4, but this was due to more than just the absence of an encoder. The improvement in zero-shot capabilities in advanced models like GPT-4 is also due to larger training data, better training techniques, and increased model sizes. While encoders in seq2seq models play a crucial role in understanding the full context of input sequences, advancements in decoder-only models have made them more effective and versatile, even for tasks traditionally relying on seq2seq models.

Let's look at some code. Rather than using the `AutoModel` and `AutoTokenizer` classes we used before, let's introduce a higher-level *transformers* API called `pipeline`. This API allows you to easily load a model for a given task. `pipeline` takes care of all the pre- and post-processing, and hence, it's a great way to quickly try out models.

```
from transformers import pipeline

fill_masker = pipeline("fill-mask",
model="bert-base-uncased")
fill_masker("The [MASK] is made of milk.")
```

```
[{'score': 0.19546695053577423,
  'token': 9841,
  'token_str': 'dish',
  'sequence': 'the dish is made of milk.'},
{'score': 0.1290755718946457,
  'token': 8808,
  'token_str': 'cheese',
  'sequence': 'the cheese is made of milk.'},
{'score': 0.10590697824954987,
  'token': 6501,
  'token_str': 'milk',
  'sequence': 'the milk is made of milk.'},
{'score': 0.04112089052796364,
  'token': 4392,
  'token_str': 'drink',
  'sequence': 'the drink is made of milk.'},
{'score': 0.03712352365255356,
  'token': 7852,
  'token_str': 'bread',
  'sequence': 'the bread is made of milk.'}]
```

Good to know that the milk is made of milk! What happens under the hood? The encoder receives the input sequence and generates a contextualized representation for each token. This representation is a vector of numbers that captures the meaning of the token in the context of the entire sequence. The encoder is usually followed by a task-specific layer that uses the representations to perform tasks such as classification, question answering, or masked language modeling, as shown in Figure 2-11. The encoder is trained to generate representations that are useful for understanding-heavy tasks.

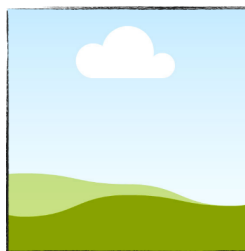


Figure 2-11. Figure 2-11. Encoder models output semantic embeddings that can be used to solve tasks such as predicting a token in the middle of a sequence.

Between encoder-only, decoder-only, and encoder-decoder models, companies and research labs have released a large number of new open and closed language models, such as GPT-4, Mistral, Falcon, Llama, Qwen, Yi, Claude, Bloom, Gemma, and hundreds more. Yann LeCun [posted](#) this delightful genealogy diagram in Twitter, taken from a survey paper⁹ shows transformers' rich and fruitful impact on the NLP landscape as of 2024.

The Power of Pre-training

Having access to existing models is quite powerful. Transformer models have shown state-of-the-art performance across many other language tasks, such as text classification, machine translation, and answering questions based on an input text. Why do transformers work so well?

The first insight is the usage of the attention mechanism, as hinted across the chapter. Attention mechanisms allow the transformers model to attend to long sequences and learn long-range relationships. In other words, transformers can estimate how relevant some tokens are to other tokens.

The second key aspect is their ability to scale. The transformer architecture has an implementation optimized for parallelization, and research has shown that these models can scale to handle high-complexity and high-scale datasets. Although initially designed for text data, the transformer architecture can be flexible enough to support different data types and handle irregular inputs.

The third key insight is the ability to do *pre-training* and *fine-tuning*. Traditional approaches to a task, such as movie review classification, were limited by the availability of labeled data. A model would be trained from scratch on a large corpus of labeled examples, attempting to predict the label from the input text directly. This approach is often referred to as supervised learning. However, it has a significant drawback: it requires a large amount of labeled data to train effectively. This is a problem because labeled data is expensive to obtain and time-consuming to label. There might not even be any available data in many domains.

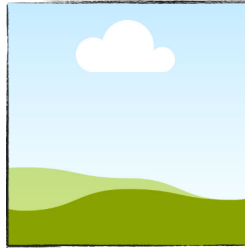


Figure 2-12. Figure 2-12. Popular model releases

To address this, researchers began looking for a way to pre-train models on existing data that could then be fine-tuned (or adjusted) for a specific task. This approach is known as *transfer learning* and is the foundation of modern ML in many fields, such as Natural Language Processing and Computer Vision. Initial works in NLP focused on finding domain-specific corpora for the language model pre-training phase, but papers such as ULMFiT¹⁰ showed that even pre-training on generic text such as Wikipedia could yield impressive results when the models were fine-tuned on downstream tasks, such as sentiment analysis or question answering. This set the stage for the rise of

transformers, which turned out to be highly well-suited to learning rich representations of language.

The idea of pre-training is to train a model on a large unlabeled dataset and then fine-tune it to a new target task, for which one would require much less labeled data. Before graduating to NLP, transfer learning had already been very successful with the Convolutional Neural Networks that form the backbone of modern Computer Vision. In this scenario, one first trains a large model with a massive amount of labeled images in a classification task. Through this process, the model learns common features that can be leveraged on a different but related problem. For example, we can pre-train a model on thousands of classes and then fine-tune it to classify whether a picture is of a hot dog.

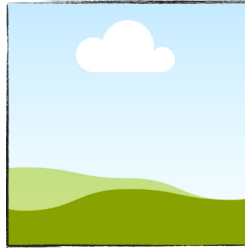


Figure 2-13. Figure 2-13. While pre-training a base model can require a significant amount of resources, fine-tuning an existing model on a new task or domain is significantly cheaper.

With transformers, things are taken further with self-supervised pre-training (training with unlabeled data). We can pre-train a model on large, unlabeled text data. How? Let's think about causal models such as GPT. The model predicts which is the next token. Well, we don't need any labels to obtain training data! Given a corpus of text, we can mask the tokens after a sequence and train the model to learn to predict them. Like in the Computer Vision case, pre-training gives the model a meaningful representation of the underlying text. We can then fine-tune the model to perform another task, such as generating text in the style of our Tweets or a specific domain (e.g., your

company chat). Given the model has already learned a representation of language, fine-tuning will require much less data than if we trained from scratch.

For many tasks, a rich representation of the input is more important than being able to predict the next token. For example, if you want to fine-tune a model to predict the sentiment of a movie review, masked language models would be more powerful. Models such as GPT-2 are designed to optimize for text generation rather than for building powerful representations of the text. On the other hand, models such as BERT are ideal for this task. As briefly mentioned before, the last layer of an encoder model outputs a dense representation of the input sequence, called embedding. This embedding can then be leveraged by adding a small, simple network on top of the encoder and fine-tuning the model for the specific task. As a concrete example, we can add a simple linear layer on top of the BERT encoder output to predict the sentiment of a document. We can take this approach to tackle a wide range of tasks:

- **Token classification.** Identify each entity in a sentence, such as a person, location, or organization.
- **Extractive question answering.** Given a paragraph, answer a specific question and extract the answer from the

input.

- **Semantic search.** The features generated by the encoder can be handy to build a search system. Given a database of a hundred documents, we can compute the embeddings for each. Then, we can compare the input embeddings with the documents' ones at inference time, hence identifying the most similar document in the database.¹¹
- And many others, including text similarity, anomaly detection, named entity linking, recommendation systems, and document classification.

Let's use a BERT-based model that was fine-tuned to perform sequence classification to determine if the sentiment of a text is positive or negative. Once again, we'll use the `pipeline` API to load the model and perform the classification.

```
from transformers import pipeline

classifier = pipeline(
    "text-classification",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
)
classifier("This movie is disgustingly good!")
```

```
[{'label': 'POSITIVE', 'score':  
0.9998536109924316}]
```

This classification model can analyze reviews and do the same as in the zero-shot classification section. The challenge section of this chapter shows how to evaluate classification models and compare between a zero-shot setup and this fine-tuned model.

Transformers Recap

We've discussed three types of architectures.

- **Encoder-based architectures** such as BERT, DistilBERT, and RoBERTa¹², are ideal for tasks that require understanding the entire input. These models output contextualized embeddings that capture the meaning of the input sequence. We can then add a small network on top of these embeddings and train it for a new specific task that relies on the semantic information.
- **Decoder-based architectures**, such as GPT-2, Qwen, Gemma, and Llama, are ideal for new text generation.
- **Encoder-decoder architectures**, or seq2seq, such as BART and T5, are great for tasks that require generating new

sentences based on a given input, such as summarization or translation.

"Wait, " you might say, " I can handle all these tasks with ChatGPT or Llama. " That's true. Given the vast and growing amount of training data, computing power, and training optimizations, the quality of generative models has significantly improved. Their zero-shot capabilities have come a long way compared to a few years ago.

There are two main schools of thought here. One perspective is that, given the resources, fine-tuning a model for your specific task and domain will yield better results than using a generalist pre-trained model. For example, if you want to use a GPT-like model to generate character dialogues in real-time within a game, fine-tuning it with similar data beforehand might improve performance. Similarly, if you need a model to extract entities from a dataset of chemistry papers, it might make sense to fine-tune an encoder-based model with relevant chemistry texts.

On the other hand, with the rise of high-quality, low-cost generalist models that perform well across a variety of tasks, some argue that fine-tuning might be unnecessary for most use

cases. Instead, prompt engineering could be a more effective and cheaper approach.

Seq2seq models were initially successful because they can encode variable-length input sequences into embeddings that summarize the input information. The decoder then uses this context to generate output. Recently, decoder-only models have gained popularity due to their simplicity, scalability, efficiency, and parallelization. In practice, different types of models are employed depending on the task—there's no single "golden" model that works for everything.

With over half a million open models, you might wonder which one to use. Chapter 6 will help you navigate this landscape, providing guidelines on how to choose the right model for your task and requirements as well as how to fine-tune a model for your specific needs.

Limitations

At this point, you might wonder what the issues are with transformers. Let's briefly go over some of the limitations:

- **Transformers are very large.** Research has consistently shown that larger models perform better. Although that's quite exciting, it also brings concerns. First, some of the

most powerful models require tens of millions of U.S. dollars to train - just in computing power. That means that only a small set of institutions can train very large base models, limiting the kind of research that institutions without those resources can do. Second, using such amounts of computing power can also have ecological implications - those millions of GPU hours are, of course, powered by lots of electricity. Third, even if some of these models are open-sourced, running them might require many GPUs. Chapter 6 will explore some techniques to use these LLMs even if you don't have multiple GPUs at home. Even then, deploying them in resource-constrained environments is a frequent challenge.

- **Sequential processing:** If you recall the decoder section, we had to process all the previous tokens for each new token. That means generating the 10,000th token in a sequence will take considerably longer than generating the initial one.¹³ In computer science terms, transformers have quadratic time complexity with respect to the input length. This means that as the length of the input increases, the time taken for processing grows quadratically, making it challenging to scale them to very long documents or use these models in some real-time scenarios. While transformers excel in many tasks, their computational

demands require careful consideration and optimization when being used in production. That said, there has been a lot of research on making transformers more efficient for extremely long sequences, such as clever caching techniques, ring attention, and Infini-attention.

- **Fixed input size:** Transformer models can handle a maximum number of tokens, which depends on the base model. The number of tokens the model can attend is called the context window and is an essential thing to look into when picking a pre-trained model. You cannot simply pass entire encyclopedias to transformers, expecting they will be able to summarize them, but this is changing quickly. While some pre-trained models can only handle up to 512 tokens, having models that can handle up to 32 thousand tokens is more common nowadays. New techniques allow to scale to hundreds of thousands or even millions of tokens! For example, Llama 3.1 can handle 131 thousand tokens, which is close to the length of this book! This is an essential thing to look into when picking a pre-trained model.
- **Limited interpretability:** Transformers are often criticized for their lack of interpretability. ¹⁴.

All of the above are very active research areas - people have been exploring how to train and run models with less computing power (e.g., QLoRA, which we'll explore in Chapter

6), make generation faster (e.g., flash attention and assisted generation), enable unconstrained input sizes (e.g., RoPE and attention sinks), and interpret the attention mechanisms.

One big concern is the presence of biases in models. If the training data used to pre-train transformers contains biases, the model can learn and perpetuate them. This is a broader issue in Machine Learning but is specially relevant to transformers.

Let's revisit the `fill-mask` pipeline. Let's say we want to predict the most likely profession. As you can check out below, the results are very different if we use the word "man" vs. "woman".

```
unmasker = pipeline("fill-mask", model="bert-  
base-uncased")  
result = unmasker("This man works as a [MASK]  
during summer.")  
print([r["token_str"] for r in result])  
  
result = unmasker("This woman works as a  
[MASK] during summer.")  
print([r["token_str"] for r in result])
```

```
['farmer', 'carpenter', 'gardener',  
'fisherman', 'miner']
```

```
['maid', 'nurse', 'servant', 'waitress',  
'cook']
```

Why does this happen? To enable pre-training, researchers usually require large amounts of data, leading to scraping all the content they can find. This content might be of all kinds of quality, including toxic content (which can be, to some extent, filtered out). The base model might end up engraining and perpetuating these biases when being fine-tuned. Similar concerns exist for conversational models, where the final model might generate toxic content learned from the pre-training dataset.

Beyond Text

Transformers have been used for many tasks representing data as text. A clear example is code generation – rather than training a language model with English data, we can use lots of code, and, by the same principles we just learned, it will learn how to auto-complete code. Another example is using transformers to answer questions from a table, such as a spreadsheet.

As transformer models have been so successful in the text domain, considerable interest has sparked in other

communities to adapt these techniques to other modalities. This has led to transformer models being used for tasks such as image recognition, segmentation, object detection, video understanding, and more.

CNN have been widely used as the go-to state-of-the-art models for most Computer Vision techniques. With the introduction of Vision Transformers (ViT)¹⁵, there has been a switch in recent years to explore how to tackle vision tasks with attention and transformers-based techniques. ViTs don't discard CNNs entirely: In the image processing pipeline, CNNs extract feature maps of the image to detect high-level edges, textures, and other patterns. The feature maps obtained from the CNNs are then divided into fixed-size, non-overlapping patches. These patches can be treated similarly to a sequence of tokens, so the attention mechanism can learn the relationships between patches in different places.

Unfortunately, ViTs required more data (300 million images!) and compute than CNNs to get good results. Further work has happened in recent years; for example, DeiT was able to use transformer-based models with mid-sized datasets (1.2M images) thanks to using augmentation and regularization techniques common in CNNs. Other models such as DETR, SegFormer, and Swin Transformer have pushed the field

further, supporting many tasks such as image classification, object detection, image segmentation, video classification, document understanding, image restoration, super-resolution, and others.

A powerful example of transformer-based image models is zero-shot image classification. Unlike traditional image classifiers that are trained on a fixed set of classes, zero-shot image classification allows the specification of classes at inference time. This provides the flexibility to use a single model for various image classification tasks, even those it was not explicitly trained for. To demonstrate, let's start by loading an image using the PIL library, a widely-used tool for vision-related pre-processing.

```
import requests
from PIL import Image

from genaibook.core import SampleURL

# Download an image and load it with the PIL
library
url = SampleURL.CatExample
image = Image.open(requests.get(url,
stream=True).raw)
image
```



Figure 2-14. Figure 2-14. A cat picture we'll pass to a zero-shot image classification pipeline.

Now, let's use the high-level `pipeline` to use a model for the given task.

```
pipe = pipeline(  
    "zero-shot-image-classification",  
    model="openai/clip-vit-base-patch32"  
) ❶  
labels = ["cat", "dog", "zebra"] ❷
```

```
pipe(image, candidate_labels=labels) ❸  
image
```

- ❶ Load the [openai/clip-vit-base-patch32](#) model.
- ❷ Define the classes we want to use at inference time.
- ❸ Pass the image and labels through the pipeline to get the model predictions.

```
[{'score': 0.9936687350273132, 'label':  
'cat'},  
 {'score': 0.006043245084583759, 'label':  
'dog'},  
 {'score': 0.0002880473621189594, 'label':  
'zebra'}]
```

As we'll explore in Chapter 9, transformer models can also be used for audio tasks, such as transcribing audio or generating synthetic speech or music. Under the hood, the same fundamental principles of pre-training and attention mechanisms persist, but each modality has different data types, requiring different approaches and modifications.

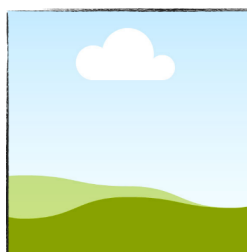


Figure 2-15. Transformers models can be used for different tasks, such as classifying images, detecting objects, and segmenting images.

Other modalities where transformers are being explored are:

- **Graphs:** An excellent introductory read is [Introduction to Graph Machine Learning by Fourrier](#). Using transformers for graphs is still very exploratory, but there are some exciting early results. Some examples of tasks that involve graph data are predicting the toxicity of molecules, predicting the evolution of systems, or generating new plausible molecules.
- **3D data:** For example, perform segmentation of data that can be represented in 3D, such as LiDAR point clouds in autonomous driving or CT scans for organ segmentation.

Another example is estimating an object's 6 degrees of freedom, which can be helpful in robotics applications.

- **Time series:** Analyzing stock price or performing weather forecasting.
- **Multimodal:** Some transformer models are designed to process or output multiple types of data (such as text, images, and audio) together. This opens new possibilities, such as multimodal systems where you can speak, write, or provide pictures and have a single model to process them. Another example is visual question answering, where a model can answer questions about provided images.

Project Time: Using LMs to Generate Text

We used the `generate()` method in the generation section to perform different decoding techniques. To better understand how it works under the hood, it's time to implement it ourselves. We'll use the `generate()` method as a reference but implement it from scratch. We'll also explore using the `generate()` method to perform different decoding techniques.

Your goal is to fill the code in the following function. Rather than use `model.generate()`, the idea is to iteratively call `model()`, passing the previous tokens as input. You have to implement greedy search when `do_sample=False`, sampling when `do_sample=True`, and Top-K sampling when `do_sample=True` and `top_k` is not `None`. This will be a challenging task, so do not worry if you don't come up with a solution quickly. We suggest you begin implementing greedy search and then build on top of it.

```
def generate(  
    model, tokenizer, input_ids,  
    max_length=50, do_sample=False, top_k=None  
):  
    """Generate a sequence without using  
    model.generate()  
  
    Args:  
        model: The model to use for  
        generation.  
        tokenizer: The tokenizer to use for  
        generation.  
        input_ids: The input IDs  
        max_length: The maximum length of the  
        sequence.  
        do_sample: Whether to use sampling.  
        top_k: The number of tokens to sample
```

```
from.  
    """  
    # Write your code here  
    # Begin by the simplest approach, greedy  
decoding.  
    # Then add sampling and finally top-k  
sampling.
```

Summary

You now have learned to load and use transformers for various tasks. This chapter also covered how transformer models sequence data such as text and how this property lets them learn valuable representations that we can use to generate or classify new sequences. As the scale of these models increases, so do their capabilities - to the point where massive models with hundreds of billions of parameters can now perform many tasks previously thought impossible for computers.

We can pick powerful existing pre-trained models and modify them for specific domains and use cases thanks to fine-tuning. The trend towards larger and more capable models has caused a shift in how people use them. Task-specific models are often out-competed by general-purpose LLMs, and most people now interact with these models via APIs, hosted solutions, local

deployments or directly via slick chat-based user interfaces. At the same time, thanks to the release of large and powerful open-access models, such as Llama, there is a strong wave in the researchers' and practitioners' ecosystems aiming to run high-quality models directly in consumer computers, resulting in privacy-first solutions. This trend extends beyond inference: novel training approaches that allow individuals to fine-tune these models without many computational resources have emerged in recent years. Chapter 6 explores this further and dives into both traditional and novel fine-tuning techniques.

Although we covered how transformers work and we'll dive into their training, diving into the internals of these models (for example, the math behind attention mechanisms) or how to pre-train a model from scratch is outside the scope of this book. Luckily for us, there are excellent resources to learn about this:

- [The Illustrated Transformer](#) by Jay Alammar is a beautiful visual guide that explains transformers in a detailed and intuitive way.
- We recommend reading the [Natural Language Processing with Transformers book](#) if you want to dive deeper into the internals of fine-tuning these models for multiple specific tasks.

- Hugging Face has a [free, open-source course](#) which teaches how to solve different NLP tasks.

If you want to dive more into the GPT family of models, we suggest to review the following papers:

- **Improving Language Understanding by Generative Pre-Training.** This is the original GPT paper, published in 2018 by Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. It introduced the idea of using a transformer-based model pre-trained on a large corpus of text to learn general language representations and then fine-tuning it on specific downstream tasks. The paper also showed that the GPT model achieved state-of-the-art results on several natural language understanding benchmarks at the time.
- **Language Models are Unsupervised Multitask Learners,** published in 2019 by Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. It presented GPT-2, a transformer-based model with 1.5 billion parameters pre-trained on a large corpus of web text called WebText. The paper also demonstrated that GPT-2 could perform well on various natural language tasks without fine-tuning, such as text generation, summarization, translation, reading comprehension, and

commonsense reasoning. Finally, it discussed large-scale language models' potential ethical and social implications.

- **Language Models are Few-Shot Learners**, published in 2020 by Tom B. Brown and others. This paper shows that scaling up language models dramatically improves their ability to perform new language tasks from only a few examples or simple instructions without fine-tuning or gradient updates. The paper also presents GPT-3, an autoregressive language model with 175 billion parameters, which achieves strong performance on many NLP datasets and tasks.

Exercises

1. What's the role of the attention mechanism in text generation?
2. In which cases would a character-based tokenizer be preferred?
3. What happens if you use a tokenizer different from the one used with the model?
4. What's the risk of using `no_repeat_ngram_size` when doing generation? (hint: think of city names)
5. What would happen if you combine Beam-search and sampling?

6. Imagine you're using a LLM that generates code in a code editor by doing sampling. What would be more convenient? A low temperature or a high temperature?
7. What's the importance of fine-tuning, and why is it different than zero-shot generation?
8. Explain the difference and application of encoder, decoder, and encoder-decoder transformers.

You can find the solutions to these exercises in the [GitHub repository of the book](#).

Challenges

9. Summarization. Use a summarization model (you can do `pipeline(" summarization ")`) to generate summaries of a paragraph. How does it compare with the results of using zero-shot? Can it be beaten by providing few-shot examples?
10. Sentiment Analysis. In the zero-shot supplementary material, we calculate some metrics using zero-shot classification. Explore using the `distilbert-base-uncased-finetuned-sst-2-english` encoder model that can do sentiment analysis. What results do you get?

11. Semantic Search. Let's build an FAQ system! Sentence transformers are powerful models that can measure semantic text similarity. While the transformer encoder usually outputs an embedding for each token, sentence transformers output an embedding for the whole input text, allowing us to determine if two texts have similar meanings based on their similarity score. Let's look at a simple example using the *sentence_transformers* library.

```
from sentence_transformers import
SentenceTransformer, util

sentences = ["I'm happy", "I'm full of
happiness"]
model = SentenceTransformer("sentence-
transformers/all-MiniLM-L6-v2")

# Compute embedding for both lists
embedding_1 = model.encode(sentences[0],
convert_to_tensor=True)
embedding_2 = model.encode(sentences[1],
convert_to_tensor=True)

util.pytorch_cos_sim(embedding_1,
embedding_2)
```

```
tensor([[0.6003]], device='cuda:0')
```

Write a list of five questions and answers about a topic. Your goal will be to build a system that, given a new question, can give the user the most likely answer. How can we use sentence transformers to solve this? The [supplementary material](#) contains the solution, but although challenging, we suggest trying it first before looking there.

NOTE

A powerful technique called Retrieval Augmented Generation (RAG) combines text generation and embeddings to retrieve relevant documents. Appendix C shows an end-to-end example of how to build a minimal RAG pipeline. Before that, we suggest reading Chapter 6, which introduces fine-tuning and how to use it to adapt models to specific tasks.

References

1. Brown, Tom B., et al. *Language Models Are Few-Shot Learners*. arXiv, 22 July 2020. arXiv.org, <http://arxiv.org/abs/2005.14165>
2. Devlin, Jacob, et al. *BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding*. arXiv, 24 May 2019. arXiv.org, <http://arxiv.org/abs/1810.04805>

3. Dosovitskiy, Alexey, et al. *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv, 3 June 2021. arXiv.org, <http://arxiv.org/abs/2010.11929>
4. Fourrier, Clémentine. *Introduction to Graph Machine Learning*, Hugging Face Blog, <https://huggingface.co/blog/intro-graphml>
5. Gao, Leo, et al. *Scaling and evaluating sparse autoencoders*. arXiv, 6 June 2024. arXiv.org, <https://arxiv.org/abs/2406.04093>
6. Holtzman, Ari, et al. *The Curious Case of Neural Text Degeneration*. arXiv, 14 Feb. 2020. arXiv.org, <http://arxiv.org/abs/1904.09751>
7. Howard, Jeremy, and Sebastian Ruder. *Universal Language Model Fine-Tuning for Text Classification*. arXiv, 23 May 2018. arXiv.org, <http://arxiv.org/abs/1801.06146>
8. Lewis, Mike, et al. *BART: Denoising Sequence-to-Sequence Pre-Training for Natural Language Generation, Translation, and Comprehension*. arXiv, 29 Oct. 2019. arXiv.org, <http://arxiv.org/abs/1910.13461>
9. Radford, Alec, et al. *Language models are unsupervised multitask learners*. OpenAI blog 1, no. 8 (2019): 9.
10. Radford, Alec, et al. *Improving language understanding by generative pre-training*. (2018).

11. Raffel, Colin, et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. arXiv, 28 July 2020. arXiv.org, <http://arxiv.org/abs/1910.10683>
12. T. Lan. *Generating human-level text with contrastive search in Transformers* 🙌 Hugging Face Blog, <https://huggingface.co/blog/introducing-csearch>
13. Vaswani, Ashish, et al. *Attention is all you need*. Advances in neural information processing systems 30 (2017). [arXiv preprint](#)
14. Yang, Jingfeng, et al. *Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond*. arXiv, 27 Apr. 2023. arXiv.org, <http://arxiv.org/abs/2304.13712>

• You can explore different tokenizers interactively in the Tokenizer Playground at <https://huggingface.co/spaces/Xenova/the-tokenizer-playground>.

• We use `<` to align the token to the left, `10` to specify the width of the field, and `.2%` to format the probability as a percentage with two decimal places.

• In statistics, a distribution is a way of describing how the values of a variable are spread out. It tells us how often different values of the variable occur.

• An excellent deep dive into contrastive search is the *Generating Human-level Text with Contrastive Search* blog post (<https://huggingface.co/blog/introducing-csearch>).

• The first example in the GPT-2 release blog post was famously a news story about unicorns (<https://openai.com/research/better-language-models>).

- 1 Vaswani et al. *Attention is all you need*. Advances in neural information processing systems 30 (2017). [arXiv preprint](#)
- 2 Lewis, Mike, et al. *BART: Denoising Sequence-to-Sequence Pre-Training for Natural Language Generation, Translation, and Comprehension*. arXiv, 29 Oct. 2019. arXiv.org, <http://arxiv.org/abs/1910.13461>
- 3 Devlin, Jacob, et al. *BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding*. arXiv, 24 May 2019. arXiv.org, <http://arxiv.org/abs/1810.04805>.
- 4 Yang, Jingfeng, et al. *Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond*. arXiv, 27 Apr. 2023. arXiv.org, <http://arxiv.org/abs/2304.13712>
- 5 Howard, Jeremy, and Sebastian Ruder. *Universal Language Model Fine-Tuning for Text Classification*. arXiv, 23 May 2018. arXiv.org, <http://arxiv.org/abs/1801.06146>
- 6 This oversimplifies how semantic search works, but we'll get a chance to build a simple search system using semantic embeddings in the challenge section of this chapter. This is the core of Retrieval Augmented Generation.
- 7 DistilBERT is a smaller model that preserves 95% of the original BERT performance while having 40% less parameters. RoBERTa is a very powerful BERT-based model trained for longer with different hyperparameters.
- 8 One can cache things to avoid processing tokens from scratch. Even then, for each new token, the model must attend to all the previous tokens.
- 9 Among the different lines of research on this, using sparse AutoEncoders to extract interpretable features from transformers is becoming increasingly popular. You can find the original paper at <https://arxiv.org/abs/2406.04093>

† Dosovitskiy, Alexey, et al. *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv, 3 June 2021. arXiv.org, <http://arxiv.org/abs/2010.11929>

Chapter 3. Compressing and Representing Information

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the third chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

This chapter introduces Machine Learning models and techniques to learn efficient data representations for tasks involving images, videos, or text. Why are efficient representations important? We want to reduce the amount of information we need to store and process while keeping the essential characteristics of the data. Rich representations

enable training models specialized on particular tasks, and making the representations compact reduces the computational requirements to train and work with data-intensive models. For example, training on a vector embedding of an image can be more efficient and expressive than doing it directly on its pixels.

Traditional compression methods like zip or JPEG focus on specific data types and use hand-crafted algorithms to reduce file sizes. While these methods are effective for their intended purposes, they lack the flexibility and adaptability of learned compression techniques. Zip, for instance, excels at lossless compression of general data by identifying and encoding repetitive patterns. On the other hand, JPEG is designed specifically for image compression and achieves significant size reduction by discarding less noticeable visual information. However, these traditional methods don't learn from the data they compress and can't automatically adapt to different types of content or optimize for specific tasks beyond size reduction. This is where Machine Learning models can be useful.

We'll begin by exploring AutoEncoders, a family of Machine Learning models that consist of an encoder that "compresses" data and a decoder that reconstructs it just using the representation. The encoder learns the essential features of the data it needs to focus on, which allows the decoder to reverse

the transformations, as shown in the left panel of Figure 3-1. This training approach is a way to build compressors automatically without relying on hand-crafted algorithms. Compressing information (even in a lossy way) is helpful in itself, but there are a few other interesting things we can do once we have a compact dataset representation.

If our system is correctly trained and the decoder can recover the original from the representations, it means that the learned representations have captured the essential information. Therefore, operating on the representations is equivalent to working with the originals but requires much less memory and computing. This is one of the key design aspects of models such as Stable Diffusion – as we'll see in Chapter 5, we can generate and manipulate large images, but most of the computation happens in the smaller *latent space* where representations reside.

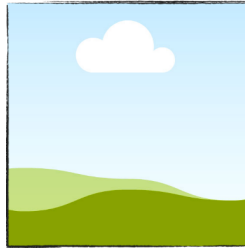


Figure 3-1. Efficient data representation methods (left) can be used for other tasks, such as classification (middle), or to generate new content (right).

Because the learned representations capture the essential information, we can split the encoder and decoder once the AutoEncoder is trained and use the encoder as a feature extraction component. Adding a small network on top of the encoder's outputs, as represented in the center panel of Figure 3-1, allows us to train the model for different tasks, such as text or image classification. These small networks don't operate on the whole input image but on the essential characteristics obtained by the encoder.

We can also encode different data types to the same latent space representations. As we saw in Chapter 2, sequence-to-sequence

language models use an encoder-decoder architecture to perform a wide variety of tasks, such as translation or summarization. Although there are more details to consider when designing such systems, one key insight is that the encoder's job is to capture the essential features that carry enough semantic information about the input text. This works across modalities, too: the job of an image captioning model, for example, is to translate image representations to textual descriptions, using a latent space as the internal working data.

A final example of the use of AutoEncoders is for generative modeling (right panel of Figure 3-1). After we have trained the encoder-decoder pair, we can throw away the encoder and generate new data by sampling from a random distribution in the latent space. This is the base of Variational AutoEncoders (or VAEs). We'll see an example of how it's done in the second section of this chapter.

We'll use image data to showcase how AutoEncoders and VAEs work, but the techniques can be applied to any data, not just images. The final section of this chapter will examine how multimodal representation learning systems, such as CLIP, bridge the gap between text and images and can be used for very interesting use cases such as semantic search, data filtering, text-to-image generation, and more.

AutoEncoders

AutoEncoders consist of two models stitched together: an encoder and a decoder, schematically represented in Figure 3-2. Both models are trained together with the objective that the encoder produces intermediate representations, which the decoder then uses to regenerate the input data. If training succeeds, the AutoEncoder learns to extract key features from the input data.

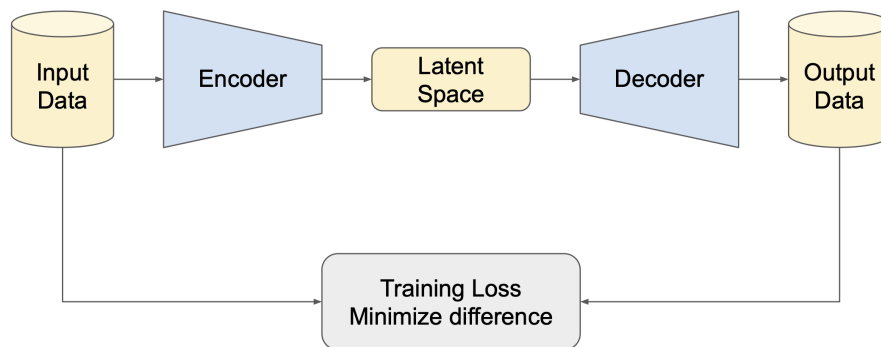


Figure 3-2. AutoEncoder: conceptual architecture diagram

Preparing the Data

In this section, we'll build a simple AutoEncoder using the MNIST dataset. MNIST is a classical dataset consisting of 70,000 low-resolution (28x28) black-and-white images of handwritten numerical digits. We'll download it from the redistribution of the dataset [hosted in Hugging Face](#). To download it, we'll use a library called [datasets](#), which provides a unified API to access thousands of datasets for any type of data. Details about how it works are not important now; just note that it will take care of downloading and caching for subsequent use. It provides two dataset splits: a `train` set with 60,000 images and a `test` dataset with the remaining 10,000 images.

```
from datasets import load_dataset

mnist = load_dataset("mnist")
```

```
mnist
```

```
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 60000
```

```
    ))  
    test: Dataset({  
        features: ['image', 'label'],  
        num_rows: 10000  
    })  
))
```

As you can see, the dataset contains a column called `image`, with the handwritten images, and `label`, which contains the number represented by the image. As we're going to train an AutoEncoder to compress and reconstruct the image, we don't really need the label data at all: we'll feed batches of random samples to the encoder, and the decoder's job will be to regenerate images that resemble the inputs. Because the input data has everything required to train without relying on external annotated information, AutoEncoder training is an example of a *self-supervised* learning process.

We will ignore the labels for training, but we'll use them later for visualization purposes. As always, before training a model, let's explore the dataset.

```
mnist["train"]["image"][1]
```

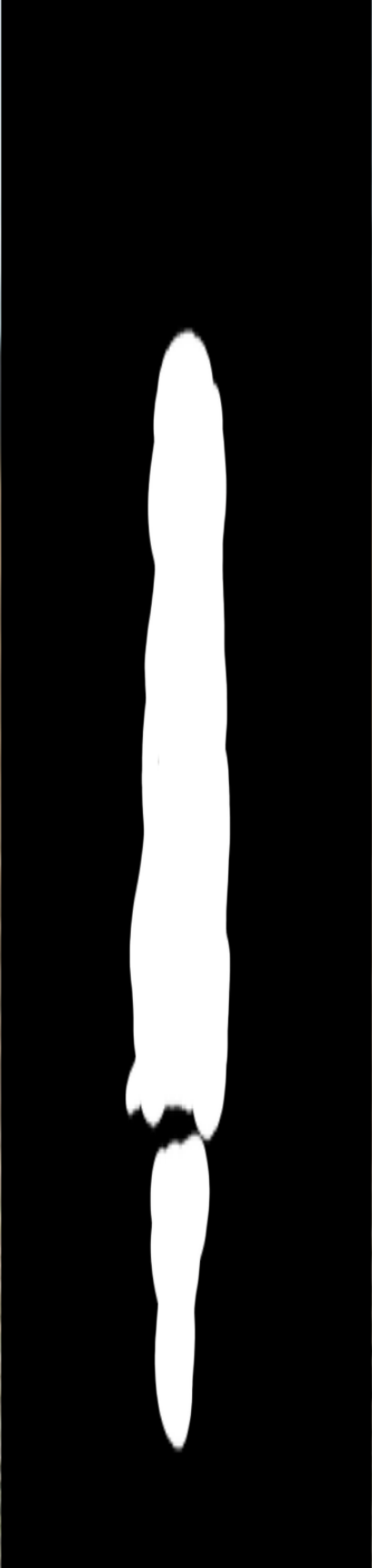


Figure 3-3. A sample of the MNIST dataset.

Images with a resolution of just 28x28 are very small by today's standards. We'll use a helper function, `show_images()`, to show them with higher resolution. `show_images()` is based on Python's *matplotlib* library, which by default uses a high-contrast color palette to represent monochrome image data, shown in Figure 3-4.

```
from genaibook.core import show_images

show_images(mnist["train"]["image"][:4])
```

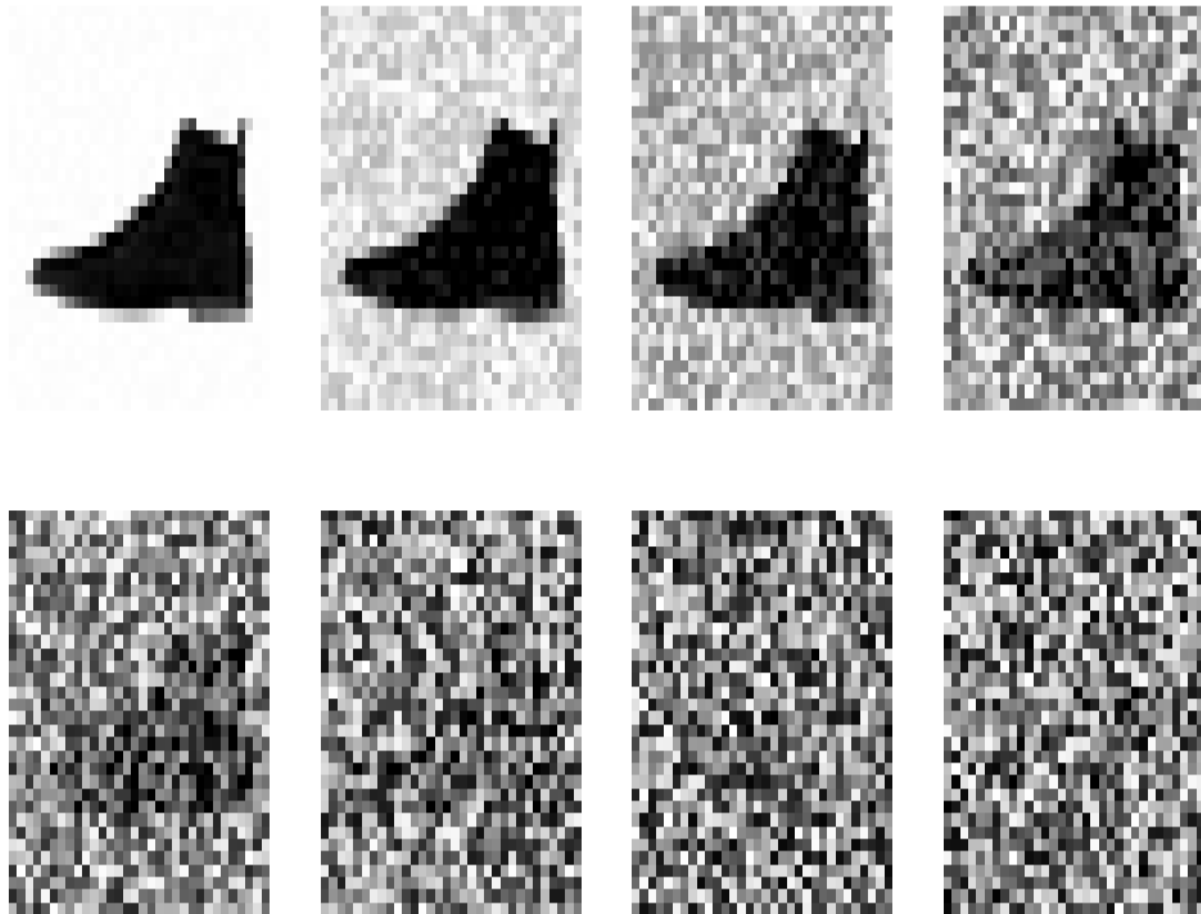


Figure 3-4. Some samples from the MNIST dataset, rendered by *matplotlib*.

Because the originals are black and white, we'll configure *matplotlib* to just use gray colors instead. We choose "`reversed gray`" (`gray_r`) to get black numbers on a white background, shown in Figure 3-5. Note that the originals are the other way around (pixels with number data are white while the background is all zeros, which means black color).

```
import matplotlib as mpl

mpl.rcParams["image.cmap"] = "gray_r"

show_images(mnist["train"]["image"][:4])
```

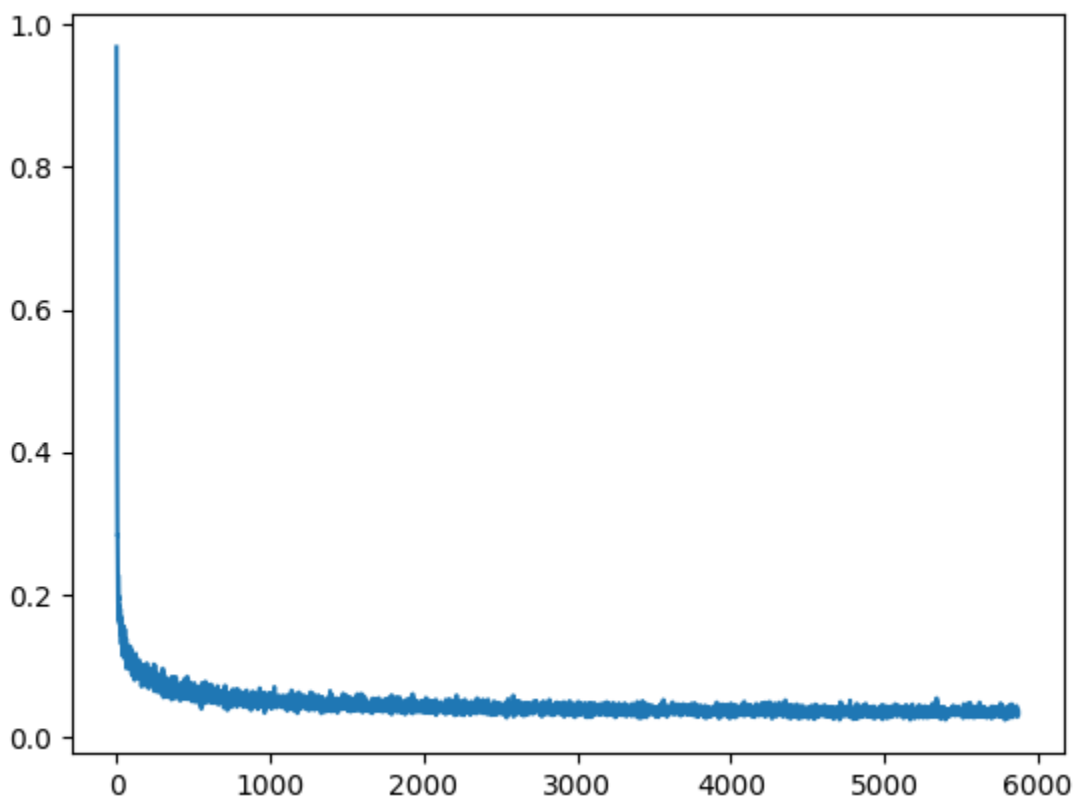


Figure 3-5. MNIST samples drawn using a monochrome palette.

We convert the images to Pytorch tensors and shuffle the training dataset. We'll use `ToTensor()`, from the *torchvision* library¹, to convert the input pixels, in the `[0, 255]` range, to

PyTorch tensors between 0 and 1. We don't apply any other manipulations. For convenience, `show_images` can also draw tensors representing images, as shown in Figure 3-6.

```
from torchvision import transforms

def mnist_to_tensor(samples):
    t = transforms.ToTensor()
    samples["image"] = [t(image) for image in
samples["image"]]
    return samples
```

```
mnist = mnist.with_transform(mnist_to_tensor)
mnist["train"] =
mnist["train"].shuffle(seed=1337)
```

Let's check out a single image from the dataset and confirm that the input pixels range from 0 to 1.

```
x = mnist["train"]["image"][0]
x.min(), x.max()
```

```
(tensor(0.), tensor(1.))
```

```
show_images(mnist["train"]["image"][0])
```

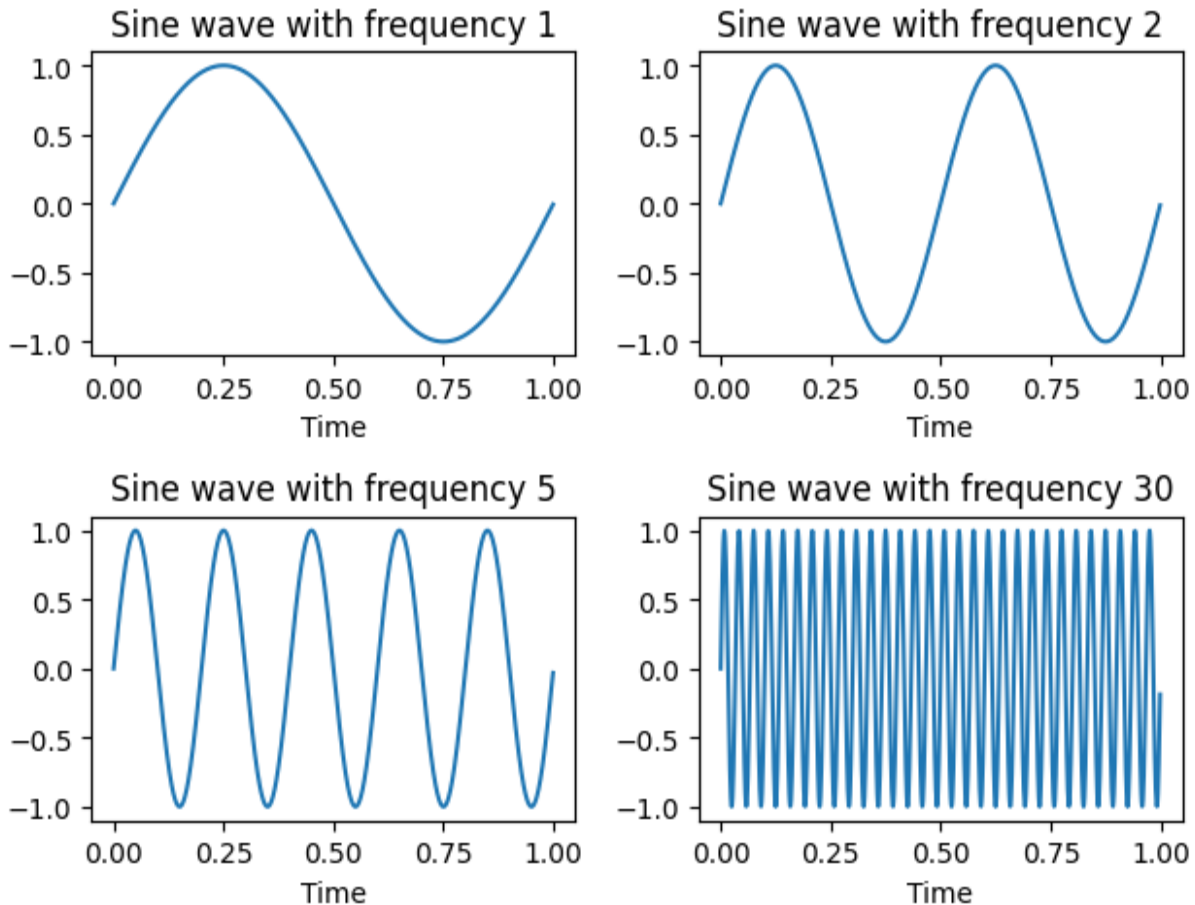


Figure 3-6. `show_images()` can also display tensors.

We now create a PyTorch `DataLoader` to prepare the training data. Because training an AutoEncoder is a self-supervised process, we'll just work with the `image` column of the dataset and ignore the labels. Later, we'll return and use the labels to visualize results. A `DataLoader` is an abstraction whose primary mission is to collate inputs, which means gathering

and combining individual samples into training batches with the same shape. In our case, all images have the same size, and therefore, all tensors have the same shape, so the dataloader will just concatenate them together. In more complicated cases, dataloaders may need to deal with irregular input shapes using strategies such as padding or truncation.

```
from torch.utils.data import DataLoader

bs = 64
train_dataloader = DataLoader(mnist["train"]
                              ["image"], batch_size=bs)
```

Modeling the Encoder

First, we'll create a model definition for the encoder part of the AutoEncoder. Because we are working with image data, a natural choice is to use convolutional layers, which are good at capturing image features. We could consider many other alternatives for this problem: linear layers, transformer blocks, use of residual skip connections, etc. We'll use a simple convolutional encoder that is based on a [convolutional AutoEncoder implementation](#) from the excellent *pythae* library. This is a great starting point for exploration!

WHAT ARE CONVOLUTIONAL LAYERS?

Convolutional layers are a collection of small 2D filters applied repeatedly to different regions of the input image. These filters can detect patterns such as lines or circular areas. Traditionally, 2D filters have been used in digital image processing, where they are carefully hand-crafted to match specific features in input images. The main difference with convolutional layers is that the filters are not prepared beforehand – instead, convolutional layers *learn them* as part of the training process of the network. By stacking multiple convolutional layers, the model can progressively extract more abstract features from the input image, learning filters that effectively resolve the task. There's a thrilling field of interpretation and explainability that aims to visualize and understand the internal workings of model layers, and it's been shown that filters learned by neural networks sometimes resemble classic filters designed manually to detect edges, colors, or contours.

For an in-depth look at Convolutional Neural Networks, we recommend Chapter 13 of [*Deep Learning for Coders with fastai & PyTorch*](#), by Jeremy Howard & Sylvain Gugger.

Because we will stack several convolutional layers, we'll write a simple helper function to create them. Our `conv_block()` helper makes a 2D convolution, then appends a batch normalization layer and a non-linearity (we'll use the ReLU activation function for this example). During training, batch normalization uses the mean and standard deviation of the current batch to normalize input data so it remains within a predictable range, which most of the time results in smoother and faster training.²

```

from torch import nn

def conv_block(in_channels, out_channels,
               kernel_size=4, stride=2, padding=1):
    return nn.Sequential(
        nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding,
        ),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(),
    )

```

As described, the encoder implementation will be a sequence of convolutional layers. Each layer progressively reduces the image resolution while it increases the number of channels of the representations to 1024. Finally, we'll append a linear layer at the end to create 16-dimensional vector representations. The comments in the `forward()` method show how the shape of the input data is transformed as it travels through the layers.

```

class Encoder(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.conv1 = conv_block(in_channels,
128)

        self.conv2 = conv_block(128, 256)
        self.conv3 = conv_block(256, 512)
        self.conv4 = conv_block(512, 1024)
        self.linear = nn.Linear(1024, 16)

    def forward(self, x):
        x = self.conv1(x) # (batch size,
128, 14, 14)
        x = self.conv2(x) # (bs, 256, 7, 7)
        x = self.conv3(x) # (bs, 512, 3, 3)
        x = self.conv4(x) # (bs, 1024, 1, 1)
        # Keep batch dimension when
flattening
        x =
self.linear(x.flatten(start_dim=1)) # (bs,
16)

        return x

```

Let's verify that we can run our input images through the encoder. They have a `[1, 28, 28]` shape because they only contain one channel of (black or white) pixel data. However,

note that we coded our encoder in a general way: we could use it for 3-channel images as well.

```
mnist["train"]["image"][0].shape
```

```
torch.Size([1, 28, 28])
```

Let's select and put a single image inside a batch, creating a new dimension with PyTorch's `None` indexing. We also need to set the encoder in `eval` mode. `eval` mode configures the model for inference instead of training. If we don't do this, the last `BatchNorm2d` layer will fail because it will receive a tensor with shape `[1, 1024, 1, 1]`, and it can't compute the mean and standard deviation of a single sample³.

```
in_channels = 1

x = mnist["train"]["image"][0][None, :]
encoder = Encoder(in_channels).eval()

encoded = encoder(x)
encoded.shape
```

```
torch.Size([1, 16])
```

The encoder model works! It converts 28x28 images (784 pixels each) into vectors with just 16 numbers. If we can train it effectively, the representations computed by the encoder will have much lower dimensionality than the original pixel data.

Of course, the representations are currently meaningless, as the model has yet to be trained.

```
encoded
```

```
tensor([[ -0.0145, -0.0318, -0.0109,  0.0080,  
        -0.0218,  0.0305,  0.0183, -0.0294,  
                0.0075,  0.0178, -0.0161, -0.0018,  
        0.0208, -0.0079,  0.0215,  0.0101]],  
        grad_fn=<AddmmBackward0>)
```

Let's see if it can handle a batch of 64 images.

```
batch = next(iter(train_dataloader))  
encoded = Encoder(in_channels=1)(batch)  
batch.shape, encoded.shape
```

```
(torch.Size([64, 1, 28, 28]), torch.Size([64,  
16]))
```


This completes our `Encoder` model, which transforms images to intermediate representations. Let's move to the `Decoder` now.

Decoder

The decoder begins with the *latent* representation obtained by the encoder (vectors with 16 dimensions) and turns them into images of the original size.

The decoder architecture does not have to be the reverse of the encoder's – it can be anything that "`understands`" the encoder representations and is able to translate them to images. In our case, we'll create a more or less symmetrical network to the encoder: we'll apply transposed convolutions⁴ to increase the resolution while decreasing the number of channels until we reach our desired output resolution of 28x28 pixels. We'll prepend the transposed convolutions with a linear layer to create tensors of 16384 (1024x4x4) pixels. This layer will be reshaped to a 4x4 resolution (`[1024, 4, 4]`) which will be the input to the first transposed convolution. From there, we progressively reduce the channels and increase the resolution until we reach the original image shape. There are other ways to achieve the same; remember that the input is a flat vector

with 16 channels, and the output must consist of 1 channel and 28x28 pixels.

```
def conv_transpose_block(
    in_channels,
    out_channels,
    kernel_size=3,
    stride=2,
    padding=1,
    output_padding=0,
    with_act=True,
):
    modules = [
        nn.ConvTranspose2d(
            in_channels,
            out_channels,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding,
            output_padding=output_padding,
        ),
    ]
    if with_act: # Controlling this will be
        handy later

    modules.append(nn.BatchNorm2d(out_channels))
```

```
modules.append(nn.ReLU())
return nn.Sequential(*modules)
```

```
class Decoder(nn.Module):
    def __init__(self, out_channels):
        super().__init__()

        self.linear = nn.Linear(
            16, 1024 * 4 * 4
        ) # note it's reshaped in forward
        self.t_conv1 =
conv_transpose_block(1024, 512)
        self.t_conv2 =
conv_transpose_block(512, 256,
output_padding=1)
        self.t_conv3 =
conv_transpose_block(256, out_channels,
output_padding=1)

    def forward(self, x):
        bs = x.shape[0]
        x = self.linear(x) # (bs, 1024*4*4)
        x = x.reshape((bs, 1024, 4, 4)) #
(bs, 1024, 4, 4)
        x = self.t_conv1(x) # (bs, 512, 7,
7)
        x = self.t_conv2(x) # (bs, 256, 14,
```

```
14)
    x = self.t_conv3(x) # (bs, 1, 28,
28)
    return x
```

```
decoded_batch = Decoder(x.shape[0])(encoded)
decoded_batch.shape
```

```
torch.Size([64, 1, 28, 28])
```

Training

So far, we have created the `Encoder`, which reduces the dimensionality of the input images, and the `Decoder`, which expands low-dimensional latent to the original image resolution. In addition to being initialized with random weights, these two components are completely unconnected at the moment. We need to train them together so they both understand the same latent representations.

To do so, we'll create an `AutoEncoder` model that passes the input data through the encoder and the decoder in sequence. We'll train it to minimize the difference between the decoded image at the output and the original image we supplied as

input. If we are successful, the output images will resemble the inputs. This process is useful as it allows data compression, but it becomes even more interesting when we realize that we can use the two components separately after training. This will enable us to do many exciting things in the following chapters. For example, we can use the encoder to convert arbitrary images to more compressed representations, which can be used as inputs by other models. We can also use the decoder to generate new images that resemble the ones from the training dataset. Excited? Let's get started on training our AutoEncoder.

```
class AutoEncoder(nn.Module):  
    def __init__(self, in_channels):  
        super().__init__()  
        self.encoder = Encoder(in_channels)  
        self.decoder = Decoder(in_channels)  
  
    def encode(self, x):  
        return self.encoder(x)  
  
    def decode(self, x):  
        return self.decoder(x)  
  
    def forward(self, x):  
        return self.decode(self.encode(x))
```

```
model = AutoEncoder(1)
```

We can use the *torchsummary* library to print a summary of the model, which shows the number of parameters and the output shape of each layer. This is a useful tool to check if the model is correctly defined and to understand the model's architecture.

```
import torchsummary

torchsummary.summary(model, input_size=(1,
28, 28), device="cpu")
```

```
-----
-----
          Layer (type)                Output
Shape          Param #
=====
          Conv2d-1                [-1, 128, 14,
14]                2,176
          BatchNorm2d-2            [-1, 128, 14,
14]                256
          ReLU-3                  [-1, 128, 14,
14]                0
          Conv2d-4                [-1, 256, 7,
7]                524,544
          BatchNorm2d-5            [-1, 256, 7,
```

7]	512	
	ReLU-6	[-1, 256, 7,
7]	0	
	Conv2d-7	[-1, 512, 3,
3]	2,097,664	
	BatchNorm2d-8	[-1, 512, 3,
3]	1,024	
	ReLU-9	[-1, 512, 3,
3]	0	
	Conv2d-10	[-1, 1024, 1,
1]	8,389,632	
	BatchNorm2d-11	[-1, 1024, 1,
1]	2,048	
	ReLU-12	[-1, 1024, 1,
1]	0	
	Linear-13	[-1,
16]	16,400	
	Encoder-14	[-1,
16]	0	
	Linear-15	[-1,
16384]	278,528	
	ConvTranspose2d-16	[-1, 512, 7,
7]	4,719,104	
	BatchNorm2d-17	[-1, 512, 7,
7]	1,024	
	ReLU-18	[-1, 512, 7,
7]	0	
	ConvTranspose2d-19	[-1, 256, 14,

```
14]          1,179,904
      BatchNorm2d-20      [-1, 256, 14,
14]          512
      ReLU-21             [-1, 256, 14,
14]          0
      ConvTranspose2d-22   [-1, 1, 28,
28]          2,305
      BatchNorm2d-23      [-1, 1, 28,
28]          2
      ReLU-24             [-1, 1, 28,
28]          0
      Decoder-25          [-1, 1, 28,
28]          0
```

=====

```
Total params: 17,215,635
Trainable params: 17,215,635
Non-trainable params: 0
```


```
Input size (MB): 0.00
Forward/backward pass size (MB): 2.86
Params size (MB): 65.67
Estimated Total Size (MB): 68.54
```


We'll create a simple training loop that repeatedly goes through the training data and uses a constant learning rate. To focus on the essentials, we won't bother running validations on the test set (but you are encouraged to do it to practice!). We use the popular *tqdm* library for progress display, but we won't describe it here for the sake of brevity – we'll see more examples in other chapters. It's not necessary to understand everything that's going on; just pay attention to the high-level operations:

- Load a batch from the dataloader
- Get the model predictions
- Calculate the loss with respect to the original images
- Perform an optimizer step to update the model weights

```
import torch
from matplotlib import pyplot as plt
from torch.nn import functional as F
from tqdm.notebook import tqdm, trange

from genaibook.core import get_device

num_epochs = 10
lr = 1e-4

device = get_device()
```

```

model = model.to(device)
optimizer =
torch.optim.AdamW(model.parameters(), lr=lr,
eps=1e-5)

losses = [] # List to store the loss values
for plotting
for _ in (progress := trange(num_epochs,
desc="Training")):
    for _, batch in (
        inner :=
tqdm(enumerate(train_dataloader),
total=len(train_dataloader))
    ):
        batch = batch.to(device)

        # Pass through the model and obtain
reconstructed images
        preds = model(batch)

        # Compare the prediction with the
original images
        loss = F.mse_loss(preds, batch)

        # Display loss and store for plotting
        inner.set_postfix(loss=f"
{loss.cpu().item():.3f}")
        losses.append(loss.item())

```

```
        # Update the model parameters with
the optimizer based on this loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        progress.set_postfix(loss=f"
{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
```

Let's plot the loss curve in Figure 3-7 to see how the training went.

```
plt.plot(losses)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("AutoEncoder - Training Loss
Curve")
plt.show()
```



Figure 3-7. Training loss of the AutoEncoder.

We didn't perform validation during the training loop, but we can see how the AutoEncoder fares with the test set. Training with visual data is a great way to learn and iterate, as we can see the final results and judge for ourselves.

We'll create a batch with 16 samples from the test set, pass them through the trained encoder and decoder, and display the reconstructions. Let's begin creating the evaluation dataloader.

```
eval_bs = 16
eval_dataloader = DataLoader(mnist["test"])
```

```
["image"], batch_size=eval_bs)
```

We'll use `model.eval()` to put the model in evaluation mode (in our case, it will disable `BatchNorm` updates) and the `inference_mode` context manager to turn off gradient computation.

```
model.eval()
with torch.inference_mode():
    eval_batch = next(iter(eval_dataloader))
    predicted =
model(eval_batch.to(device)).cpu()
```

Now that we have the predictions, let's display the original images and their reconstructions.

```
batch_vs_preds = torch.cat((eval_batch,
predicted))
show_images(batch_vs_preds, imsize=1,
nrows=2)
```

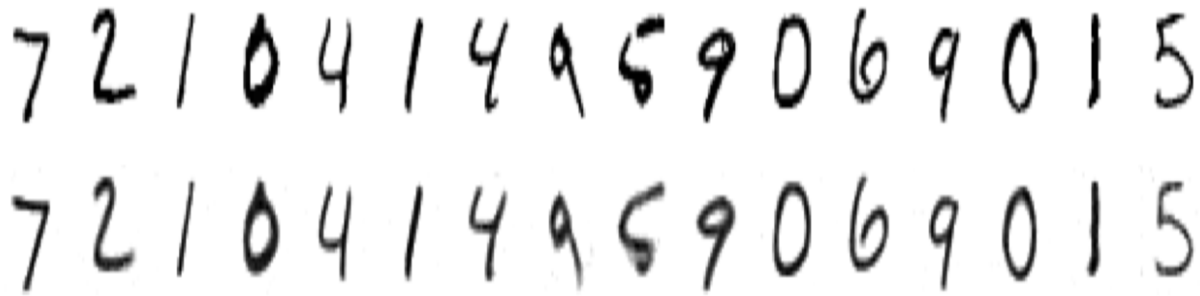


Figure 3-8. Original MNIST images (top) and reconstructions from our AutoEncoder (bottom)

The results shown in Figure 3-8 look pretty good! Remember that the numbers in the second row are approximations of the originals obtained by the decoder from concise vector representations.

At this point, we suggest ensuring you understand the different concepts just introduced. We also recommend trying to get better reconstruction results. Some experiment ideas:

- Progressively decrease the learning rate
- Try different batch sizes
- Use a sigmoid function at the decoder's end to encourage the final pixel values to be either black or white. Note that our input data is between 0 and 1, so the sigmoid output must match that range
- Play with the network depth or topology

We also suggest experimenting with the training loop by adding and logging the evaluation loss during training.

Exploring the Latent Space

One important hyperparameter of the `AutoEncoder` is the number of dimensions we use to represent the encoded inputs. We arbitrarily chose 16, and our results show that this seems enough to represent the wide variety of hand-drawn numbers in the MNIST dataset.

For our next experiment, we will use just two dimensions to represent the vectors in the latent space. We'll force the encoder to squeeze as much information as possible about input images into just two float numbers, and we'll figure out if this is enough to recover the inputs. In addition, using two dimensions is very convenient for visualization. After training our new model, we can draw some interesting plots in 2D space that will help us gain additional intuition.

We'll slightly refactor our code with the following changes:

- We include the dimensionality of the latent space as a hyperparameter.

- We use a container (`nn.Sequential`) for the convolution layers to make it easier to adjust the network depth if we want to experiment with that later.
- We replace the activation after the final decoder convolution with a sigmoid function. We want to encourage the decoder to produce pixels that are either black or white, and a sigmoid is better suited than a ReLU for that purpose. This is because the sigmoid function squashes the output to the range `(0, 1)`, the same range as pixels in the images

This can also be an excellent time to put our training loop inside a function.

TIP

Don't try to create code with many options and parameters from the start. It's better to start with the simplest working code you can write and progressively make it richer as needed (and *if* required).

Figure 3-9 shows a plot of the `ReLU` vs. the `Sigmoid` function. By using Sigmoid as the activation function, we ensure that the output after each layer lies within the range `(0, 1)`, the same range input images use. This is not strictly necessary for the network to learn; we are trying to help it because we know our desired output range.

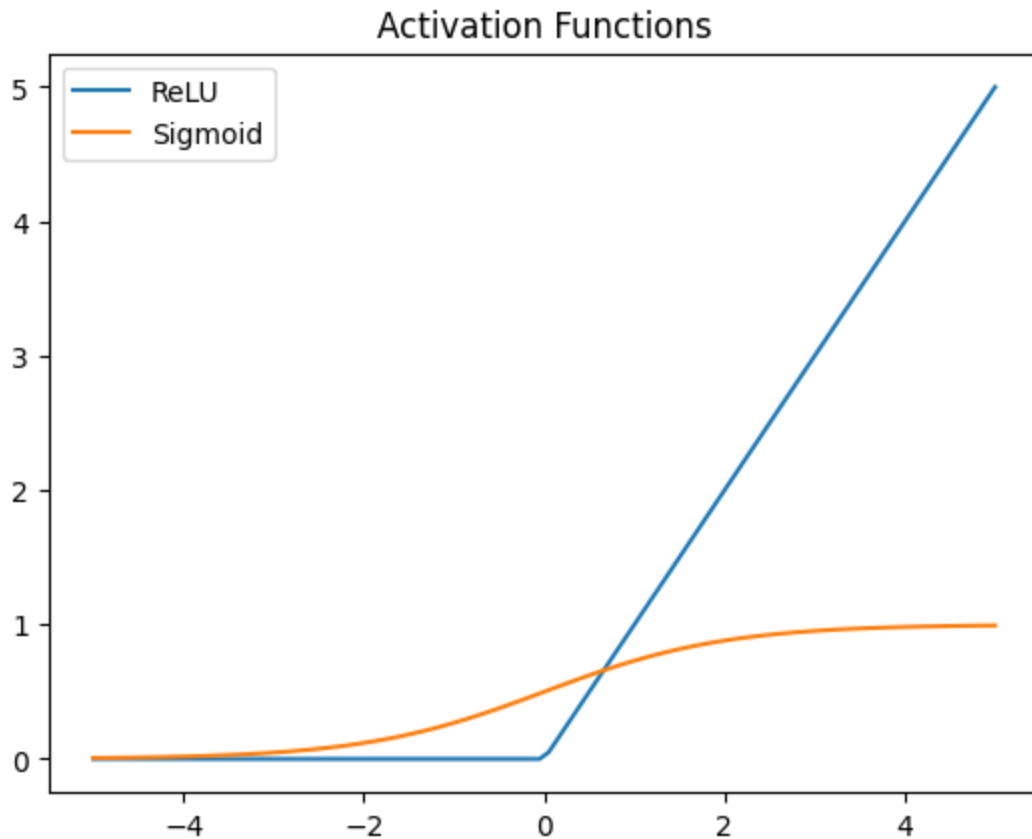


Figure 3-9. Activation functions: ReLU vs Sigmoid

```
class Encoder(nn.Module):
    def __init__(self, in_channels,
latent_dims):
        super().__init__()

        self.conv_layers = nn.Sequential(
            conv_block(in_channels, 128),
            conv_block(128, 256),
            conv_block(256, 512),
            conv_block(512, 1024),
```

```

        )
        self.linear = nn.Linear(1024,
latent_dims)

    def forward(self, x):
        bs = x.shape[0]
        x = self.conv_layers(x)
        x = self.linear(x.reshape(bs, -1))
        return x

```

```

class Decoder(nn.Module):
    def __init__(self, out_channels,
latent_dims):
        super().__init__()

        self.linear = nn.Linear(latent_dims,
1024 * 4 * 4)
        self.t_conv_layers = nn.Sequential(
            conv_transpose_block(1024, 512),
            conv_transpose_block(512, 256,
output_padding=1),
            conv_transpose_block(
                256, out_channels,
output_padding=1, with_act=False
            ),
        )
        self.sigmoid = nn.Sigmoid()

```

```

def forward(self, x):
    bs = x.shape[0]
    x = self.linear(x)
    x = x.reshape((bs, 1024, 4, 4))
    x = self.t_conv_layers(x)
    x = self.sigmoid(x)
    return x

```

The new version of the `AutoEncoder` class is almost identical to the previous one: it calls the decoder on the encoder's outputs. It simply accepts an additional argument, `latent_dims`, so that we can specify the desired dimensionality of the representations.

```

class AutoEncoder(nn.Module):
    def __init__(self, in_channels,
latent_dims):
        super().__init__()
        self.encoder = Encoder(in_channels,
latent_dims)
        self.decoder = Decoder(in_channels,
latent_dims)

    def encode(self, x):
        return self.encoder(x)

    def decode(self, x):

```

```

        return self.decoder(x)

    def forward(self, x):
        return self.decode(self.encode(x))

```

The training loop is the same as before, but we put it inside a function to reuse it and call it whenever needed.

```

def train(model, num_epochs=10, lr=1e-4):
    optimizer =
    torch.optim.AdamW(model.parameters(), lr=lr,
    eps=1e-5)

    model.train() # Put model in training
mode
    losses = []
    for _ in (progress := trange(num_epochs,
    desc="Training")):
        for _, batch in (
            inner := tqdm(
                enumerate(train_dataloader),
            total=len(train_dataloader)
            )
        ):
            batch = batch.to(device)

            # Pass through the model and

```

```

obtain another set of images
        preds = model(batch)

        # Compare the prediction with the
original images
        loss = F.mse_loss(preds, batch)

        # Display loss and store for
plotting
        inner.set_postfix(loss=f"
{loss.cpu().item():.3f}")
        losses.append(loss.item())

        # Update the model parameters
with the optimizer based on this loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        progress.set_postfix(loss=f"
{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
    return losses

```

We create and train an AutoEncoder with just two latent variables. The training curve is shown in Figure 3-10.

```

ae_model = AutoEncoder(in_channels=1,
latent_dims=2)

```

```
ae_model.to(device)
```

```
losses = train(ae_model)
```

```
plt.plot(losses)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("Training Loss Curve (two latent dimensions)")
plt.show()
```

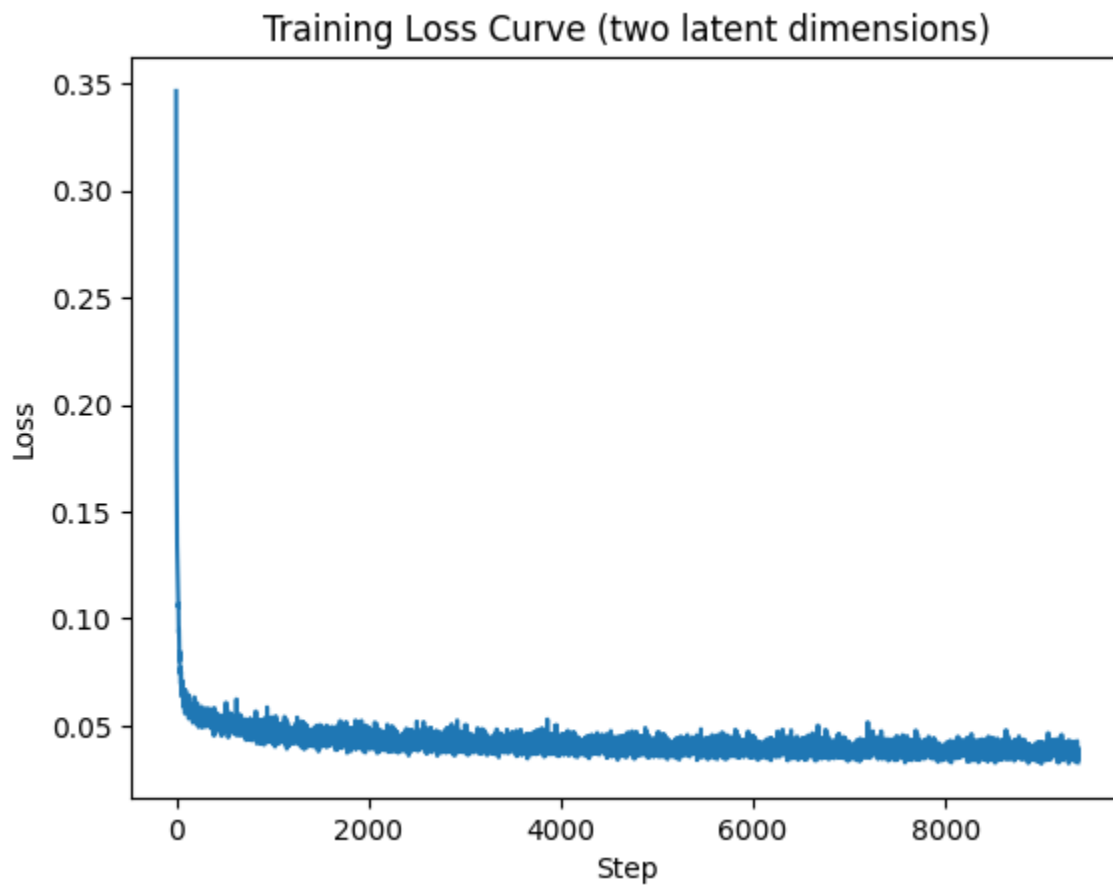


Figure 3-10. Training loss of an AutoEncoder with just two latent variables.

Let's once again load the trained model and look at some reconstructions.

```
ae_model.eval()  
with torch.inference_mode():  
    eval_batch = next(iter(eval_dataloader))  
    predicted =  
    ae_model(eval_batch.to(device)).cpu()
```

```
batch_vs_preds = torch.cat((eval_batch,  
                             predicted))  
show_images(batch_vs_preds, imsize=1,  
            nrows=2)
```

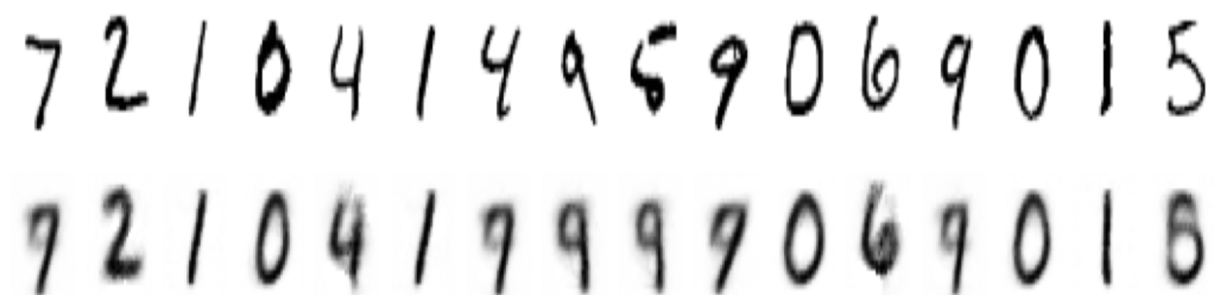


Figure 3-11. Reconstructions from an AutoEncoder that uses two latent variables

The results, shown in Figure 3-11, are not quite as good as before, but remember, we now use just *two* floats to represent a 28x28 handwritten image. We see some confusion with the 4s, the 5s, and the 9s, but overall, the recovered images are very similar to the input ones!

Visualizing the Latent Space

We used just two dimensions for the latent space to visualize its structure easily. In Figure 3-12, we represent all the encoded vectors from the test dataset, using the `label` column to assign different colors to each class. The first value of the encoded vectors will be displayed on the X-axis, and the second value will be represented on the Y-axis.

```
images_labels_dataloader =  
DataLoader(mnist["test"], batch_size=512)
```

```
import pandas as pd  
  
df = pd.DataFrame(  
    {  
        "x": [],  
        "y": [],  
        "label": [],
```



```

    }
)

for batch in tqdm(
    iter(images_labels_dataloader),
    total=len(images_labels_dataloader)
):
    encoded =
    ae_model.encode(batch["image"].to(device)).cpu()
    new_items = {
        "x": [t.item() for t in encoded[:,
0]],
        "y": [t.item() for t in encoded[:,
1]],
        "label": batch["label"],
    }
    df = pd.concat([df,
pd.DataFrame(new_items)], ignore_index=True)

```

0%| | 0/20 [00:00<?, ?it/s]

```

plt.figure(figsize=(10, 8))

for label in range(10):
    points = df[df["label"] == label]
    plt.scatter(points["x"], points["y"],

```

```
label=label, marker=".")
```

```
plt.legend();
```

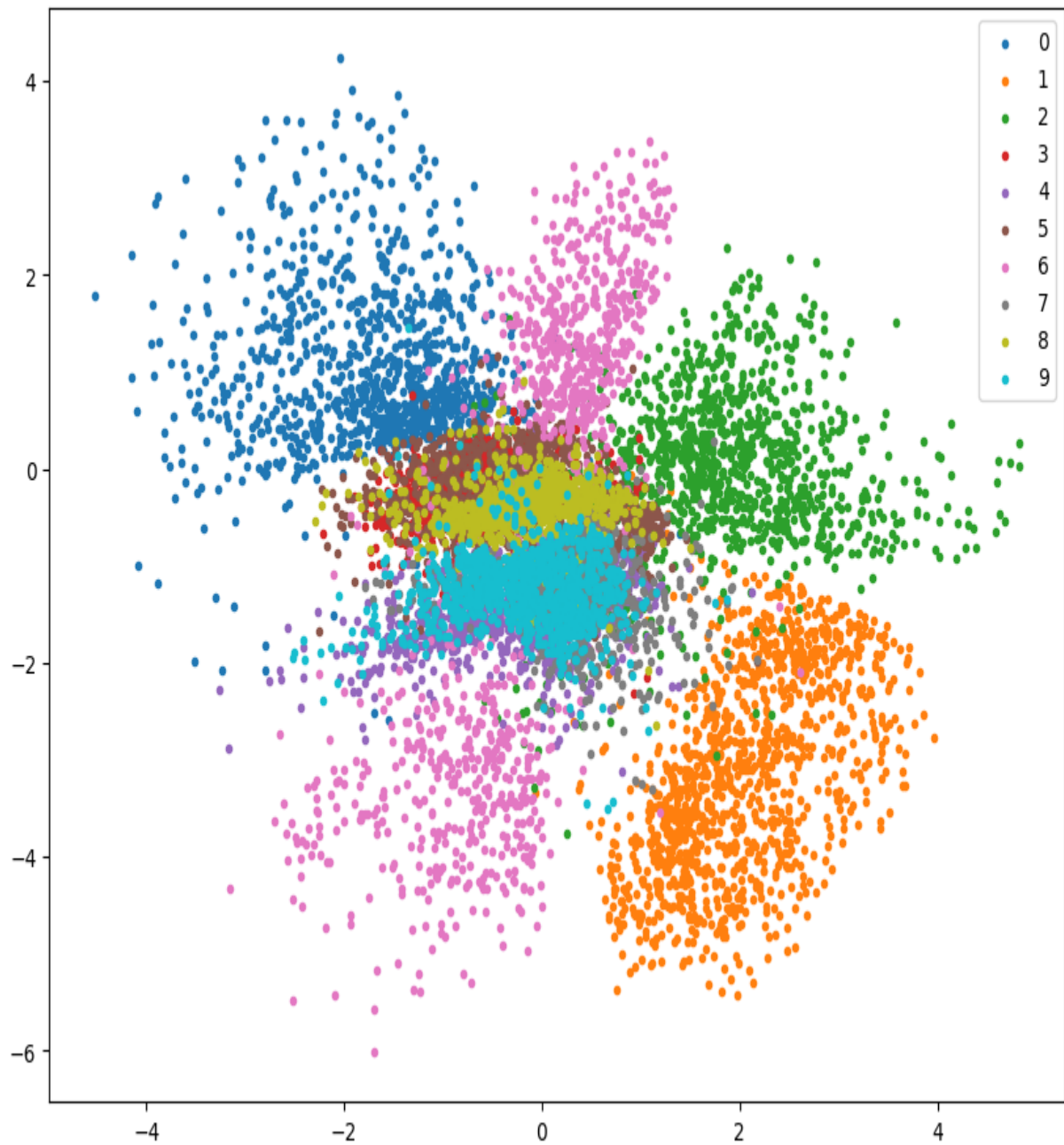


Figure 3-12. Visualization of the MNIST AutoEncoder 2D latent space.

The AutoEncoder has done a good job at separating different areas of the latent space for the various images in our dataset. Note, for example, how images representing the number 0 (dark blue dots) are distant from the representations of ones (orange dots). Remember that no information about the image labels was used during training, but even so, data points were automatically grouped in different regions according to their visual features. However, this process took place in an entirely unconstrained way, so there is no guarantee about the shape or structure of the latent space.

Therefore, the latent space is rich enough to capture the relevant image features in our dataset, but it's still not clear how we can use it for generative purposes. Ideally, to generate new images similar to the ones in MNIST, we'd want to discard the encoder and supply random samples from the latent space to the decoder. However, there are some issues we can see in the plot:

- The space taken by the representation is spread out in all directions.

- There are many overlaps at the center and big regions of empty space.
- The plot is non-symmetric: negative values in the Y-axis are used more than positive ones.

This makes it challenging to select appropriate regions in the latent space that could lead to great generations. Let's see an example of image generation with the decoder. We begin by generating random latent samples (usually denoted z).

```
N = 16 # We'll generate 16 points
z = torch.rand((N, 2)) * 8 - 4
```

Let's visualize the generated latent samples overlayed on top of the latent space representation we showed before (Figure 3-13).

```
plt.figure(figsize=(10, 8))

for label in range(10):
    points = df[df["label"] == label]
    plt.scatter(points["x"], points["y"],
                label=label, marker=".")

plt.scatter(z[:, 0], z[:, 1], label="z",
```

```
marker="s", color="black")  
plt.legend();
```

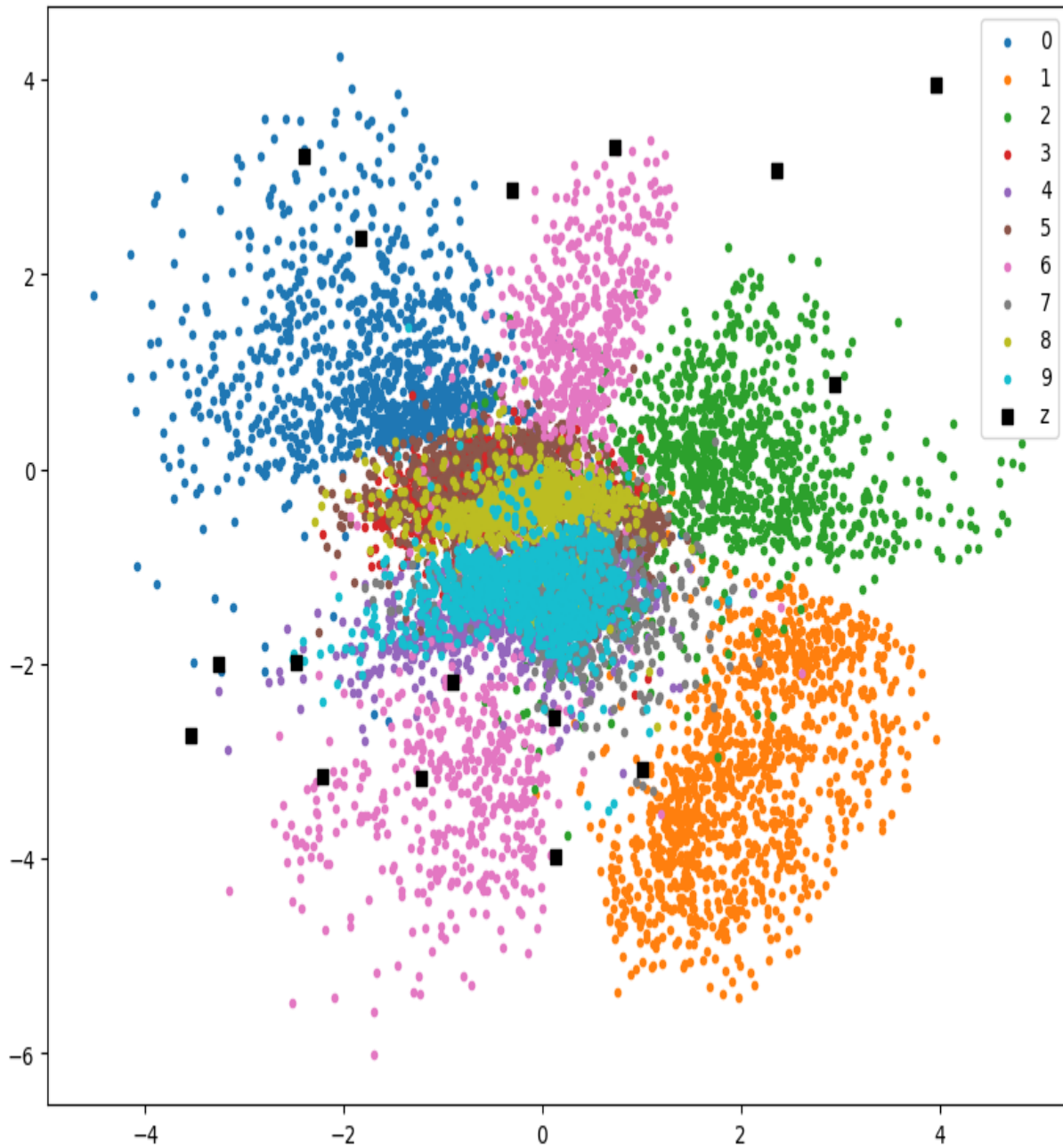


Figure 3-13. A few samples generated randomly on the latent space.

Finally, let's ask the decoder to generate images from the latent samples we just created.

```
ae_decoded = ae_model.decode(z.to(device))  
show_images(ae_decoded.cpu(), imsize=1,  
nrows=1, suptitle="AutoEncoder")
```



Figure 3-14. Images generated from the random samples.

The generated images in Figure 3-14 are reasonable in the places where the samples are close to one of the regions carved by the model in the latent space, but they are much less convincing when they lie outside those areas. In addition, note that some numbers will be overrepresented because their assigned regions in latent space are larger.

The next section discusses how we can use a different type of AutoEncoder to impose some order in latent space and how this can make generation easier.

Before moving on, here are some (progressively more challenging) exercises that you can tackle now (or later on at the end of the chapter) to reinforce your understanding of the concepts:

1. How well does generation work if the model is trained with 16 latent dimensions?
2. Train the model again with the same parameters we used (just run the code shown in the chapter) but with different random number initialization ⁵, and visualize the latent space. Chances are that the shapes and structure are different. Is this something you would expect? Why?
3. How good are the image features extracted by the encoder? Discard the decoder part of the AutoEncoder and build a number classifier on top of the encoder. You can, for example, train a couple of linear layers with a non-linearity between them. The final linear layer should output a vector with ten dimensions representing the ten labels in the dataset. Train only these layers without updating the weights of the encoder. What accuracy can you get? How does the model with 16 latent dimensions compare to the one with just two?

Variational AutoEncoders (VAEs)

In the previous section, we explored how a simple AutoEncoder can learn efficient representations of the input data in a lower-dimensional latent space. The AutoEncoder can faithfully *encode* any sample and recover (or *decode*) it later. This works great for feature extraction or data representation but is not well suited for generating new samples. As discussed before, the reason is that the AutoEncoder is not incentivized to separate the representations into consistent portions of the latent space. As we saw, representations of similar inputs are usually clustered close together, but there's a significant amount of overlap and empty space, as well as substantial variability in the amount of latent space dedicated to each class. If we choose a random point in the latent space, we can't faithfully predict what result will come up after we run it through the decoder.

Variational AutoEncoders (VAEs) address this by learning a probability distribution for each feature in the latent space. Instead of mapping inputs to specific points, VAEs represent each feature with a Gaussian distribution⁶, capturing the variability of that feature within the data.

Consider, for example, a dataset consisting of images of multiple breeds of dogs and cats. We don't know what the features extracted by the encoder will be, but we could imagine that some could be used to represent characteristics such as furry patches, eyes, ears, legs, or tails. These may have a great degree of overlap between all the images in the dataset (all these animals have two ears, four legs, and a tail), but there's also variability in how ears look in dogs versus how they look in cats. Our "ear" feature could be represented by a Gaussian distribution that covers all of this variability, with the mean of the distribution representing the average shape of an animal ear. If we move away from the mean towards different directions, we'll get a continuous and homogeneous transition towards different ear shapes that may appear in various breeds.

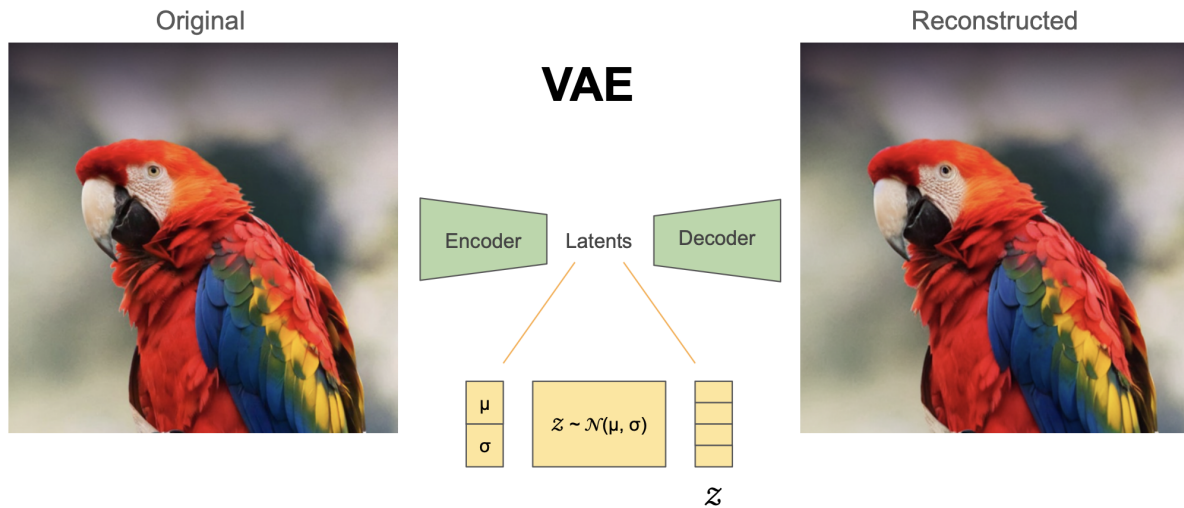


Figure 3-3. Figure 3-15. Conceptual diagram of a VAE. VAEs learn Gaussian representations of the features and describe them through their means and variances. Points z in the latent space are sampled from the predicted Gaussian distributions

This approach, conceptually represented in Figure 3-14, creates a more structured latent space, where sampling from these distributions allows us to generate new, plausible instances. Just as with AutoEncoders, class information is typically not used in VAEs.⁷ Let's see how we can code and train them.

VAE Encoders and Decoders

VAE encoders are very similar to the basic encoders we saw in the previous section. In our example, we used a few convolutional layers and a linear layer to project to the desired size of the latent representation.

To create our first VAE Encoder, we'll use the same architecture. The only difference is that, instead of a linear layer to predict the latent space of an image, we want to use linear layers to learn the distribution. A distribution is characterized by two parameters, the mean and the variance, so we'll need two linear layers:

- One of the linear layers will represent the `mean` of the distribution we are trying to learn.
- The other linear layer will learn the `variance` of the distribution.⁸

In terms of code, this is what it looks like:

```
class VAEEncoder(nn.Module):
    def __init__(self, in_channels,
latent_dims):
        super().__init__()

        self.conv_layers = nn.Sequential(
            conv_block(in_channels, 128),
            conv_block(128, 256),
            conv_block(256, 512),
            conv_block(512, 1024),
        )

        # Define fully connected layers for
```

```

mean and log-variance
        self.mu = nn.Linear(1024,
latent_dims)
        self.logvar = nn.Linear(1024,
latent_dims)

    def forward(self, x):
        bs = x.shape[0]
        x = self.conv_layers(x)
        x = x.reshape(bs, -1)
        mu = self.mu(x)
        logvar = self.logvar(x)
        return (mu, logvar)

```

You'll find minimal differences if you compare the code snippet with the `Encoder` example from the previous section. We use two linear layers instead of one to compute two different values from the same representation extracted by the convolutional layers, and we return those two values in the `forward()` method.

The purpose of these two computed values is to represent the mean and the variance of a probability distribution. However, they are initially just two identical linear layers. The challenge is to ensure that they learn to represent what we intend—mean and variance—during the training process, as we'll show.

Before that, note that the computed value `mu` represents the mean, and `logvar` represents the logarithm of the variance. We use `mu` for the mean after the Greek letter μ , frequently used in math notation to represent the mean of a normal distribution. The reason for using `logvar` rather than directly outputting the variance is primarily numerical stability, as we'll explain later.

How about the decoder? It turns out we don't need to make any changes to it. The difference between a VAE and a simple AutoEncoder lies in the way we find a point in latent space to represent an input item, but the mission of the decoder is the same: given a point in latent space (`z`), show the pixels whose encoded representation is most similar to `z`. In the case of the AutoEncoder, `z` is a linear projection of the features extracted by the convolutional layers. When we use a VAE encoder, we obtain a normal distribution, and then we *sample* from that distribution to obtain `z`. Therefore, we can use the same `Decoder` class we used in the previous section, but we do need to modify the model to *sample* from the distribution.

Sampling from the Encoder Distribution

Our updated VAE Encoder returns the mean and variance of a normal distribution that tries to match the input data

representations. To obtain a decoded output, we must sample from that distribution, as shown in the following snippet.

```
class VAE(nn.Module):
    def __init__(self, in_channels,
latent_dims):
        super().__init__()
        self.encoder =
VAEEncoder(in_channels, latent_dims) ❶
        self.decoder = Decoder(in_channels,
latent_dims)

    def encode(self, x):
        # Returns mu, log_var
        return self.encoder(x)

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        # Obtain parameters of the normal
(Gaussian) distribution
        mu, logvar = self.encode(x) ❷

        # Sample from the distribution
        std = torch.exp(0.5 * logvar) ❸
        z = self.sample(mu, std) ❹
```

```

        # Decode the latent point to pixel
space
        reconstructed = self.decode(z) ❸

        # Return the reconstructed image, and
also the mu and logvar
        # so we can compute a distribution
loss
        return reconstructed, mu, logvar ❹

def sample(self, mu, std):
    # Reparametrization trick
    # Sample from N(0, I), translate and
scale
    eps = torch.randn_like(std) ❺
    return mu + eps * std

```

- ❶ We use `latent_dims` dimensions to represent the mean and log variance of the distributions.
- ❷ The encoder computes two variables now: mean and log variance.
- ❸ Compute the standard deviation from the log variance.
- ❹
- ❺

Sample from the distribution using the computed mean and standard deviation.

- ⑤ The decoder converts the sample to an image.
- ⑥ We return not only the reconstructed image but also the mean and the log variance.
- ⑦ Reparametrization trick: sample from a standard normal distribution and then translate and scale.

So far, we've appealed to intuition to explain how VAEs work. If you allow us a brief detour into a couple of statistical concepts, we can quickly review the code above while trying to be more precise in terminology. Feel free to skip this section or come back to it later. It's not necessary to understand it to use or train VAEs, but it may help if you want to dive deeper and read papers about the topic.

First, note that we use multidimensional Gaussian distributions, not just real-valued 1D normal curves. In this VAE encoder example, we use `latent_dims` for both the mean and the variance. We could use an arbitrary number of dimensions, like the 16 we used for our first MNIST AutoEncoder, or 2 for easier visualization. A real-valued (1D) normal distribution is denoted as $\mathcal{N}(\mu, \sigma^2)$ and is defined by two magnitudes: μ , the mean of

the distribution, and σ , the *standard deviation*, which is the square root of the *variance* σ^2 .

One useful characteristic of normal distributions is that all of them can be expressed in terms of the *standard* normal distribution, whose mean is 0 and variance is 1, by translating and scaling it:

$$\mathcal{N}(\mu, \sigma^2) = \mu + \sigma \mathcal{N}(0, 1)$$

This means that to obtain a sample from an arbitrary normal distribution $\mathcal{N}(\mu, \sigma^2)$, we can instead sample from $\mathcal{N}(0, 1)$, then multiply by σ and add μ . This is called *reparametrization* and will be quite helpful when we look into diffusion models.

Multidimensional Gaussian distributions are called *multivariate*. They can still be defined by two parameters, with the difference that μ is a vector and σ (the covariance matrix, now denoted with Σ) is a matrix. Hence, the distribution is defined as $\mathcal{N}(\mu, \Sigma)$. If the distribution is independent in all dimensions, meaning each variable is uncorrelated with the others and has the same variance, it is called *isotropic*. In an isotropic multivariate Gaussian distribution, the covariance matrix Σ is a diagonal matrix where all the items in the diagonal are equal and can be expressed as $\sigma^2 \mathbf{I}$, where $\mathbf{I}$ is the

identity matrix. The standard multivariate Gaussian is then expressed as $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Our VAE example above models a multivariate, isotropic Gaussian distribution, as there's no reason to think that sample coordinates depend on each other (and it's simpler!). This means we can use the so-called reparametrization trick to sample from the *standard* Gaussian, then translate and scale to obtain the latent space vector we'll decode.

The reparametrization trick is not just used for convenience – it's a crucial ingredient for training. When we use the expression $\mu + \epsilon \cdot \sigma$, where ϵ is a sample from the standard Gaussian, the gradients with respect to the model inputs are independent of the stochastic process (sampling from a distribution) and can, therefore, be computed. This makes it possible to train the model with the familiar gradient-descent methods we follow to train any neural network.

Speaking of stability, our model predicts the *log* of the variance instead of the variance to increase numerical stability and facilitate training. Mathematically, it makes no difference to compute one or the other. In practice, we know the variance is always a positive number, usually close to 0. However, there's no reason for the model to produce positive and small values

when we start training. Furthermore, numbers are represented in floating point format, which makes it difficult to discriminate values very close together. By taking the *log*, we get two benefits: - We expand the range of acceptable values to $-\infty$, so the model has a lot more latitude to express the results with floating point values. - We ensure the variance is always positive, because it's the exponential of the logvar.

NOTE

Gaussian distributions are frequently used in many other areas of Machine Learning, sometimes for convenience because their mathematical characteristics are well known. In Chapter 4 we'll explore their use to model the noise corruption that is an essential part of diffusion models.

Training the VAE

The key to training the VAE is the loss function. In the AutoEncoder section, the loss function we used measured the difference between the reconstructed and original images. We still want the reconstructed images to resemble the originals as much as possible, but we now introduce a second factor to the loss to impose the VAE constraint we've been talking about: we want the features to (more or less) follow a Gaussian distribution. The way we achieve that goal is by using

something called the *Kullback–Leibler divergence*, also known as *relative entropy*, between the distributions. Kullback-Leibler divergence, or KL divergence (KLD), is a way to measure how much a probability distribution differs from another one. In the case of multivariate isotropic Gaussian distributions, it can be shown that KLD can be computed as:

$$D_{KL}[\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)] = -\frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2)$$

To combine the two loss factors, we create a loss function called `vae_loss` that receives the original images and the outputs from the encoder⁹ and does the following:

- Compute the reconstruction loss as the mean squared error between the pixels generated by the decoder and the original images. This loss factor is identical to the one we used to train AutoEncoders.
- Compute the KLD term following the equation we just presented.
- Add them both together. We could assign more importance to one or the other to balance reconstruction fidelity and conformance to a Gaussian distribution. We'll sum them together for now, but playing with this balance is a great experiment to try.

The loss function returns three values: the total loss, the reconstruction loss, and the KLD term. We only need the total loss for training, but we keep track of the others for visualization and analysis.

```
def vae_loss(batch, reconstructed, mu,
logvar):
    bs = batch.shape[0]

    # Reconstruction loss from the pixels - 1
    per image
    reconstruction_loss = F.mse_loss(
        reconstructed.reshape(bs, -1),
        batch.reshape(bs, -1),
        reduction="none",
    ).sum(dim=-1)

    # KL-divergence loss, per input image
    kl_loss = -0.5 * torch.sum(1 + logvar -
mu.pow(2) - logvar.exp(), dim=-1)

    # Combine both losses and get the mean
    across images
    loss = (reconstruction_loss +
kl_loss).mean(dim=0)
```

```
    return (loss, reconstruction_loss,
            kl_loss)
```

Now that we've defined the loss, we can proceed to train the model. We'll use the total loss to update the model weights, but we'll also keep track of the reconstruction loss and the KLD term to understand how the model is learning.

```
def train_vae(model, num_epochs=10, lr=1e-4):
    model = model.to(device)
    losses = {
        "loss": [],
        "reconstruction_loss": [],
        "kl_loss": [],
    }

    model.train()
    optimizer =
    torch.optim.AdamW(model.parameters(), lr=lr,
                       eps=1e-5)
    for _ in (progress := trange(num_epochs,
                                desc="Training")):
        for _, batch in (
            inner := tqdm(
                enumerate(train_dataloader),
                total=len(train_dataloader)
            )
```

```

):

    batch = batch.to(device)

    # Pass through the model
    reconstructed, mu, logvar =
model(batch)

    # Compute the losses
    loss, reconstruction_loss,
kl_loss = vae_loss(
        batch, reconstructed, mu,
logvar
    )

    # Display loss and store for
plotting
    inner.set_postfix(loss=f"
{loss.cpu().item():.3f}")

losses["loss"].append(loss.item())

losses["reconstruction_loss"].append(
reconstruction_loss.mean().item()
    )

losses["kl_loss"].append(kl_loss.mean().item())

```

```
        # Update model parameters based
on the total loss
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        progress.set_postfix(loss=f"
{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
    return losses
```

```
vae_model = VAE(in_channels=1, latent_dims=2)
```

```
losses = train_vae(vae_model, num_epochs=10,
lr=1e-4)
```

Let's analyze the three loss terms we stored during training:

- **Reconstruction loss:** measures how much the output images resemble the originals.
- **KLD:** measures how well the features follow a Gaussian distribution.
- **Total loss:** The addition of the two previous losses.

```
for k, v in losses.items():
    plt.plot(v, label=k)
```



```
plt.legend();
```

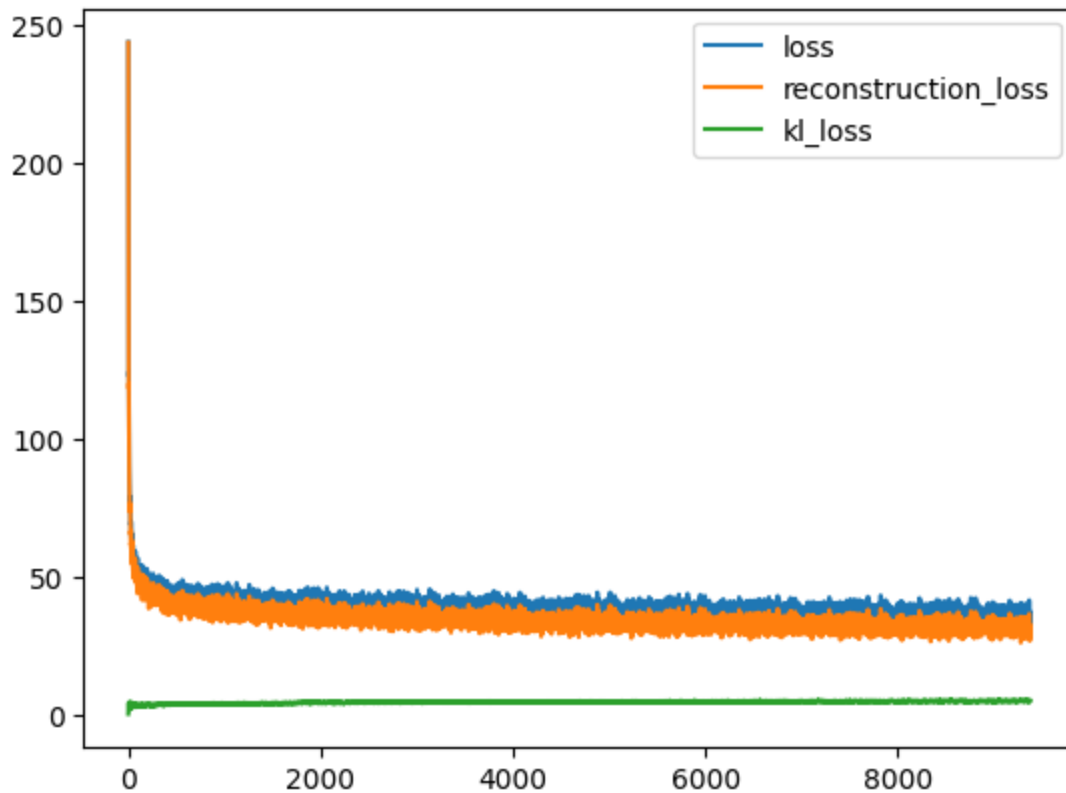


Figure 3-16. Loss components while training a VAE. The total loss is the sum of the reconstruction loss plus the KL term.

Even though the total loss was taken as the sum of the two losses, it is dominated, in this case, by the reconstruction loss because its magnitude is much larger than KLD (Figure 3-16). Let's plot the total loss and its terms separately in Figures 3-17, 3-18, and 3-19 to see how they evolve during training.

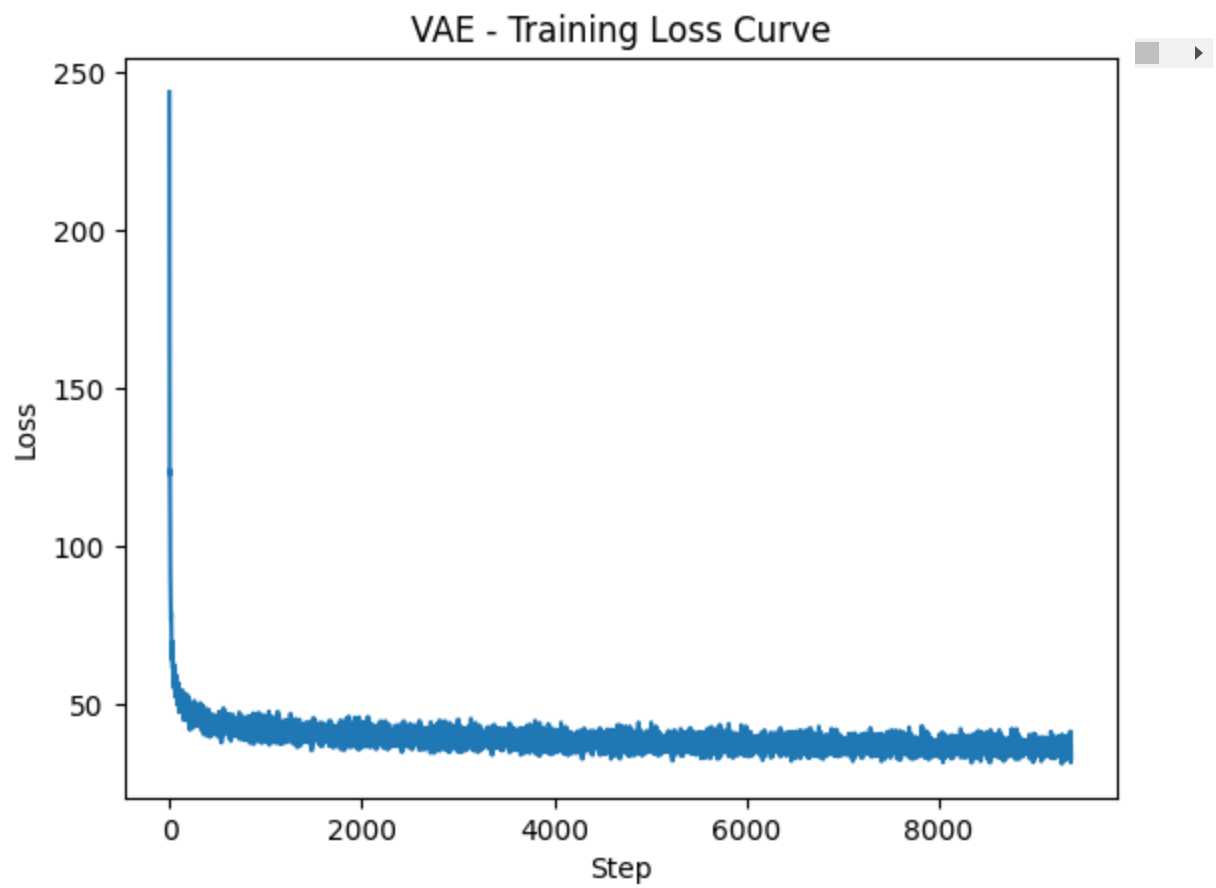


Figure 3-17. VAE loss component during VAE training.

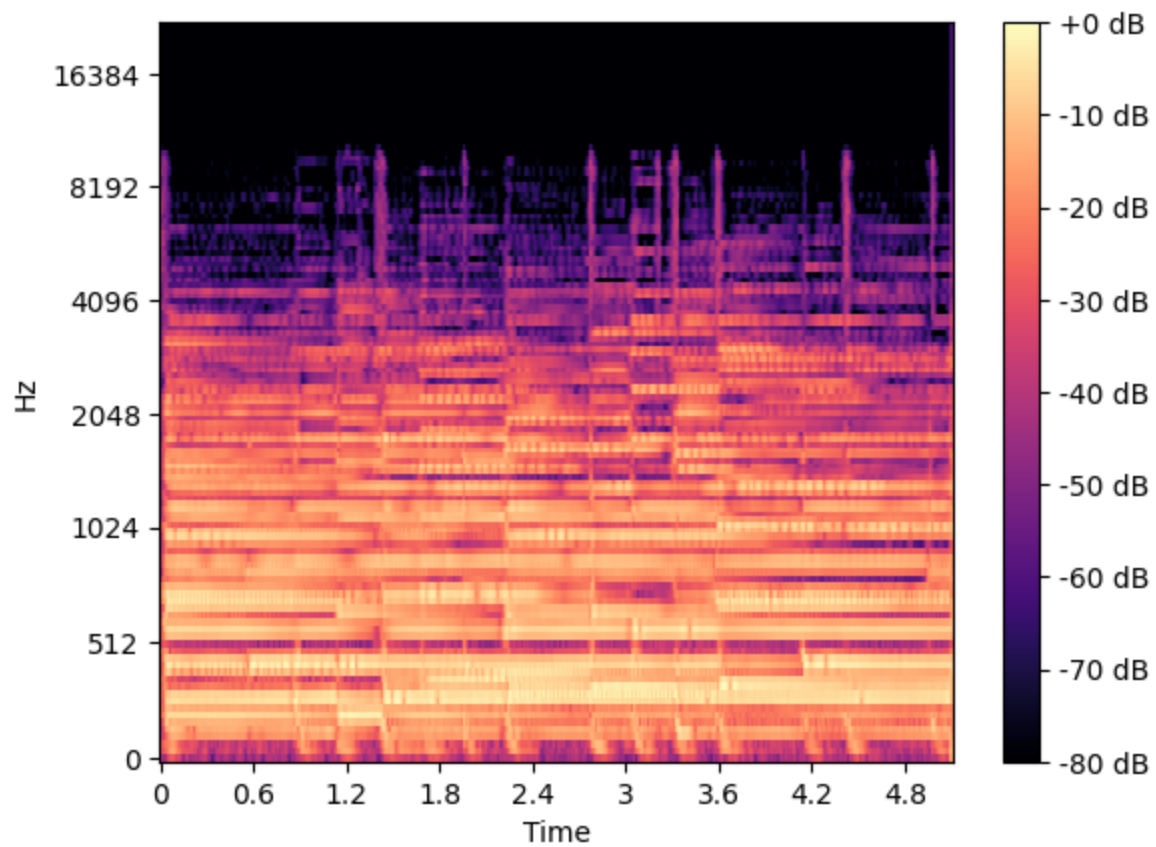


Figure 3-18. KL loss component during VAE training.

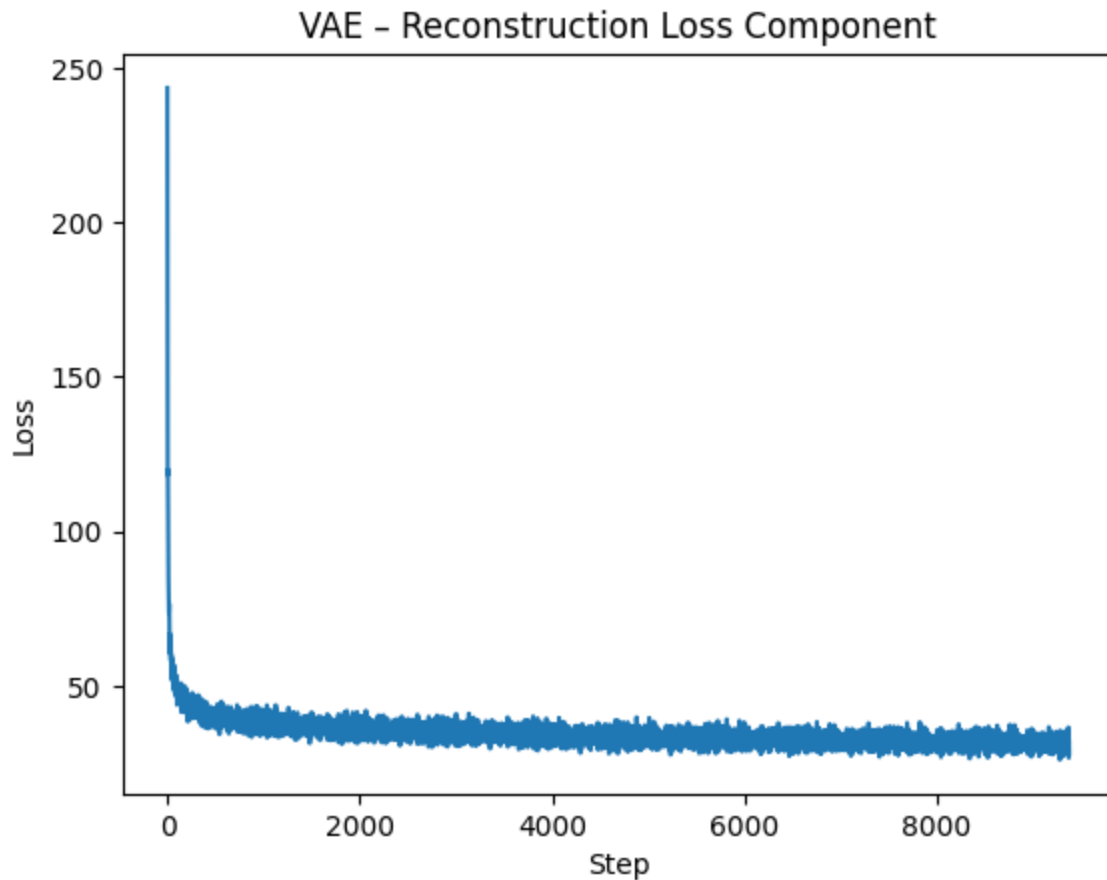


Figure 3-19. Reconstruction loss component during VAE training.

Note it has a peculiar form: it spikes at the beginning of training, then decreases, then slowly *increases* again. Why does that happen? We think these are, conceptually, the phases KLD goes through in training:

- When training starts, the VAE encoder and decoder are initialized with random weights, and the model knows

nothing about the input data. Therefore, the outputs resemble a random distribution, and the KLD is very low.

- When the network has seen just a few batches of data, the reconstructions will be low-quality but not random anymore, and KLD spikes.
- Still in the early training stages, the model knows just enough to represent the average characteristics of the input data. The KLD term drives the model to produce outputs that get closer and closer to a Gaussian distribution, decreasing the KLD loss.
- As the encoder and decoder learn to produce more faithful representations, it becomes harder to improve quality while still matching a Gaussian distribution. The reconstruction loss dominates, and KLD increases, but there's a balance between the two. If we increased the importance of KLD in the loss term, we could achieve better Gaussian conformance, but the cost would be worse pixel representations.

Let's reconstruct some images and visualize the results in Figure 3-20.

```
vae_model.eval()  
with torch.inference_mode():  
    eval_batch = next(iter(eval_dataloader))
```

```
predicted, mu, logvar = (v.cpu() for v in
vae_model(eval_batch.to(device)))
```

```
batch_vs_preds = torch.cat((eval_batch,
predicted))
show_images(batch_vs_preds, imsize=1,
nrows=2)
```

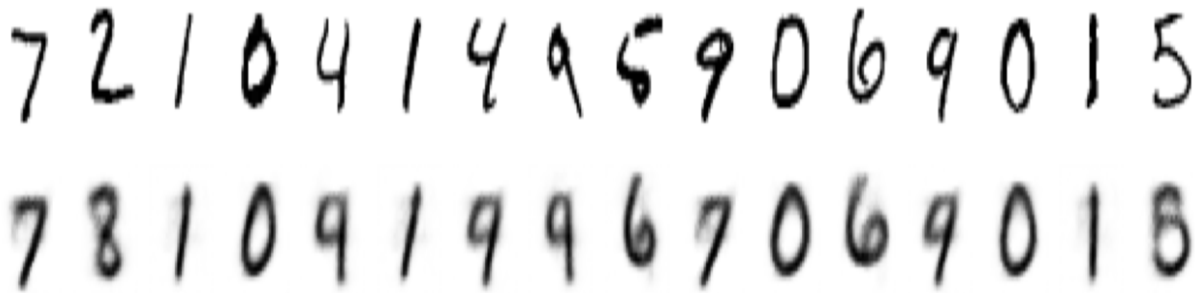


Figure 3-20. VAE reconstructions from MNIST samples.

Visual results are worse than in the AutoEncoder case because the model not only has to learn how to encode the input images but is also constrained to trying to avoid diverging too much from a normal distribution. Let's explore what happened after we added this new goal.

If we plot the means of the standard distribution encoded by the model for the test set, we obtain a slightly better-behaved result than in the AutoEncoder case (Figure 3-21). Results are

now better centered around zero and don't get as far away as in the AutoEncoder case. Areas taken by the different classes are of similar size, although there's still overlap between similar-looking numbers.

```
df = pd.DataFrame(  
    {  
        "x": [],  
        "y": [],  
        "label": [],  
    }  
)  
  
for batch in tqdm(  
    iter(images_labels_dataloader),  
    total=len(images_labels_dataloader)  
):  
    mu, _ =  
    vae_model.encode(batch["image"].to(device))  
    mu = mu.to("cpu")  
    new_items = {  
        "x": [t.item() for t in mu[:, 0]],  
        "y": [t.item() for t in mu[:, 1]],  
        "label": batch["label"],  
    }
```

```
df = pd.concat([df,  
pd.DataFrame(new_items)], ignore_index=True)
```

```
0%|          | 0/20 [00:00<?, ?it/s]
```

```
plt.figure(figsize=(10, 8))  
  
for label in range(10):  
    points = df[df["label"] == label]  
    plt.scatter(points["x"], points["y"],  
label=label, marker=".")  
  
plt.legend();
```

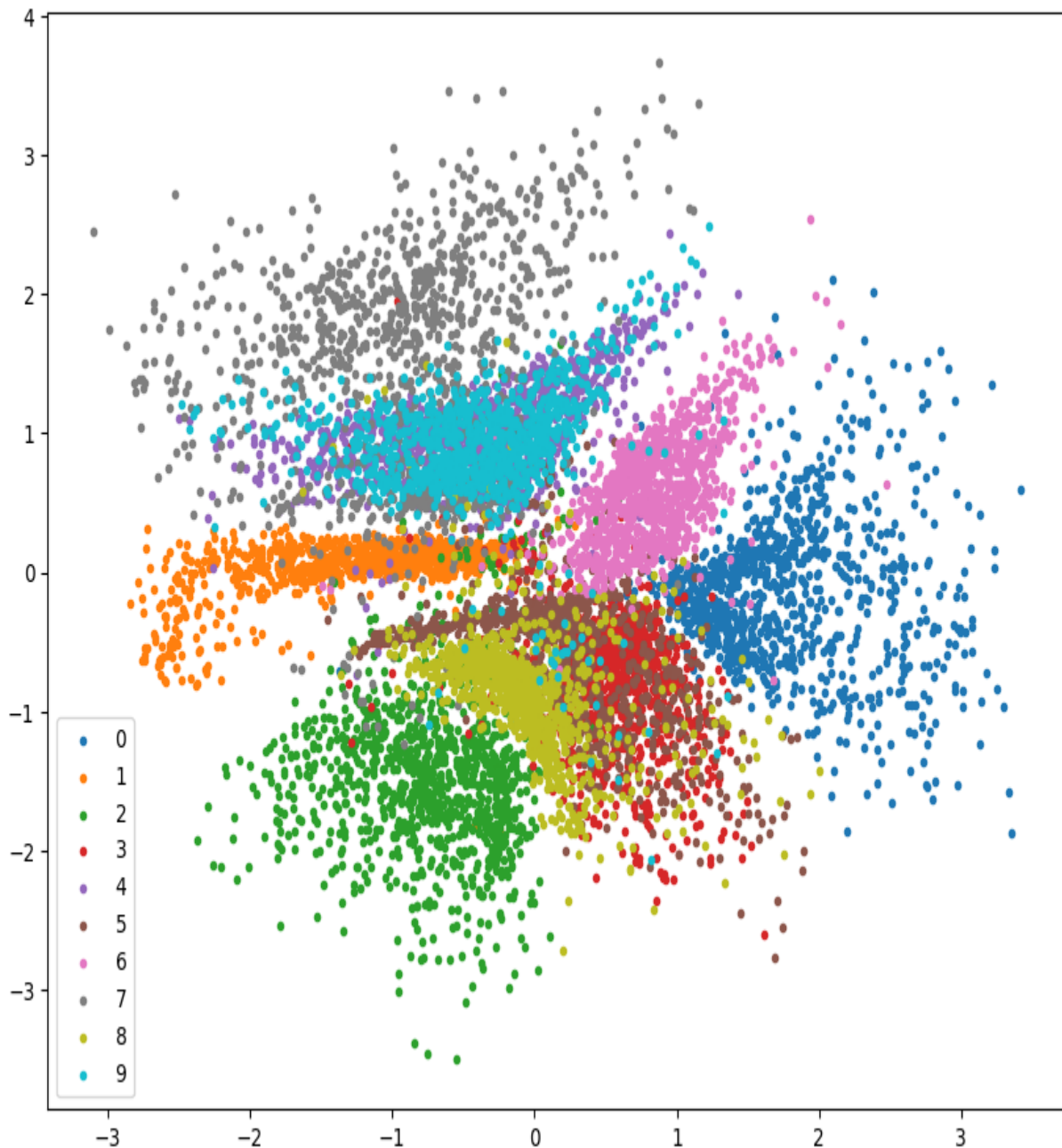



Figure 3-21. Visualization of the VAE latent space.

The main advantage of trying to fit the encoder to a normal distribution is that we should now be able to sample random data from the distribution and, hopefully, obtain images that

resemble our input dataset. Let's see how it works for both the AutoEncoder and the VAE.

```
z = torch.normal(0, 1, size=(10, 2))
ae_decoded = ae_model.decode(z.to(device))
vae_decoded = vae_model.decode(z.to(device))
```

```
show_images(ae_decoded.cpu(), imsize=1,
nrows=1, suptitle="AutoEncoder")
show_images(vae_decoded.cpu(), imsize=1,
nrows=1, suptitle="VAE")
```

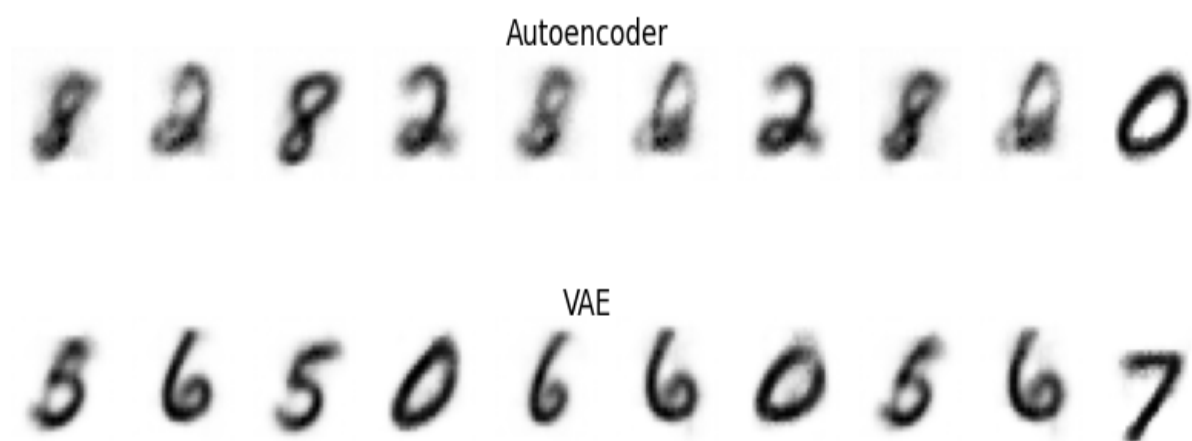


Figure 3-22. AutoEncoder vs VAE generation from normally-distributed features.

We sample pure random data from a normal distribution and then use the AutoEncoder (top row of Figure 3-22) and VAE

(bottom row) decoders to display how those points would be reconstructed. Of course, we will see different reconstructions, as the AutoEncoder and VAE were trained separately, and they allocated different portions of the latent space to each class.

The results from the VAE are more number-like than those of the AutoEncoder. This is because the VAE training process encouraged the encoder to not veer away too much from a normal distribution, while the AutoEncoder had no such restriction. Can you run the sampling code a few times and observe it yourself?

A fun exercise is to show how representations morph as we travel in 2D through latent space. We can fix a vertical line at -0.8 (Figure 3-23) and explore different points in this line. If we select points from $y = -2$ to $y = 2$, we see (Figure 3-24) that the reconstructions match the latent space areas that represent various numbers.

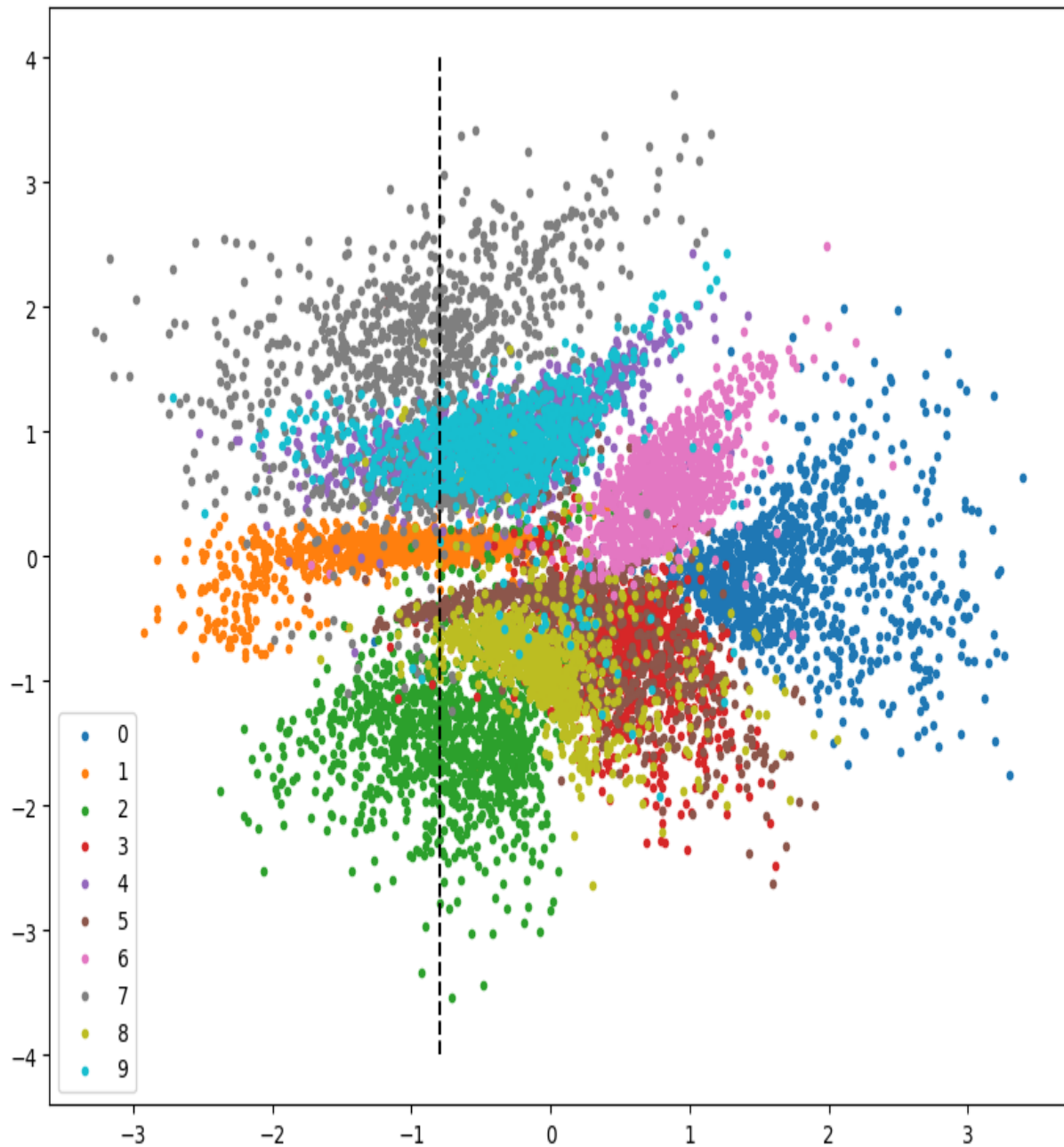


Figure 3-23. VAE latent space: focusing on samples where $x = 0.8$.

```
import numpy as np
```

```

with torch.inference_mode():
    inputs = []
    for y in np.linspace(-2, 2, 10):
        inputs.append([-0.8, y])
    z = torch.tensor(inputs,
dtype=torch.float32).to(device)
    decoded = vae_model.decode(z)
    show_images(decoded.cpu(), imsize=1, nrows=1)

```

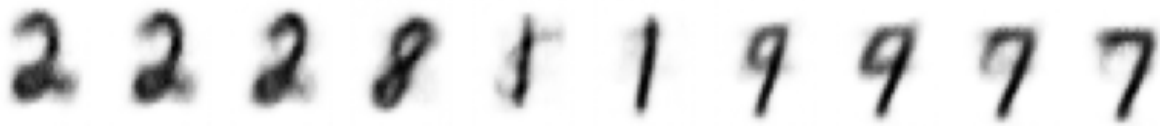


Figure 3-24. Reconstructions from various samples whose $x=0.8$.

Let's expand the idea of exploring the latent space to a 2D grid (Figure 3-25). This is a visual representation of what the model learned, and it's interesting to see that transitions are not too crazy!¹⁰

```

inputs = []
for x in np.linspace(-2, 2, 20):
    for y in np.linspace(-2, 2, 20):
        inputs.append([x, y])
z = torch.tensor(inputs,
dtype=torch.float32).to(device)
decoded = vae_model.to(device).decode(z)

```

```
show_images(decoded.cpu(), imsize=0.4,
nrows=20)
```

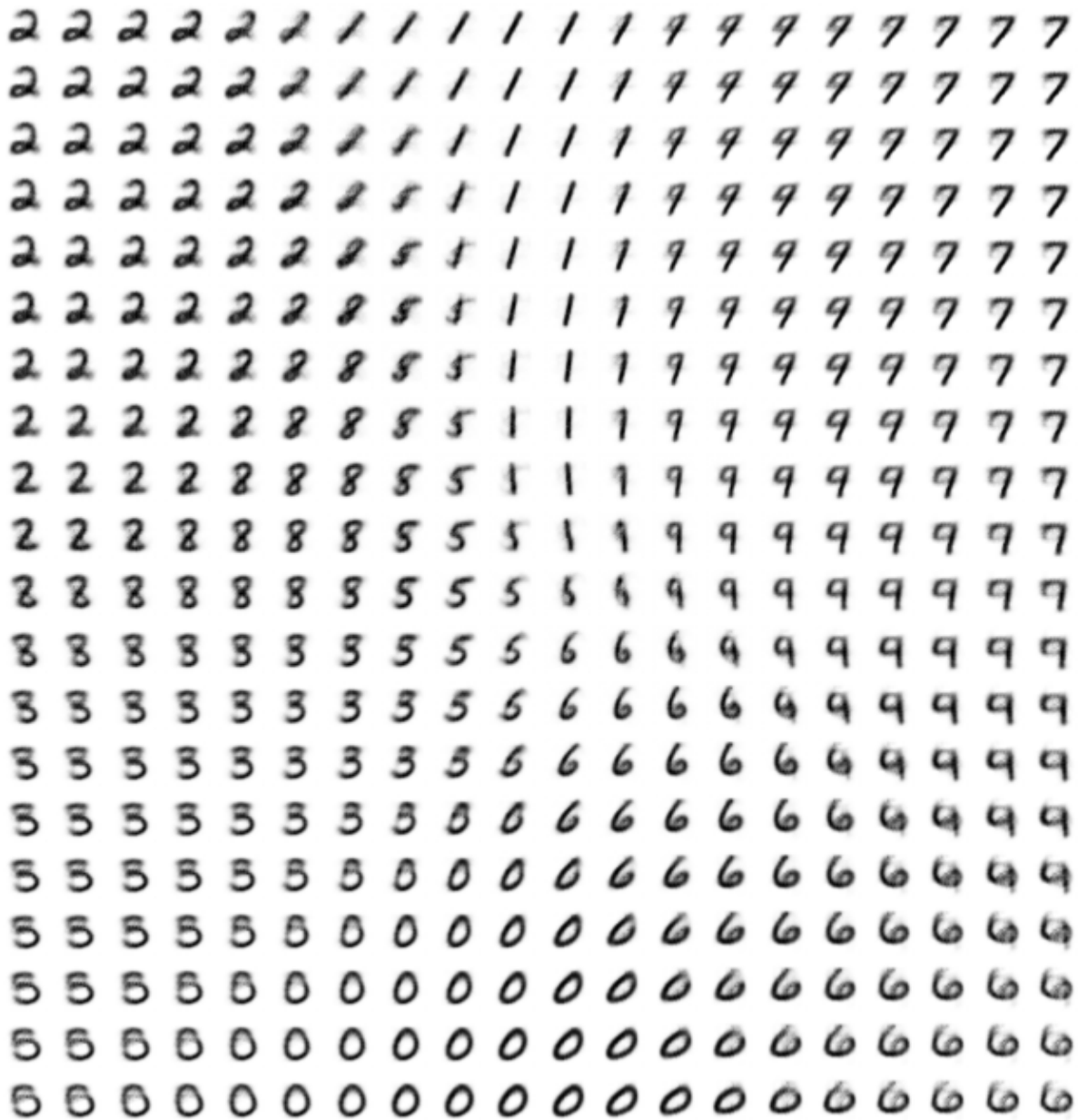


Figure 3-25. 2D exploration of the VAE latent space.

Once again, here are some exercises to dive into these topics:

1. When we trained the VAE, we added the reconstruction and KL-divergence losses. However, both have different scales. What will happen if we give more importance to one vs. the other? Can you run a few experiments and explain the results?
2. The VAE we explored in this section only uses two dimensions to represent the distribution's mean and the *logvar*. Can you repeat a similar exploration using 16 dimensions?
3. Humans are trained to look at faces and easily identify unrealistic features. Can you train an AutoEncoder and a VAE for a dataset containing faces and analyze the results? You can start with the [Frey Face dataset](#) that was used in the VAE paper – it's a homogenous set of monochrome faces from the same person sporting different facial expressions. If you want to be more ambitious, you can try your hand at the [CelebFaces dataset](#), easily usable from the Hugging Face Hub at <https://huggingface.co/datasets/nielsr/CelebA-faces>. Another interesting example is the [Oxford pets dataset](#), also available [on the Hugging Face Hub](#).

VAEs for Generative Modeling

Training an encoder constrained to be close to a distribution is a key insight into VAEs and one of the cornerstones of generative modeling. With AutoEncoder, we could learn efficient representations of a dataset. Still, there was no guarantee that the latent space learned by the model would help us generate new data that resembled the original. By aiming to learn a distribution, VAEs allow us to generate plausible new images, simply starting from random points in the latent space. This idea of sampling from random noise takes a step further in diffusion models, which, in addition to statistical modeling, introduces the concept of iterative refinement. We'll discuss them at length in future chapters.

CLIP

So far, we've focused on image data. With CLIP (Contrastive Language-Image Pre-training), we'll steer away from AutoEncoder/VAE methods and explore a different technique to match images with text. The process is similar in the sense that we aim to create rich representations from the input data, but the method is different, and, more importantly, it can deal with both images and text at once.

Our dataset now consists of two *modalities*: images and text captions describing those images. The goal of CLIP is: given that training data, would it be possible to create a model that measures how accurately a text describes the contents of an image for an arbitrary text-image pair that we supply? Turns out we can! The key, as usual, is the loss function we use.

Contrastive Loss

CLIP was [introduced by OpenAI](#) in 2021¹¹. It was part of the tools they developed to create the initial [DALL·E](#), an impressive text-to-image model that took the world by storm. Even though DALL·E was not open-sourced (the model weights remain private), CLIP was. This was extraordinary news, as the ability to relate images with text enables quite a few tricks. CLIP and CLIP-like models have since become indispensable tools in the generative landscape.

CLIP uses a loss function called *contrastive loss*. The way it works is shown schematically in Figure 3-26, inspired by [OpenAI's CLIP blog post](#). The training dataset consists of millions of images with their associated descriptions or captions. For each image-caption pair, we encode the image using any image encoder to obtain an *embedding vector* in the encoder's latent space. Each of the figure's $I_1, I_2, \dots, I_N$ boxes

represent embedding vectors for different images. The text is also encoded, usually with a transformer model like the ones we saw in Chapter 2. Crucially, we use encoders such that the dimensions of the embedding vectors of images and text are the same. This way, we can calculate the inner product (or dot product) between the text and image embeddings to determine how close they are.

Training progresses by supplying a lot of image-text pairs in the same batch. We compute the dot products of all the image embeddings in the batch with all the text embeddings in the same batch and try to maximize the product of the items originally from the same pair (i.e., the blue diagonal in the image) while minimizing the rest. This way, texts and images that are *similar* will be represented by vectors close in the latent space, while different concepts will be far away in other regions.

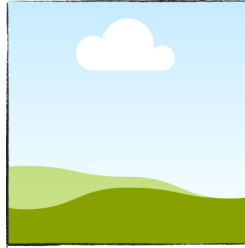


Figure 3-4. Figure 3-26. CLIP: Contrastive Pre-Training Process. Image from <https://openai.com/research/clip>

WHY USE THE DOT PRODUCT?

If you don't remember or haven't studied calculus before, an important relationship that holds in vector spaces is:

$$A \cdot B = |A||B| \cos(\theta)$$

This means that the dot product between two vectors is the same as the product of the lengths of the two vectors multiplied by the angle between them. This can be shown using the [law of cosines](#) from Euclidean geometry and the definition of dot product as $A \cdot B = \sum_{n=1}^N a_i b_i$. In our brief discussion, we didn't mention that the vector embeddings are normalized to unit length; therefore, their dot product is the angle between the vectors. This is called *cosine similarity*, and it measures the proximity between two vectors. Even though the vectors have many dimensions, the dot product is just a scalar (i.e., a real number) which can be used as a similarity score.

Training a CLIP model requires huge amounts of data and lots of compute. The original CLIP models released by OpenAI used a proprietary dataset of 400 million image-text pairs and large batch sizes of 32K pairs. Since then, there have been multiple efforts of other models that can encode different modalities:

- [OpenCLIP](#) is an open implementation of CLIP. It was used to train several models with different datasets, image resolutions, and model sizes.
- [CLAP](#) (Contrastive Language-Audio Pretraining) allows obtaining representations of audio rather than images. This can be used to train models that generate audio, as we'll explore in Chapter 9.

Using CLIP, step by step

We'll now use pre-trained CLIP models to gain intuition about how they work and see a few examples of what they can be used for. In the following example, we use [clip-vit-large-patch14](#), one of the OpenAI CLIP models. It uses a Vision Transformer (ViT) as the image encoder (other versions exist that use the ResNet convolutional architecture). There are larger and smaller versions as well; you can experiment with a few of them to see how they work on your hardware in terms of speed, memory, and quality.

Consider the photo in Figure 3-27, a royalty-free resource from Pixabay showing an adorable lion cub looking at us behind a tree branch. Let's see how we can use it with CLIP.



Figure 3-5. Figure 3-27. Photo of a cute lion cub behind a branch

So far we've only used *transformers* to load text models, but we can also use them to work with other modalities. Same as we had `GPT2LMHeadModel` in the previous chapter, we can use `CLIPModel`. Let's try it out:

```
import requests
from PIL import Image
```

```
from transformers import CLIPModel,
CLIPProcessor

from genai.core import SampleURL

clip =
CLIPModel.from_pretrained("openai/clip-vit-
large-patch14").to(device)
processor =
CLIPProcessor.from_pretrained("openai/clip-
vit-large-patch14")

url = SampleURL.LionExample
image = Image.open(requests.get(url,
stream=True).raw)
```

The CLIP model we loaded, `clip`, contains two components: a vision model to encode images and a text model to encode text. The `CLIPProcessor`, which you can think of as the tokenizer equivalent, prepares input data to match the pre-processing steps used during model training: image resizing, normalization, etc. This is essential to ensure that the inputs we provide for inference have the same characteristics as the data the model saw when it was trained. Let's process the image first.

```
image_inputs = processor(images=image,  
return_tensors="pt")  
pixel_values = image_inputs["pixel_values"]  
pixel_values.shape, pixel_values.min(),  
pixel_values.max()
```

```
(torch.Size([1, 3, 224, 224]),  
tensor(-1.7923), tensor(2.0179))
```

The image has been resized to a square size of 224x224 and normalized. You can examine the image processor for the full set of applied transformations. Note that the strategy used for resizing is to center-crop (i.e., select a square-sized block around the center of the image and then downscale it to 224x224). This method cuts the left and right portions from landscape images or the top and bottom bands from images with a portrait aspect ratio. Be mindful that this may result in information loss if some of the subjects you want to examine are in those areas.

```
processor.image_processor
```

```
CLIPImageProcessor {  
  "crop_size": {
```

```
    "height": 224,
    "width": 224
},
"do_center_crop": true,
"do_convert_rgb": true,
"do_normalize": true,
"do_rescale": true,
"do_resize": true,
"image_mean": [
    0.48145466,
    0.4578275,
    0.40821073
],
"image_processor_type":
"CLIPImageProcessor",
"image_std": [
    0.26862954,
    0.26130258,
    0.27577711
],
"resample": 3,
"rescale_factor": 0.00392156862745098,
"size": {
    "shortest_edge": 224
}
}
```


Let's verify that our lion cub photo can survive a center crop (Figure 3-28).

```
width, height = image.size
crop_length = min(image.size)

left = (width - crop_length) / 2
top = (height - crop_length) / 2
right = (width + crop_length) / 2
bottom = (height + crop_length) / 2

cropped = image.crop((left, top, right,
bottom))
cropped
```



Figure 3-28. Example photo, center-cropped. The subject is intact.

The subject in our photo is fully preserved. Depending on your data, you may need to crop the source images before passing them to the processor to ensure the subject is visible.

We'll now get the embedding vector from the pre-processed image. We use the vision model stored inside the `clip` instance to do that. This sub-component is sometimes called the *vision tower*.

```
with torch.inference_mode():
    output =
clip.vision_model(pixel_values.to(device))
image_embeddings = output.pooler_output
image_embeddings.shape
```

```
torch.Size([1, 1024])
```

The vision model returns a dictionary with the last hidden states and the pooler output. This vector with shape `[1, 1024]` represents the result of the encoding process. Let's now turn our attention to the language portion of the model. We'll follow a similar process to get the embeddings for two text

prompts: "a photo of a lion" and "a photo of a zebra". Our end goal is to compare the cosine similarity between the image embeddings and each prompt embedding. Hopefully, the lion description should match the image better!

```
prompts = [  
    "a photo of a lion",  
    "a photo of a zebra",  
]  
  
# Padding makes sure all inputs have the same  
length  
text_inputs = processor(text=prompts,  
    return_tensors="pt", padding=True)
```

```
{'attention_mask': tensor([[1, 1, 1, 1, 1, 1,  
    1],  
    [1, 1, 1, 1, 1, 1, 1]]),  
  'input_ids': tensor([[49406, 320, 1125,  
    539, 320, 5567, 49407],  
    [49406, 320, 1125, 539, 320,  
    22548, 49407]])}
```

Text processing tokenizes the input strings just as in the previous chapter. We can now pass the tokenized text inputs

through the *language tower* part of the model to get the prompt embeddings.

```
text_inputs = {k: v.to(device) for k, v in
text_inputs.items() }
```

```
with torch.inference_mode():
    text_output =
clip.text_model(**text_inputs)
```

```
text_embeddings = text_output.pooler_output
text_embeddings.shape
```

```
torch.Size([2, 768])
```

We got two vectors in the output using a batch of two input prompts. However, each vector has 768 dimensions, while the embeddings from the image had 1024 dimensions. Remember that to compute the dot product of two vectors, they have to have the same number of dimensions, and we insisted in the introduction to this section that both the text encoder and the image encoder must produce embeddings with the same dimensionality. There's an additional step we didn't mention before. Instead of selecting encoder models that produce the

same dimensions, we can take arbitrary text and image encoders and compute a projection to vectors with the same dimensionality. These projections were learned during the CLIP training process and are also part of the `clip` model wrapper we downloaded before.

In this case, the learned projections are just linear layers that map their inputs to vectors with 768 dimensions. There's a projection for the text encoder and a different one for the vision part of the model.

```
print(clip.text_projection)
print(clip.visual_projection)
```

```
Linear(in_features=768, out_features=768,
bias=False)
Linear(in_features=1024, out_features=768,
bias=False)
```

```
with torch.inference_mode():
    text_embeddings =
clip.text_projection(text_embeddings)
    image_embeddings =
```

```
clip.visual_projection(image_embeddings)
text_embeddings.shape, image_embeddings.shape
```

```
(torch.Size([2, 768]), torch.Size([1, 768]))
```

We are almost ready to compute the cosine similarities. We just need to remember to use normalized vectors with unit norms, which we achieve by scaling our embeddings and dividing by their respective norms. We can then compute the two dot products at once using matrix multiplication.

```
text_embeddings = text_embeddings /
text_embeddings.norm(
    p=2, dim=-1, keepdim=True
)
image_embeddings = image_embeddings /
image_embeddings.norm(
    p=2, dim=-1, keepdim=True
)
```

```
similarities = torch.matmul(text_embeddings,
image_embeddings.T)
similarities
```

```
tensor([[0.2171],  
        [0.1888]], device='cuda:0')
```

During training, these cosine similarities were interpreted as the logits to be fed to a cross-entropy loss classifier that predicts the label for each image-text pair. Since CLIP training used 32K-sized batches, labels are each one of the 32768 positions in the batch. If you refer again to the CLIP image at the beginning of the section, once the model has been trained, the cross-entropy process will select " `class` " T_1 for I_1 , class T_2 for I_2 , and so on.

There's a final detail, though. Because the vectors are normalized, logits can only lie within the range `[-1, 1]`, which, due to floating point format limitations, may not have enough dynamic range to expressively capture the categorical probability distributions of the 32K items. The authors used a learnable *temperature* parameter to scale the logits to have a wider range. However, they also clipped this scale value to a maximum of `100` for numerical stability. In all the training runs, they found that the scale always reached the maximum value of 100. Therefore, CLIP inference uses a scale factor of 100 before interpreting the logits as probabilities.

Let's apply that correction to our previous code snippet, and then we can convert the scaled similarity logits to probabilities.

These probabilities represent how confident the model is that the image corresponds to one of the two text captions we used.

```
similarities = 100 *  
torch.matmul(text_embeddings,  
image_embeddings.T)  
similarities.softmax(dim=0).cpu()
```

```
tensor([[0.9441],  
        [0.0559]])
```

The model matches the prompt "a photo of a lion" with the image, with a confidence of 94.4%.

Zero-shot Image Classification with CLIP

The process we followed in the previous section is a detailed walkthrough of the steps needed to implement a *zero-shot classification task* with CLIP. Fortunately, software libraries such as Transformers or OpenCLIP provide higher layers of abstraction that make the process much easier.

WHY IS THIS CALLED ZERO-SHOT CLASSIFICATION?

Classification is one of the quintessential Machine Learning problems: given a data point and a set of predefined classes, estimate the probability that the point corresponds to one of the classes. According to this definition, the set of classes must be fixed in advance, so the model we train will only know about these classes and nothing else. The ImageNet dataset, for example, contains images from 20,000 different classes. For years, the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) was a test benchmark for Computer Vision systems, with the goal of classifying objects belonging to *just 1,000* of the ImageNet classes. In 2012, a deep CNN known as AlexNet easily won that year's challenge, and this started a revolution where accuracy figures increased year after year as newer and better deep learning models were designed.

Zero-shot classification refers to the capability of a model to correctly classify data without having been trained explicitly for the classes we are asking for. In the previous chapter, we saw an example of zero-shot sentiment classification using language models, and the previous section is another prime demonstration of this capability. CLIP was trained to match image-caption pairs, but we can leverage this behavior for classification if we construct captions that could reasonably describe the images we want to classify. For example, if we want to classify cats vs. dogs, our prompts can be “A photo of a cat” and “A photo of a dog.” CLIP will match the best prompt for the image we supply, giving us the classification result we're after.

Let's replicate the example in the previous section with a higher-level transformers API. We load the CLIP model, processor, and test image the same way we did before.

```
clip =  
CLIPModel.from_pretrained("openai/clip-vit-
```

```
large-patch14").to(device)
processor =
CLIPProcessor.from_pretrained("openai/clip-
vit-large-patch14")

url =
"https://cdn.pixabay.com/photo/2014/12/12/19/45/1
565820_1280.jpg"
image = Image.open(requests.get(url,
stream=True).raw)
```

We can leverage the processor to compute the inputs for both the image and the text prompts simultaneously. We can also conveniently invoke the clip model with the full set of inputs to retrieve the logits—or scaled cosine similarities—between the image and each prompt. Let's use a few more prompts to make things more fun.

```
prompts = [
    "a photo of a lion",
    "a photo of a zebra",
    "a photo of a cat",
    "a photo of an adorable lion cub",
    "a puppy",
    "a lion behind a branch",
]
```

```
inputs = processor(  
    text=prompts, images=image,  
    return_tensors="pt", padding=True  
)  
inputs = {k: v.to(device) for k, v in  
    inputs.items()}  
  
outputs = clip(**inputs)  
logits_per_image = outputs.logits_per_image  
probabilities =  
    logits_per_image.softmax(dim=1)
```

```
probabilities =  
probabilities[0].cpu().detach().tolist()
```

```
for prob, prompt in sorted(zip(probabilities,  
    prompts), reverse=True):  
    print(f"{100*prob: =2.0f}%: {prompt}")
```

```
89%: a photo of an adorable lion cub  
9%: a lion behind a branch  
2%: a photo of a lion  
0%: a photo of a zebra  
0%: a photo of a cat  
0%: a puppy
```

Similarly, we can supply multiple images and prompts in the same input batch and get all classification probabilities simultaneously. Feel free to explore and adapt to your use case.

Zero-shot Image Classification Pipeline

Now that we know how CLIP works and how to use it for zero-shot image classification, we can use an even higher-level API for simplicity and convenience. We'll use the `pipeline` abstraction, which we already presented in Chapter 1. In that case we demonstrated how to use it for the text classification task, but we can also apply it for many other tasks, including zero-shot image classification.

To instantiate a pipeline, we simply give it the task name (`zero-shot-image-classification`, in this case), and the model we want to use.

```
from transformers import pipeline

classifier = pipeline(
    "zero-shot-image-classification",
    model="openai/clip-vit-large-patch14",
    device=device,
)
```

The pipeline takes care of all the details for us: tokenization, image pre-processing, logits post-processing. We just need to invoke the pipeline instance with the image we want to classify, and a set of candidate labels. The pipeline returns a dictionary, conveniently sorted by score, containing all the scores associated to the labels we provided.

```
scores = classifier(  
    image,  
    candidate_labels=prompts,  
    hypothesis_template="{}",  
)
```

The `+hypothesis_template+` is a Python format string that is applied to each candidate label to build the text prompt for classification. If we omit it, the pipeline will automatically use `+"This is a photo of a {}"+`, which is appropriate to format class labels indicated by their name, such as `"cat"` or `"lion"`. Since we already built prompts that work well with CLIP, we use `+"{}"+` to use our labels untouched.

```
[{'label': 'a photo of an adorable lion cub',  
  'score': 0.886413037776947},  
 {'label': 'a lion behind a branch', 'score':  
  0.09321863204240799},
```

```
{'label': 'a photo of a lion', 'score':  
0.018809959292411804},  
{'label': 'a photo of a zebra', 'score':  
0.0011134858941659331},  
{'label': 'a photo of a cat', 'score':  
0.0004198708338662982},  
{'label': 'a puppy', 'score':  
2.4912407752708532e-05}]
```

CLIP Use Cases

The original use case that CLIP was designed to solve is zero-shot image classification. The results are impressive: it achieves similar performance as models trained for ImageNet classification, without ever using the ImageNet labels during training. As a consequence, performance remains equally strong on many other datasets, with no need for fine-tuning on them, as you can read in the [original blog post](#).

The previous sections have shown that the basis for solving zero-shot image classification is the ability to compute the similarity between an arbitrary image and a text prompt, and this is possible because both the image and the text embeddings capture the essential semantics of the data. The *way* CLIP works

has enabled the community to use it for many tasks, not just zero-shot classification.

Being able to compute the **similarity between text and images** enables applications such as *semantic search*, which makes it possible to search for photos based on natural-language descriptions of their contents or find images similar to an example image we provide. Applications of these techniques exist in multiple domains, including consumer hardware (such as phones), medical systems, fashion, and others. In the previous chapter, we proposed a similar challenge, but is constrained to textual data: building an FAQ system by computing similarities between the embedding outputs of a language model. At the end of this chapter we propose a challenge using CLIP for semantic search.

CLIP can also be used to obtain **rich embeddings** for downstream tasks. For example, some text-to-image models use CLIP to obtain semantically rich representations of the prompts supplied by the user.

CLIP was also adopted as an essential tool for generative use cases. The CLIP Guidance method, developed by [Ryan Murdock](#), [Katherine Crowson](#), and others, uses CLIP as a loss to guide model gradients towards the desired representation (expressed

with a prompt). It spawned a creative explosion in the generative art community. Later, CLIP conditioning became essential in models such as Stable Diffusion, which we'll explore in Chapter 5.

CLIP scoring capabilities have also been used to filter and score massive datasets of image-caption pairs, such as [LAION](#), crawled from Internet sources. Relying on CLIP allows dataset creators to select pairs where the similarity between the image and the caption yields a high score and discard the others. This has made it possible to build and refine datasets in the order of billions of items, which can be used to build better models that can refine datasets even more precisely. So meta!

Alternatives to CLIP

Because CLIP is extensively used in industry and research, there has been much activity around the ideas presented in the model and how to make them better, faster, or adapted to other tasks. A significant effort has been made to make CLIP fully open-source and reproducible. The [OpenCLIP repository](#) provides open-source code to implement CLIP and a number of architecture variants. Using the OpenCLIP codebase and the huge LAION dataset we mentioned in the previous section, the

LAION team trained very powerful CLIP models of various sizes and made all the checkpoints available for everyone to use.

Recent research has shown that better text understanding can lead to better text-image models. BLIP¹², CoCa¹³ and CapPa¹⁴ demonstrate that the captioning task (generate a detailed description of what an image represents) can produce models that are capable of generating excellent image representations and solve a wide range of vision-language tasks. In parallel research, the use of a different loss function (sigmoid loss used in SigLIP¹⁵, instead of softmax normalization) can make training easier and alleviate CLIP's problem of requiring huge batch sizes to train effectively.

Another promising direction is building smaller, faster models that can run on personal computers and mobile devices. This is the case with Apple's MobileCLIP¹⁶, which achieves the performance of OpenAI's CLIP models with much smaller (and faster) models. Another Apple effort, Data Filtering Networks¹⁷, aims to improve the quality of text-image datasets and also trained several CLIP variants with those methods.

As we'll see throughout the rest of this book, CLIP is an essential component of image generation systems. This healthy research

on more robust, capable, and faster CLIP-like models makes us very optimistic about the future

Project Time: Semantic Image Search

A fun project is building a semantic search engine for your photos. When you are done, you should be able to look for photos in your library by simply describing their contents (for example: "dog jumping into the water on a hot summer day" or "woman with umbrella walking down a busy street"), instead of trying to remember where in your collection those photos are stored. You can also use any other image dataset you like, but using content that means something to you will be rewarding and allow you to evaluate how well the system works. It may also make you want to think of ideas for improvement or share them with your family.

These are some suggested steps to tackle the project:

- Choose a text-image model, such as CLIP, that can produce embeddings for both images and text descriptions. You can explore other alternatives, but CLIP should be a good start. Choose a family of models with multiple-size variants that you can easily replace with one another. This way, you can

use a small model to iterate faster while working and see how much performance increases when you use a larger model.

- Find a good number of photos and copy them all to a folder on your computer. Several hundred or thousand photos should be fine.
- Write a loop to create embeddings from your photos using the model you chose.
 - Read the photos from disk, crop and/or resize them so they are the same size, and create a batch. You can use a PyTorch `DataLoader`, as we did in the training loops in this chapter. For pre-processing, you may do it manually with `torchvision.transforms`, or you can leverage the model's built-in pre-processor if it exists. Choose a batch size that fits your hardware.
 - Run each batch of images through the image portion of the CLIP model. Use inference mode (no gradients need to be computed, as you won't be training anything).
 - Get the embeddings from the output and save them to disk. You'll get a multidimensional vector for each image file. You can convert them to *numpy* arrays and write them all into the same file. Don't forget to store the names or paths to the original photos, you'll need them to retrieve the photos later.

- At this point, you have an array of vectors in *numpy* format. You can now use the text portion of the model to run queries:
 - Write a function that receives an input prompt and generates an embedding vector using the text *tower* of the model.
 - Compute the cosine similarity between that vector and *all the vectors in the image embedding table*. If you have enough RAM, you can do it by simply using PyTorch's `matmul` operation.
 - Sort the outputs and select the top ones.
 - Find the images associated with the top scores and visualize them. Do they match the prompt you used?

Bonus tasks:

- Can you try to find images that *look similar* to another image? (i.e., semantic search based on an input image, not a text description).
- If you have a lot of photos, you may have trouble computing the scores. How could you solve this problem? Are there any frameworks or services that help with this? How many photos does it take to reach the limits of your computer?

- Pre-trained models don't know anything about the subjects that appear in your photos. What could you do to be able to search by personal names or places?
- If you are adventurous, you can use MobileCLIP to run the search engine on your phone. This is a big challenge in itself; don't underestimate the effort!

Summary

This chapter showed how learning compressed representations from input data is a way to capture the dataset's essential characteristics and how those representations can be effectively used for many additional downstream tasks. We started this exploration by looking at a classical system, the AutoEncoder, whose goal is to encode input samples into a latent space of reduced dimensionality and then recover the original data points from the latent representations. By splitting the AutoEncoder into two components –the encoder and the decoder–we can imagine new applications beyond reconstruction. The encoder, for example, can be used as a feature extractor. Because it learned the essential features of the input dataset, we can use it to train other systems, such as classifiers, whose input data are latent representations. We also explored the idea of using the decoder for generative purposes.

If the latent space is a representation of the original dataset, can we move to arbitrary points there and see what outputs we get? AutoEncoders, however, have some limitations for this task because of how they are trained.

VAEs are a special kind of AutoEncoder that tries to achieve "better behaved" representations in latent space. By trying to make latent features match a probability distribution, we can sample from the desired distribution to obtain new random latent features. If we feed those features to the decoder, we can generate data points that look like they came from the original dataset. This is a crucial result for generative applications. Exploring this further, the fact that the latent space is a compact representation of the data is an essential idea behind generative systems such as Stable Diffusion. As we'll see in more detail in Chapter 5, we can conduct computation in the latent space that represents images rather than on the raw image data. This enabled the training of high-quality image generation systems that can run fast and efficiently on consumer hardware.

The final section of the chapter focuses on CLIP, a highly influential model developed and published by OpenAI that encodes image and text data into the same latent space. New tricks are possible with CLIP that were pretty hard problems to

solve. For example, given an image and a few sentences, we can measure which sentence matches the image better. Conversely, given a caption and a few image candidates, we can select the image that best matches the caption. CLIP was published as part of OpenAI's Dall·e image generation project (which was not published itself), and it spawned a revolution in generative research. CLIP-like models are key components of Stable Diffusion and other text-to-image models, but they are used for many other applications: natural-language image retrieval, semantic search, extracting images that are similar based on content or style, and a lot more.

All of this came about from the initial realization that learning how to compress data is equivalent to learning about the data. Many variations on these concepts use different types of data extraction and representation techniques: CNNs, transformers, or combining the best of both with systems like VQGAN. We just offered a glimpse of the motivations and ideas behind these foundational blocks with the hope that they will be useful for navigating this rich and fascinating space.

Exercises

Most of these exercises are the same ones we proposed during the chapter, compiled here for your convenience. Depending on your learning style, you may like to work on them as you go through the chapter, or you may try them all after a first read.

1. How does generation work if the AutoEncoder model is trained with 16 latent dimensions? Can you compare generations between the model with 16 latent dimensions and the one with just 2?
2. Train the model again with the same parameters we used (just run the code shown in the chapter) but with different random number initialization, and visualize the latent space. Chances are that the shapes and structure are different. Is this something you would expect? Why?
3. How good are the image features extracted by the encoder? Explore it by training a number classifier on top of the encoder.
4. When we trained the VAE, we added the reconstruction and KL-divergence losses. However, both have different scales. What will happen if we give more importance to one vs. the other? Can you run a few experiments and explain the results?

5. The VAE we trained only uses two dimensions to represent the mean and the *logvar* of the distribution. Can you repeat a similar exploration using 16 dimensions?
6. Humans are trained to look at faces and easily identify unrealistic features. Can you try to train an AutoEncoder and a VAE for a dataset containing faces, and see what the results look like? You can start with the [Frey Face dataset](#) that was used in the VAE paper – it's an homogenous set of monochrome faces from the same person sporting different facial expressions. If you want to be more ambitious, you can try your hand at the [CelebFaces dataset](#), also hosted on the Hugging Face [Hub](#). Another interesting example could be to try the [Oxford pets dataset](#), also available [on the Hub](#).

Challenge

7. BLIP-2 for search. The hands-on project on semantic image search is quite challenging, but here's another idea. Can you use the [BLIP-2 model](#) for similarity tasks, just like we did with CLIP in this chapter? How would you go about it, and how does it compare with CLIP? What other tasks can you solve with BLIP-2?

You can find the solutions to these exercises in the [supplementary material](#).

References

1. Chiaki Yanagisawa. *Conv2d and ConvTransposed2d*.
https://indico.cern.ch/event/996880/contributions/4188468/attachments/2193001/3706891/ChiakiYanagisawa_20210219_Conv2d_and_ConvTransposed2d.pdf
2. Clément Chadebec. [pythae](#), a unified implementation of multiple Autoencoder models, that allows easy experimentation, comparison and benchmarking.
3. Stanford University. *CS231n Convolutional Neural Networks for Visual Recognition*. Explains convolutions at <https://cs231n.github.io/convolutional-networks/#conv>.
David Foster. *Generative Deep Learning (2nd Edition)*.
Published by O'Reilly Media, Inc. [Chapter 3. Variational Autoencoders](#).
4. Esser, Patrick, Robin Rombach, and Bjorn Ommer. *Taming transformers for high-resolution image synthesis*. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 12873-12883. 2021.
[arXiv preprint](#)

5. Fang, Alex, et al. *Data Filtering Networks*. arXiv, 29 Sept., 2023. arXiv.org, <https://arxiv.org/abs/2309.17425>
6. François Chollet. *Variational AutoEncoder*. Keras implementation of a VAE.
<https://keras.io/examples/generative/vae/>
7. François Floret, *_Deep Learning Course _*(2024 version). Available at <https://fleuret.org/dlc/>, revised VAE handouts (PDF) at <https://fleuret.org/dlc/materials/dlc-handout-7-4-VAE.pdf>.
8. Howard, Jeremy, and Gugger, Sylvain. *Deep Learning for Coders with fastai & PyTorch*. O'Reilly 2020.
9. Johannes Maucher. *Animations of Convolution and Deconvolution*.
<https://hannibunny.github.io/mlbook/neuralnetworks/convolutionDemos.html>.
10. Jonathan Whitaker. *A Deep Dive Into OpenCLIP from OpenAI*. <https://wandb.ai/johnnowhitaker/openclip-benchmarking/reports/Exploring-OpenCLIP-VmldzoyOTIzNzIz>
11. Kingma, Diederik P., and Max Welling. *Auto-Encoding Variational Bayes*. arXiv, 20 Dec., 2023. arXiv.org, <https://arxiv.org/abs/1312.6114>
12. LAION (various authors). [Large Scale OpenCLIP trained on LAION-2B](#).

13. Li, Junnan, et al. *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*. In International conference on machine learning, pp. 12888-12900. PMLR, 2022. [arXiv preprint](#).
14. Li, Junnan, et al. *BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models*. In International conference on machine learning, pp. 19730-19742. PMLR, 2023. [arXiv preprint](#).
15. ML Foundations (various authors). [OpenCLIP](#).
16. [pytorch-mnist-vae](#), a VAE PyTorch implementation that follows the paper.
17. PyTorch team. *PyTorch VAE example*.
<https://github.com/pytorch/examples/tree/main/vae>
18. Radford, Alec, et al. Learning Transferable Visual Models From Natural Language Supervision. In International conference on machine learning, pp. 8748-8763. PMLR, 2021. Predicting which caption goes with which image is enough to learn powerful image representations. [arXiv preprint](#)
19. Stefano Ermon, et al. *CS 228 - Probabilistic Graphical Models*. [The variational auto-encoder](#).
20. Tschannen, Michael, et al. *Image captioners are scalable vision learners too*. Advances in Neural Information Processing Systems 36 (2024). [arXiv preprint](#).

21. Vasu, Pavan Kumar Anasosalu, et al. *MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training*, CVPR 2024, [arXiv preprint arXiv:2311.17049](https://arxiv.org/abs/2311.17049). Code repository: <https://github.com/apple/ml-mobileclip>
22. Yu, Jiahui, et al. *Coca: Contrastive captioners are image-text foundation models*. [arXiv preprint arXiv:2205.01917](https://arxiv.org/abs/2205.01917) (2022).
23. Zhai, Xiaohua, et al. Sigmoid Loss for Language Image Pre-Training. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 11975-11986. 2023. [arXiv preprint](https://arxiv.org/abs/2301.00596).

`torchvision transforms`, as we'll see later in the book, is a collection of common image transformation, conversion, augmentation, and manipulation routines.

For an in-depth discussion of these and many other design choices, as well as an excellent and practical overview of deep learning, we recommend *Deep Learning for Coders with fastai and PyTorch*, by Jeremy Howard and Sylvain Gugger, available at <https://www.oreilly.com/library/view/deep-learning-for/9781492045519>

In eval mode, `BatchNorm2d` applies the mean and standard deviation learned from all the mini-batches during training. Since we haven't trained the model yet, this will be random data, but we are only interested in seeing if the model definition works. During actual training, we use a batch size larger than 1, and `BatchNorm2d` will work fine.

A transposed convolution works just like a convolution, but instead of applying it to the 2D input data, it is applied to an enlarged version of it (the 2D input is filled with zeros between the rows and the columns). This results in an output 2D matrix that is

larger than the input after the filter is applied. [This blog post](#) shows excellent visualizations of how convolutions and transposed convolutions work.

You can use `torch.manual_seed(num)` to specify a seed

A Gaussian distribution, also called normal, has a bell-shaped curve with most values clustered around the mean and fewer values at the extremes

There's a family of VAEs, called conditional VAEs (or C-VAEs), that uses class information to further separate distributions in latent space while still keeping Gaussian representation of the features. This makes it easier to generate samples resembling the specific class we're interested in.

Actually, not really the variance, as we'll see shortly.

Remember that the VAE encoder returns not only the reconstructed images but also the mean and the log variance of the distributions. This is the reason why.

This is a rather crude way to show the *learned manifold* of the model. For a better way to display this result and additional experiments, we recommend this GitHub repo by Jackie Loong: <https://github.com/dragen1860/pytorch-mnist-vae>

Radford, Alec, et al. *Learning Transferable Visual Models From Natural Language Supervision*. In International Conference on Machine Learning, pp. 8748-8763. PMLR, 2021.

Li, Junnan, et al. *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*. In International Conference on Machine Learning, pp. 12888-12900. PMLR, 2022. [arXiv preprint](#).

Yu, Jiahui, et al. *Coca: Contrastive captioners are image-text foundation models*. [arXiv preprint arXiv:2205.01917](#) (2022)

- 1 Tschannen, Michael, et al. *Image captioners are scalable vision learners too*. Advances in Neural Information Processing Systems 36 (2024). [arXiv preprint](#)
- 2 Zhai, Xiaohua, et al. Sigmoid Loss for Language Image Pre-Training. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 11975-11986. 2023. [arXiv preprint](#)
- 3 Vasu, Pavan Kumar Anasosalu, et al. *MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training*, CVPR 2024, [arXiv preprint arXiv:2311.17049](#). Code repository: <https://github.com/apple/ml-mobileclip>
- 4 Fang, Alex, et al. *Data Filtering Networks*. arXiv, 29 Sept., 2023. arXiv.org, <https://arxiv.org/abs/2309.17425>

Chapter 4. Diffusion Models

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the fourth chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at jleonard@oreilly.com.

The field of image generation became widely popular with Ian Goodfellow’s introduction of Generative Adversarial Nets (GANs) in 2014. The key ideas of GANs led to a big family of models that could quickly generate high-quality images. However, despite their success, GANs posed challenges, requiring many parameters and help to generalize effectively. These limitations sparked parallel research endeavors, leading

to the exploration of diffusion models—a class of models that would redefine the landscape of high-quality, flexible image generation.

In late 2020, a little-known class of models called diffusion models began causing a stir in the Machine Learning world. Researchers figured out how to use these diffusion models to generate higher-quality images than those produced by GANs. A flurry of papers followed, proposing improvements and modifications that pushed the quality up even further. By late 2021, models like GLIDE showcased incredible results on text-to-image tasks. Just a few months later, these models had entered the mainstream with tools like DALL-E 2 and Stable Diffusion. These models made it easy for anyone to generate images just by typing in a text description of what they wanted to see.

In this chapter, we will dig into how these models work. We'll outline the key insights that make them so powerful, generate images with existing models to get a feel for how they work, and then train our own to deepen this understanding further. The field is still rapidly evolving, but the topics covered here should give you a solid foundation to build on, which will be extended further in Chapters 5, 7, and 8.

The high-level idea of diffusion models is that they receive images blurred with noise and learn to denoise them, outputting a clear image. When diffusion models are trained, the dataset contains images with different amounts of noise (even when the input is pure noise). In inference, we can begin with pure noise, and the model will generate an image that matches the training distribution. The model does multiple iterations to accomplish this, correcting itself and leading to impressively high-quality generations.

The Key Insight: Iterative Refinement

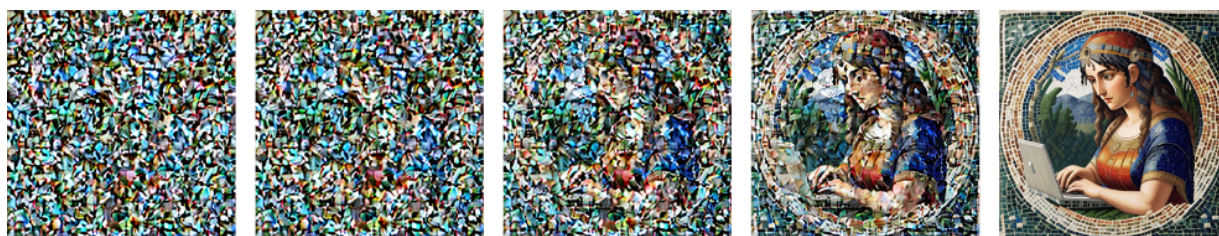


Figure 4-1. Figure 4-1. Progressive denoising process.

So, what is it that makes diffusion models so powerful?

Previous techniques, such as VAEs or GANs, generate their final output via a single forward pass of the model. This means the model must get everything right on the first try. If it makes a mistake, it can't go back and fix it. Diffusion models, on the other hand, generate their output by iterating over many steps¹. This *iterative refinement* allows the model to correct mistakes

made in previous steps and gradually improve the output, as shown in Figure 4-1. To illustrate this, let's look at an example of a diffusion model in action.

We can load a pre-trained diffusion model using the Hugging Face *diffusers* library. The library provides a high-level pipeline that can be used to create images directly. We'll load the [ddpm-celebahq-256 model](#), one of the first shared diffusion models for image generation. This model was trained with the CelebA-HQ dataset, a then popular dataset of high-quality images of celebrities, so it will generate images that look like they came from that dataset. We'll use this model to generate an image from noise.

```
import torch
from diffusers import DDPMPipeline

from genaibook.core import get_device

# We can set the device to use our GPU or CPU
device = get_device()

# Load the pipeline
image_pipe =
DDMPipeline.from_pretrained("google/ddpm-
celebahq-256")
```

```
image_pipe.to(device)

# Sample an image
image_pipe().images[0]
```

```
Loading pipeline components...: 0%|
| 0/2 [00:00<?, ?it/s]
```

```
0%|                                | 0/1000 [00:00<?, ?it/s]
```



Figure 4-2. Example image generated with the DDPM pipeline after 1000 iterations.

The pipeline does not show us what is under the hood, so let's dive into its internals. If you run the code, you will notice that generation took 1,000 steps. This diffusion pipeline has to go

through 1,000 refinement steps (and forward passes) to get to the final image shown in Figure 4-2. This is one of the major drawbacks of the vanilla diffusion models compared to the GANs - they require many steps to generate high-quality images, making the models slow at inference time.

We can re-create this *sampling* process step by step to understand better what is happening under the hood. At the beginning of the diffusion process, we initialize our sample $\mathbf{x}$ with a batch of four random images (in other words, we sample some random noise). We'll run 30 steps to progressively denoise the input images and end up with a sample from the real distribution.

Let's generate some images! On the left side of Figure 4-3, you can see the input at a given step (beginning with the random noise). You can see the model's prediction for the final images on the right. The results of the first row are not particularly good. Instead of jumping right to that final predicted image in a given diffusion step, we only modify the input $\mathbf{x}$ (shown on the left) by a small amount in the direction of the prediction. We then feed this new, slightly better $\mathbf{x}$ through the model again for the next step, hopefully resulting in a slightly improved prediction, which can be used to update $\mathbf{x}$ a little more, and so

on. With enough steps, the model can produce some impressively realistic images.

```
from genaibook.core import
plot_noise_and_denoise

# The random starting point is a batch of 4
images
# Each image is 3-channel (RGB) 256x256 pixel
image
image = torch.randn(4, 3, 256,
256).to(device)

# Set the specific number of diffusion steps
image_pipe.scheduler.set_timesteps(num_inference_

# Loop through the sampling timesteps
for i, t in
enumerate(image_pipe.scheduler.timesteps):
    # Get the prediction given the current
sample x and the timestep t
    # As we're running inference, we don't
need to calculate gradients,
    # so we can use torch.inference_mode().
    with torch.inference_mode():
        # We need to pass in the timestep t
so that the model knows what
        # timestep it's currently at. We'll
```

```

learn more about this in the
    # coming sections.
    noise_pred = image_pipe.unet(image,
t) ["sample"]

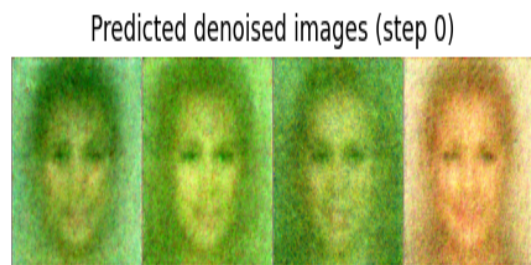
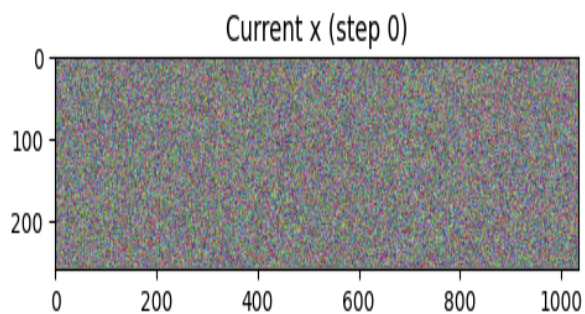
    # Calculate what the updated x should
look like with the scheduler
    scheduler_output =
image_pipe.scheduler.step(noise_pred, t,
image)

    # Update x
    image = scheduler_output.prev_sample

    # Occasionally display both x and the
predicted denoised images
    if i % 10 == 0 or i ==
len(image_pipe.scheduler.timesteps) - 1:

plot_noise_and_denoise(scheduler_output, i)

```



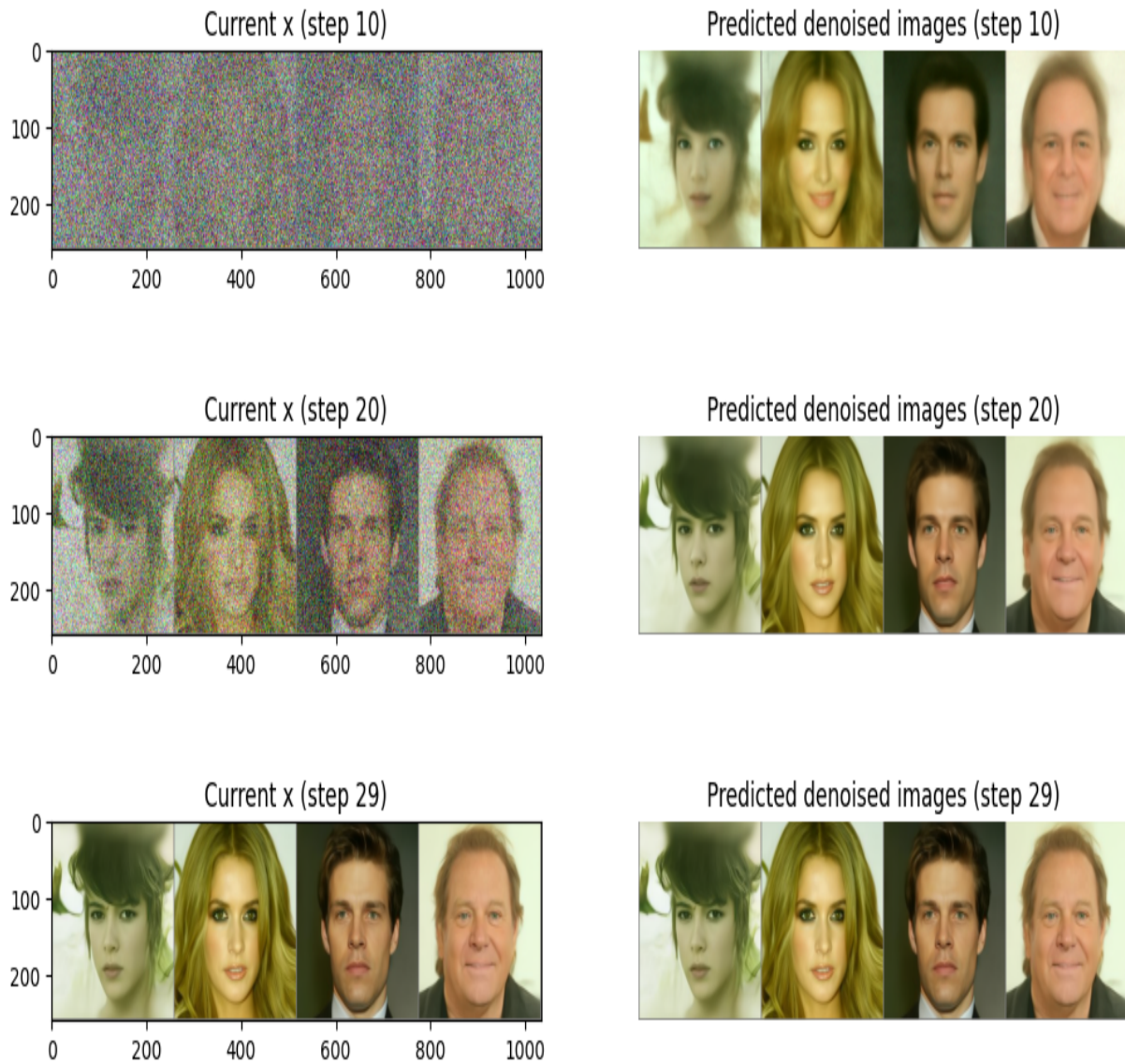


Figure 4-3. Denoising diffusion process. Predictions at step t are fed back into the model with some noise added.

Don't worry if that chunk of code looks intimidating - we'll explain how this all works throughout this chapter. Focus on the idea for now.

This core idea of learning how to iteratively refine a noisy input can be applied to a wide range of tasks. This chapter will focus on unconditional image generation, generating images that resemble the training data distribution. For example, we can train an unconditional image generation model with a dataset of butterflies so it can also generate new, high-quality images. This model would not be able to create images different than the distribution of its training dataset, so don't expect it to generate dinosaurs. In Chapter 5, we'll do a deep dive into diffusion models conditioned on text, but we can do many other things. Diffusion models have been applied to audio, video, text, 3D objects, protein structures, and other domains. While most implementations use some variant of the denoising approach we'll cover here, emerging approaches that apply different types of "corruption" (always combined with iterative refinement) may move the field beyond the current focus on denoising diffusion.

Training a Diffusion Model

In this section, we're going to train a diffusion model from scratch to gain a better understanding of how they work. We'll start by using components from the *diffusers* library. As the chapter progresses, we'll gradually demystify how each

component works. Training a diffusion model is relatively straightforward compared to other generative models. To train a model, we repeatedly:

1. Load some images from the training data
2. Add noise in different amounts. Remember, we want the model to do a good job estimating how to "fix" (denoise) both extremely noisy images and images that are close to perfect, so we want a dataset with diverse amounts of noise
3. Feed the noisy versions of the inputs into the model
4. Evaluate how well the model does at denoising these inputs
5. Use this information to update the model weights

To generate new images with a trained model, we begin with a completely random input and repeatedly feed it through the model, updating the input on each iteration by a small amount based on the model prediction. As we'll see, several sampling methods streamline this process to generate good images with as few steps as possible.

The Data

For this example, we'll use a dataset of images from the Hugging Face Hub- specifically, [this collection of 1000 butterfly](#)

[pictures](#)². Later on, in the projects section, you will see how to use your own data.

```
from datasets import load_dataset

dataset =
load_dataset("huggan/smithsonian_butterflies_subset",
split="train")
```

We must prepare the data before using it to train a model. Images are typically represented as a grid of pixels. Unlike the previous chapter, where we used grayscale images, these images are in color. Each pixel is represented with color values between 0 and 255 for each of the three color channels (Red, Green, and Blue). To process these and make them ready for training, we:

- Resize them to a fixed size. This is necessary because the model expects all images to have the same dimensions.
- (Optional) Add some augmentation by randomly flipping them horizontally, making the model more robust and allowing us to train with more data. Augmentation (Figure 4-4) is a common practice in Computer Vision tasks, as it helps the model generalize better to unseen data. Flipping

is just one technique of augmentation with image data. Other techniques are translating, scaling, and rotating.

- Convert them to a PyTorch tensor (representing the color values as floats between 0 and 1). Model inputs must always be formatted as multi-dimensional matrices, or tensors.
- Normalize them to have a mean of 0, with values between -1 and 1. This is a common practice in training deep learning models, as it helps the model learn faster and more effectively.

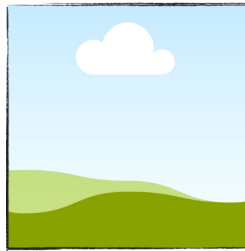


Figure 4-2. Figure 4-4. Augmentation creates more data from the training dataset, improving generalization.

We can define these transformations using

`torchvision.transforms:`³

```
from torchvision import transforms

image_size = 64

# Define data augmentations
preprocess = transforms.Compose(
    [
        transforms.Resize((image_size,
image_size)), # Resize
        transforms.RandomHorizontalFlip(), #
Randomly flip (data augmentation)
        transforms.ToTensor(), # Convert to
tensor (0, 1)
        transforms.Normalize([0.5], [0.5]),
# Map to (-1, 1)
    ]
)
```

The *datasets* library provides a convenient method,

`set_transform()`, which allows us to specify transformations that will be applied on the fly as the data is used. Finally, we can wrap the dataset with a `DataLoader`, a loading utility that

makes it easy to iterate over batches of data, simplifying our training code.

```
def transform(examples):  
    examples = [preprocess(image) for image  
in examples["image"]]  
    return {"images": examples}  
  
dataset.set_transform(transform)  
batch_size = 16  
  
train_dataloader =  
torch.utils.data.DataLoader(  
    dataset, batch_size=batch_size,  
    shuffle=True  
)
```

We can check that this worked by loading a batch and inspecting the images. Figure 4-5 shows an example batch from the training set.⁴

```
from genaibook.core import show_images  
  
batch = next(iter(train_dataloader))  
  
# When we normalized, we mapped (0, 1) to
```

```
(-1, 1)
# Now we map back to (0, 1) for display
show_images(batch["images"][:8] * 0.5 + 0.5)
```

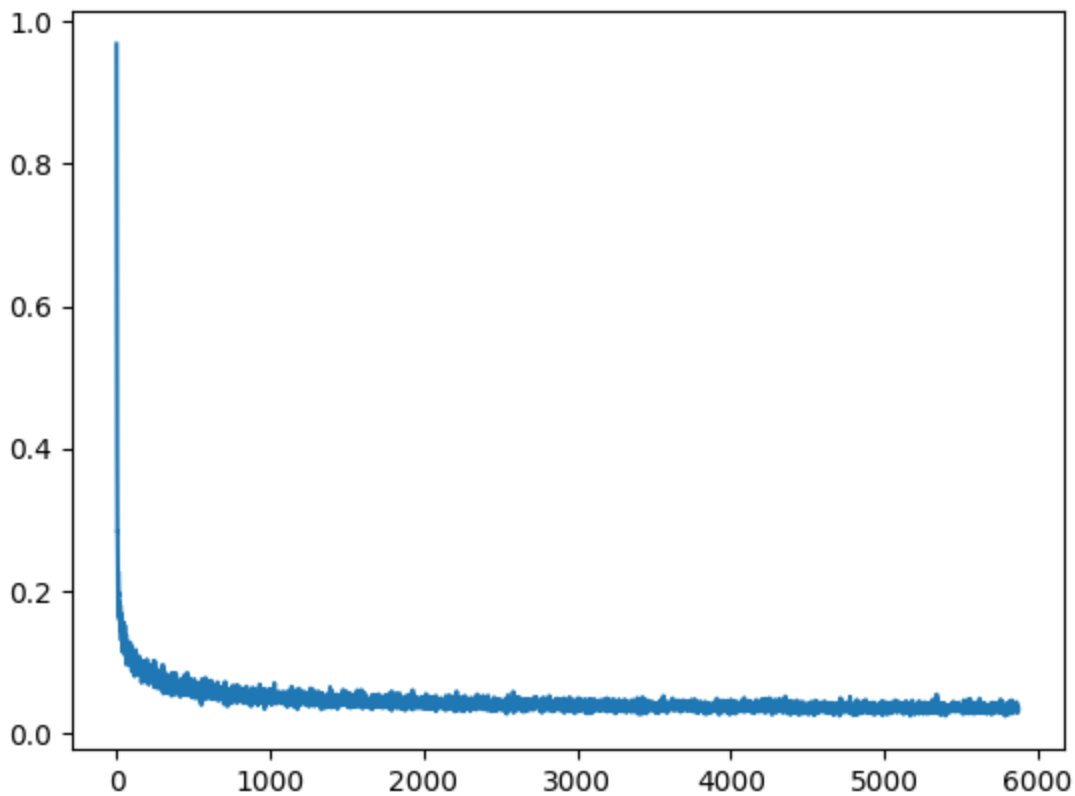


Figure 4-5. A few butterflies from the training set.

Adding Noise

How do we gradually corrupt our data? The most common approach is to add noise to the images. We will add different amounts of noise to the training data, as the goal is to train a robust model to denoise no matter how much noise is in the

input. The amount of noise we add is controlled by a noise schedule, which is a critical aspect of diffusion models.

Different papers and approaches tackle this in different ways.

For now, let's explore one common approach in action based on the *DDPM paper*.⁵ In *diffusers*, adding noise is handled by a class called a `Scheduler`, which takes in a batch of images and a list of timesteps and determines how to create the noisy versions of those images. We'll explore the math behind this later in the chapter, but for now, let's see how it works in practice. The following code snippet applies increasingly larger amounts of noise to each one of the input images, with the result shown in Figure 4-6.

```
from diffusers import DDPMScheduler

# We'll learn about beta_start and beta_end
# in the next sections
scheduler = DDPMScheduler(
    num_train_timesteps=1000,
    beta_start=0.001, beta_end=0.02
)

# Create a tensor with 8 evenly spaced values
# from 0 to 999
timesteps = torch.linspace(0, 999, 8).long()
```

```
# We load 8 images from the dataset and
# add increasing amounts of noise to them
x = batch["images"][:8]
noise = torch.rand_like(x)
noised_x = scheduler.add_noise(x, noise,
timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0,
1))
```

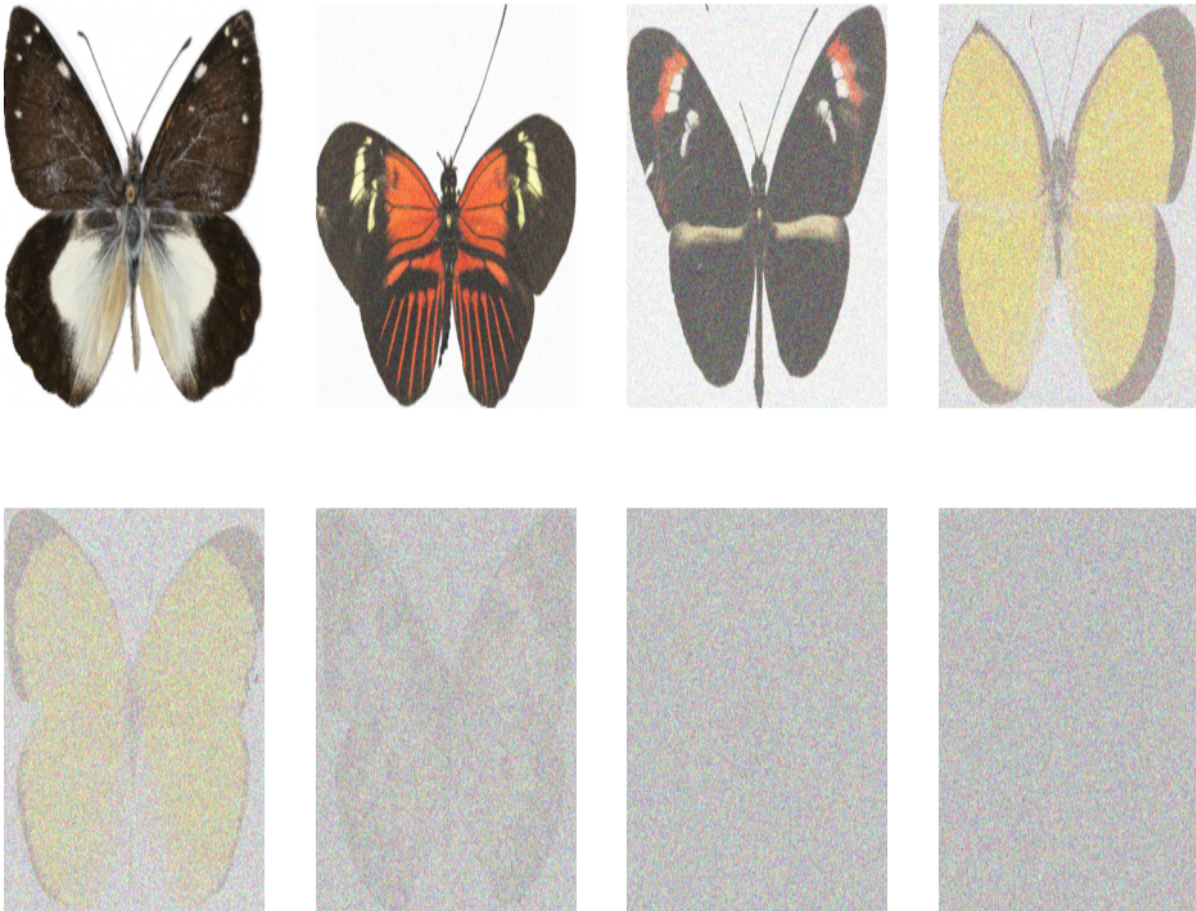


Figure 4-6. Using a scheduler to add varying amounts of noise to input images.

During training, we'll pick the timesteps at random. The scheduler takes some parameters (`beta_start` and `beta_end`), which it uses to determine how much noise should be present for a given timestep. We will cover schedulers in more detail in the *In Depth: Noise Schedules* section later on in this chapter.

The UNet

The *UNet* is a convolutional neural network invented for tasks such as image segmentation, where the desired output has the same shape as the input. For example, UNets are used in medical imaging to segment different anatomical structures.

The UNet consists of a series of *downsampling* layers that reduce the spatial size of the input, followed by a series of *upsampling* layers that increase the spatial extent of the input again. The downsampling layers are typically followed by skip connections that connect the downsampling layers outputs to the upsampling layers inputs. This allows the upsampling layers to incorporate finer details from earlier layers, preserving important high-resolution information during the denoising process.

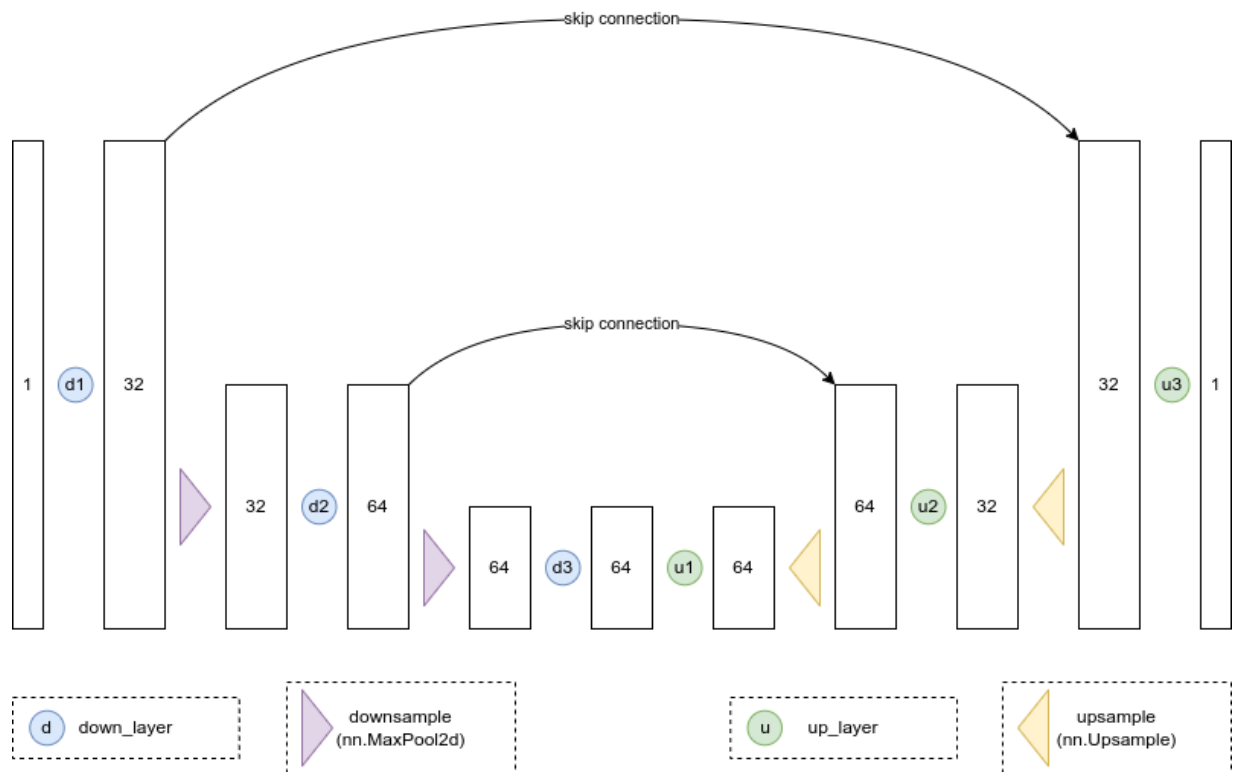


Figure 4-3. Figure 4-7. Architecture of a simplified UNet.

The UNet architecture used in the *diffusers* library is more advanced than the original UNet proposed in 2015,⁶ with additions like attention and residual blocks. We'll take a closer look later, but the key idea here is that it can take in an input and produce a prediction that is the same shape. In diffusion models, the input can be a noisy image, and the output can be the predicted noise. With this information, we can now denoise the input image.

Here's how we might create a UNet and feed our batch of noisy images through it:

```

from diffusers import UNet2DModel

# Create a UNet2DModel
model = UNet2DModel(
    in_channels=3,  # 3 channels for RGB
    images
    sample_size=64,  # Specify our input size
    # The number of channels per block
    affects the model size
    block_out_channels=(64, 128, 256, 512),
    down_block_types=(
        "DownBlock2D",
        "DownBlock2D",
        "AttnDownBlock2D",
        "AttnDownBlock2D",
    ),
    up_block_types=("AttnUpBlock2D",
        "AttnUpBlock2D", "UpBlock2D", "UpBlock2D"),
).to(device)

# Pass a batch of data through to make sure
it works
with torch.inference_mode():
    out = model(noised_x.to(device),
    timestep=timesteps.to(device)).sample

```

```
print(noised_x.shape)
print(out.shape)
```

```
torch.Size([8, 3, 64, 64])
torch.Size([8, 3, 64, 64])
```

Note that the output is the same shape as the input, which is exactly what we want.

Training

Now that we have our data and model ready, let's train it. For each training step, we:

1. Load a batch of images.
2. Add noise to the images. The amount of noise added depends on a specified number of timesteps: the more timesteps, the more noise. As mentioned, we want our model to denoise images with little noise and images with lots of noise. To achieve this, we'll add random amounts of noise, so we'll pick a random number of timesteps.
3. Feed the noisy images into the model.
4. Calculate the loss using mean squared error (MSE). MSE is a common loss function for regression tasks, including the

UNet model's noise prediction. It measures the average squared difference between predicted and true values, penalizing larger errors more. In the UNet model, MSE is calculated between predicted and actual noise, helping the model generate more realistic images by minimizing the loss. This is called the noise or epsilon objective.

5. Backpropagate the loss and update the model weights with the optimizer.

Here's what all of that looks like in code. Training will take a while, so this is a great moment to pause, review the chapter's content, or get some food.

```
from torch.nn import functional as F

num_epochs = 50 # How many runs through the
data should we do?
lr = 1e-4 # What learning rate should we use
optimizer =
torch.optim.AdamW(model.parameters(), lr=lr)
losses = [] # Somewhere to store the loss
values for later plotting

# Train the model (this takes a while)
for epoch in range(num_epochs):
    for batch in train_dataloader:
```

```
        # Load the input images
        clean_images =
batch["images"].to(device)

        # Sample noise to add to the images
        noise =
torch.randn(clean_images.shape).to(device)

        # Sample a random timestep for each
image
        timesteps = torch.randint(
            0,

scheduler.config.num_train_timesteps,
            (clean_images.shape[0],),
            device=device,
        ).long()

        # Add noise to the clean images
according
        # to the noise magnitude at each
timestep
        noisy_images =
scheduler.add_noise(clean_images, noise,
timesteps)

        # Get the model prediction for the
noise
```



```

        # The model also uses the timestep as
an input
        # for additional conditioning
        noise_pred = model(noisy_images,
timesteps, return_dict=False)[0]

        # Compare the prediction with the
actual noise
        loss = F.mse_loss(noise_pred, noise)

        # Store the loss for later plotting
losses.append(loss.item())

        # Update the model parameters with
the optimizer based on this loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        # Print out the average of the loss
values for this epoch:
        avg_loss = sum(losses[-
len(train_dataloader) :]) /
len(train_dataloader)
        print(
            f"Finished epoch {epoch}. Average

```

```
loss for this epoch: {avg_loss:05f}"  
    )
```

Now that the model is trained, let's plot the training loss.

```
from matplotlib import pyplot as plt  
  
plt.subplots(1, 2, figsize=(12, 4))  
  
plt.subplot(1, 2, 1)  
plt.plot(losses)  
plt.title("Training loss")  
plt.xlabel("Training step")  
  
plt.subplot(1, 2, 2)  
plt.plot(range(400, len(losses)),  
losses[400:])  
plt.title("Training loss from step 400")  
plt.xlabel("Training step");
```

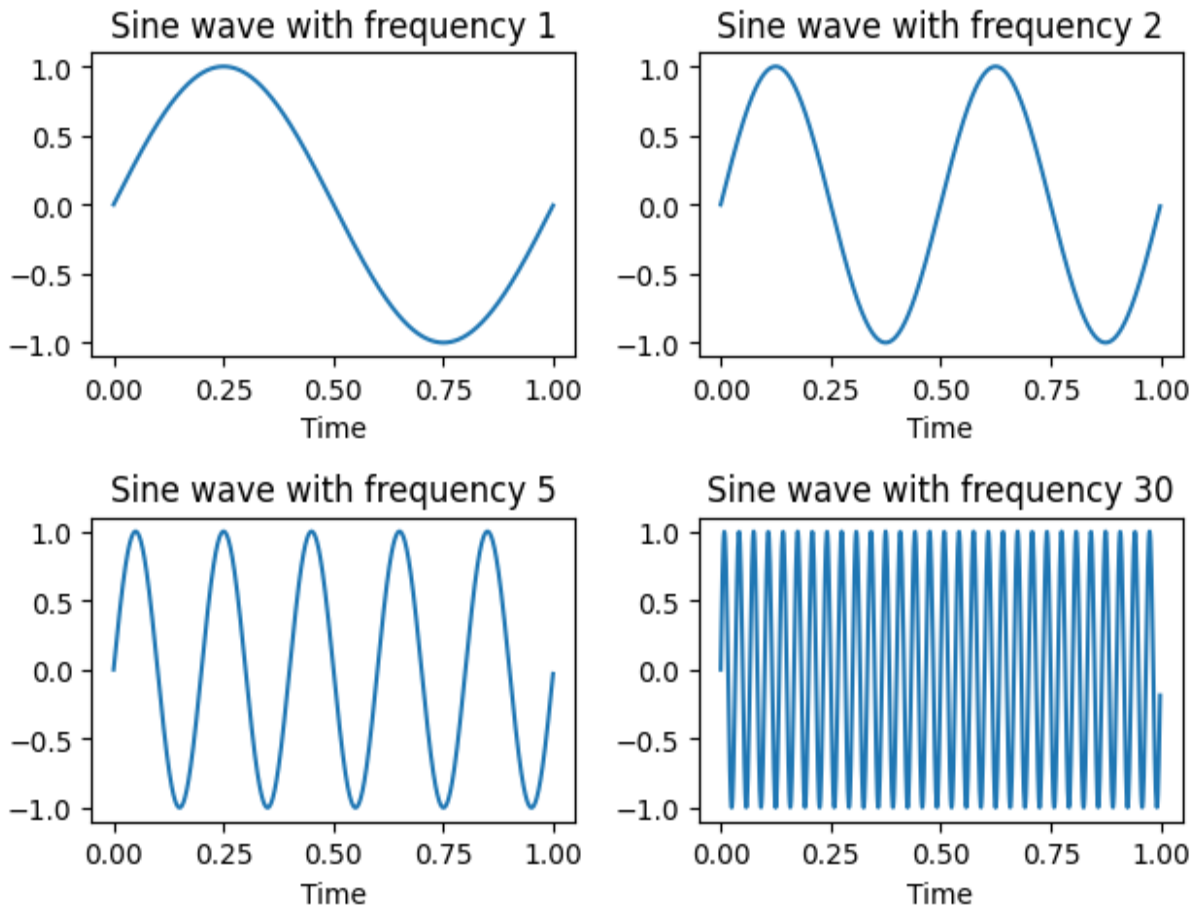


Figure 4-8. Training loss of a simple UNet for diffusion. The left shows all the steps while the right skips the first 400 steps.

The loss curve, shown in Figure 4-8, trends downwards as the model learns to denoise the images. The curve is somewhat noisy - the loss is not very stable. This is because each iteration uses different numbers of noising time steps. It is hard to tell whether this model will be good at generating samples by looking at the mean squared error of the noise predictions, so let's move on to the next section and see how well it does.

Sampling

Now that we have a model, let's do inference and generate some images. The *diffusers* library uses the idea of pipelines to bundle together all of the components needed to generate samples with a diffusion model. We can use a pipeline to test the UNet we just trained, a few generations are shown in Figure 4-9.⁷:

```
pipeline = DDPMPipeline(unet=model,  
scheduler=scheduler)  
ims = pipeline(batch_size=4).images  
show_images(ims, nrows=1)
```

```
0%|          | 0/1000 [00:00<?, ?it/s]
```



Figure 4-9. Sample low-resolution generations after training the UNet.

Offloading the job of creating samples to the pipeline doesn't show us what is going on under the hood. So, let's do a simple sampling loop showing how the model gradually refines the input image based on the code in the pipeline's `call()` method (Figure 4-10).

```
# Random starting point (4 random images):
sample = torch.randn(4, 3, 64, 64).to(device)

for t in scheduler.timesteps:
    # Get the model prediction
    with torch.inference_mode():
        noise_pred = model(sample, t)
["sample"]

    # Update sample with step
    sample = scheduler.step(noise_pred, t,
sample).prev_sample

show_images(sample.clip(-1, 1) * 0.5 + 0.5,
nrows=1)
```

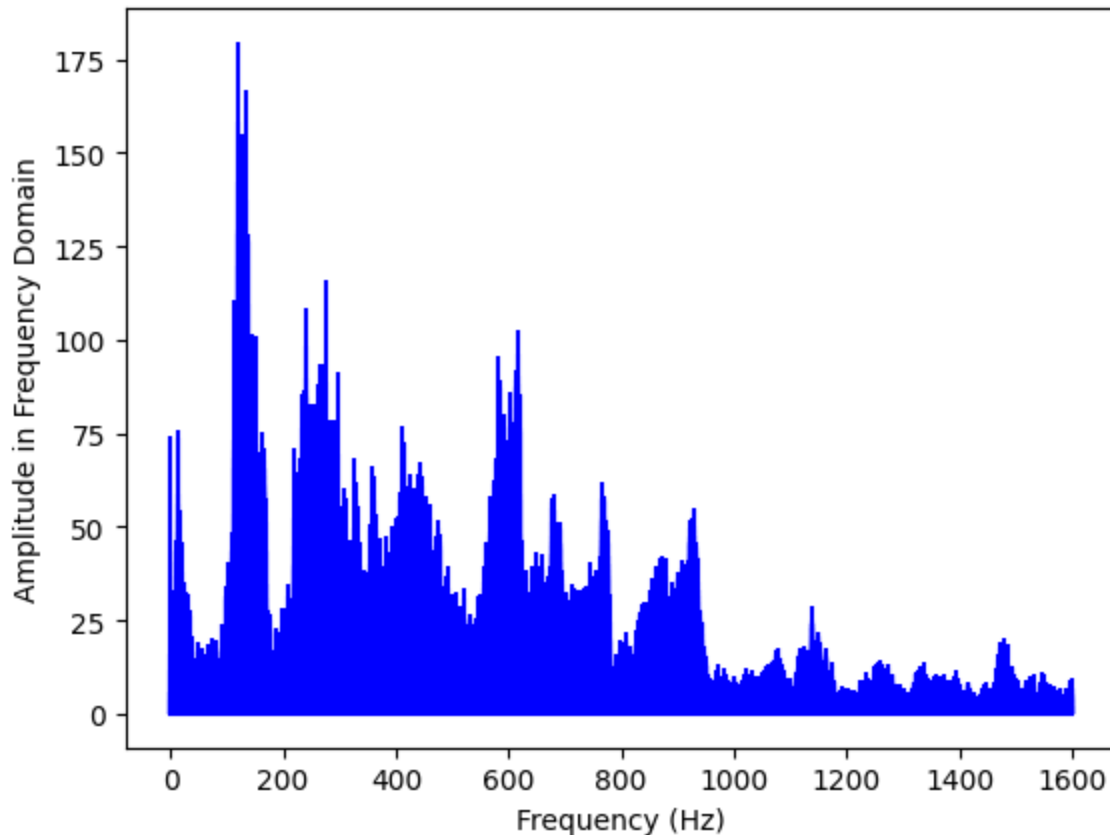


Figure 4-10: A few additional images from the same model, generated with a sampling loop.

This is the same code we used at the beginning of the chapter to illustrate the idea of iterative refinement, but now you better understand what is happening here. If you look at the implementation of the `DDPMPipeline` in the *diffusers* library, you'll see that the logic closely resembles our implementation in the previous snippet.

We start with a completely random input, which the model then refines in a series of steps. Each step is a small update to the

input based on the model's prediction for the noise at that timestep. We're still abstracting away some complexity behind the call to `pipeline.scheduler.step()` ; later, we will dive deeper into different sampling methods and how they work.

Evaluation

Evaluating generative models is complex - it's a subjective task in nature. For example, given an input prompt "image of a cat with sunglasses", there are many potential correct generations. A common approach is to combine qualitative evaluation (e.g., by having humans compare generations) and quantitative metrics, which provide a framework for evaluation but don't necessarily correspond to high image quality.

FID (Fréchet Inception Distance) scores can evaluate generative model performance. FID scores compare how similar two image datasets are. Using a pretrained neural network (example shown in Figure 4-11), they measure how closely generated samples match real samples by comparing statistics between feature maps extracted from both datasets. The lower the score, the better the quality and realism of generated images produced by a given model. FID scores are popular due to their ability to provide an "objective" comparison metric for

different types of generative networks without relying on human judgment.

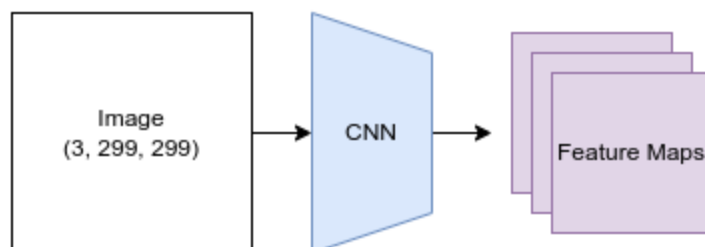


Figure 4-4. Figure 4-11: CNN network used to extract feature maps from images.

As convenient as FID scores are, there are important caveats to be aware of (which might be true for other evaluation metrics as well):

- FID scores are designed to compare two distributions. Because of this, it assumes that we have access to a source dataset for comparison. A second issue is that you cannot calculate the FID score of a single generation. If we have one image, there's no way to calculate its FID score.
- The FID score for a given model depends on the number of samples used to calculate it, so when comparing models, we need to make sure both reported scores are calculated using the same number of samples. The common practice is to use 50,000 samples for this purpose, although to save time, you may evaluate a smaller number of samples

during development and only do the complete evaluation once you're ready to publish the results.

- The FID can be sensitive to many factors. For example, a different number of inference steps will lead to a very different FID. The scheduler (DDPM in this case) will also affect the FID.
- When calculating the FID, images are resized to 299x299 images. This makes it less useful as a metric for extremely low or high-resolution images. There are also minor differences between how resizing is handled by different deep learning frameworks, which can result in slight differences in the FID score.
- The network used as a feature extractor for FID is typically a model trained on the ImageNet classification task⁸. When generating images in a different domain, the features learned by this model may be less useful. A more accurate approach is to first train a classification network on domain-specific data, making comparing scores between different papers and techniques harder. For now, the ImageNet model is the standard choice.
- If you save generated samples for later evaluation, the format and compression can affect the FID score. Avoid low-quality JPEG images where possible.

Even if you account for all these caveats, FID scores are just a rough measure of quality and do not perfectly capture the nuances of what makes images look more " `real` ". The evaluation of generative models is an active research area. Standard metrics like Kernel Inception Distance (KID) and Inception Score share similar issues with FID. So, use these metrics to get an idea of how one model performs relative to another, but also look at the actual images generated by each model to get a better sense of how they compare.

Image quality, as measured by FID or KID, is only one of the metrics we can use to evaluate the performance of text-to-image models. Efforts such as [HEIM](#) (*Holistic Evaluation of Text-To-Image Models*) attempt to take into account additional desirable characteristics of text-to-image models, such as prompt adherence, originality, reasoning capabilities, multilinguality, absence of bias and toxicity, and others.

Human preference is still the gold standard for quality in what is ultimately a fairly subjective field. For example, the [Parti Prompts dataset](#) contains 1600 prompts of varying difficulties and categories and allows comparing text-to-image models such as the ones we'll explore in Chapter 5.⁹

In Depth: Noise Schedules

In the training example above, one of the steps was to "add noise in different amounts". We achieved this by picking a random timestep between 0 and 1000 and then relying on the scheduler to add the appropriate amount of noise. Likewise, during inference, we again relied on the scheduler to tell us which timesteps to use and how to move from one to the next, given the model predictions. Choosing how much noise to add is a crucial design decision that can drastically affect the performance of a given model. In this section, we'll see why this is the case and explore different approaches used in practice.

Why Add Noise?

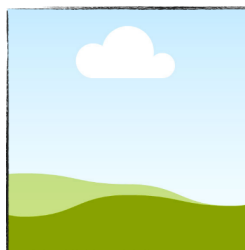


Figure 4-5. Figure 4-12: The general principles of diffusion work for other types of corruption, not just Gaussian noise (image from the Cold Diffusion paper, <https://arxiv.org/pdf/2208.09392>)

At the start of this chapter, we said that the key idea behind diffusion models is that of iterative refinement. During training, we *corrupt* an input by different amounts. During inference, we begin with a maximally corrupted input (that is, a pure noise image) and iteratively de-corrupt it, expecting to end up with a nice final result eventually.

So far, we've focused on one specific kind of corruption: adding Gaussian noise. Gaussian noise is a type of noise that follows a normal distribution, which as we saw in Chapter 3 has most

values around the mean and fewer values as we get further away.¹⁰ One reason for this focus is the theoretical underpinnings of diffusion models, which assume the use of Gaussian noise – if we use a different corruption method, we are no longer technically doing diffusion. However, a paper titled Cold Diffusion¹¹ demonstrated that we do not necessarily need to constrain ourselves to this method just for theoretical convenience. They showed (Figure 4-12) that a diffusion-model-like approach works for many different corruption methods. That means that rather than using noise, we can use other image transformations. For example, models such as Muse, MaskGIT, and Paella have used random token masking or replacement as equivalent corruption methods.

Nonetheless, adding noise remains the most popular approach for several reasons:

- We can easily control the amount of noise added, giving a smooth transition from "perfect" to "completely corrupted". This is not the case for something like reducing the resolution of an image, which may result in "discrete" transitions.
- We can have many valid random starting points for inference, unlike some methods, which may only have a

limited number of possible initial (fully corrupted) states, such as a completely black image or a single-pixel image.

So, for now, we'll add noise as our corruption method. Next, let's explore how we add noise to our images.

Starting Simple

We have some images `x`, and we'd like to add some random noise to them. We generate pure Gaussian noise of the same dimensions as the input images with `torch.rand_like()`.

```
x = next(iter(train_dataloader))["images"]
[:8]
noise = torch.rand_like(x)
```

One way we could add varying amounts of noise is to linearly interpolate ("lerp" for short) between the images and the noise by some amount. This gives us a function that smoothly transitions from the original image `x` to pure noise as the `amount` varies from 0 to 1:

```
def corrupt(x, noise, amount):
    # Reshape amount so it works correctly
    with the original data
```

```
    amount = amount.view(-1, 1, 1, 1) # make
sure it's broadcastable

    # Blend the original data and noise based
on the amount
    return (
        x * (1 - amount) + noise * amount
    ) # equivalent to x.lerp(noise, amount)
```

Let's see this in action on a batch of data, with the amount of noise varying from 0 to 1:

```
amount = torch.linspace(0, 1, 8)
noised_x = corrupt(x, noise, amount)
show_images(noised_x * 0.5 + 0.5)
```

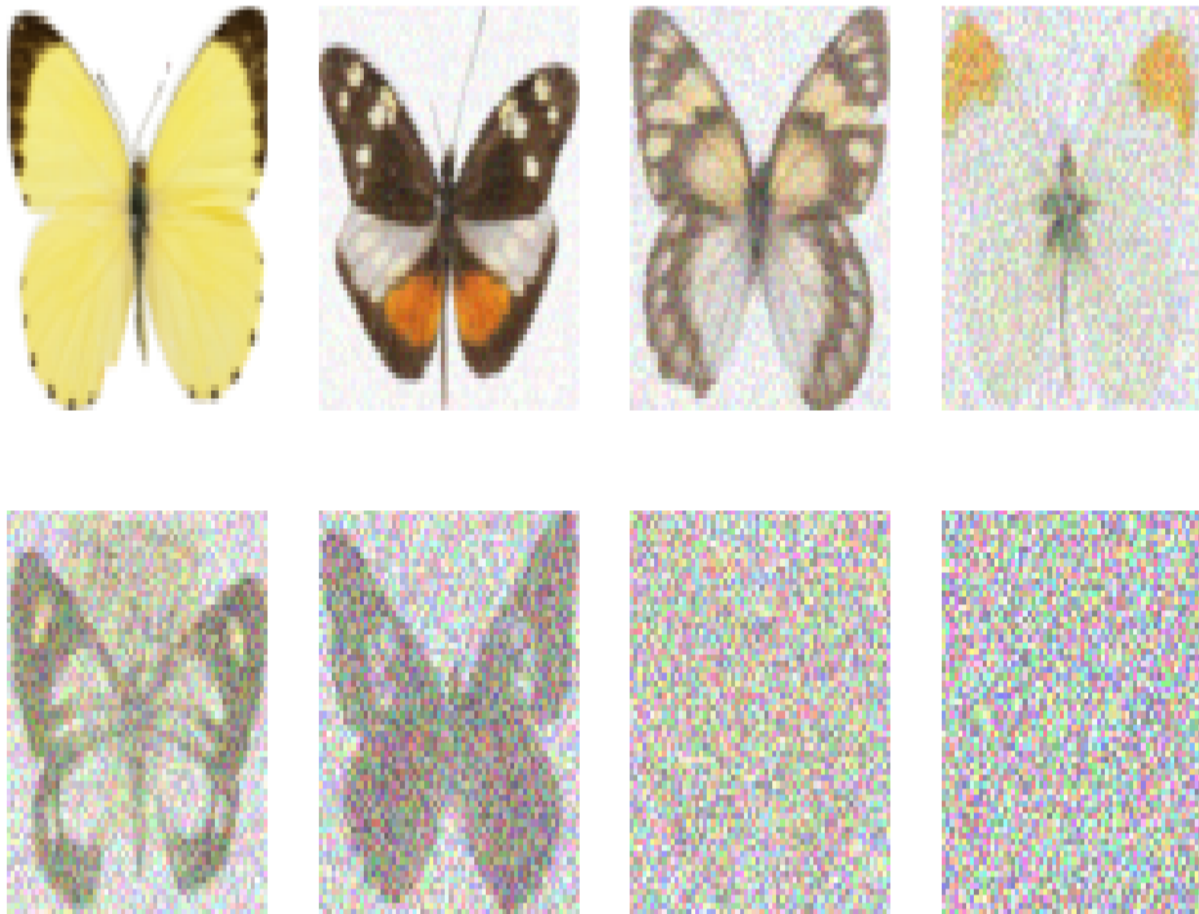



Figure 4-13: Applying noise with linear interpolation between the image and pure noise.

This is doing what we want: smoothly transitioning from the original image to pure noise (Figure 4-13). We've created a noise schedule with the *continuous time* approach, where we represent the full path on a time scale from 0 to 1. Other approaches use a *discrete time* approach, with some large integer number of *timesteps* used to define the noise scheduler. We can wrap our function into a class that converts from

continuous time to discrete timesteps and adds noise appropriately (Figure 4-14):

```
class SimpleScheduler:
    def __init__(self):
        self.num_train_timesteps = 1000

    def add_noise(self, x, noise, timesteps):
        amount = timesteps /
self.num_train_timesteps
        return corrupt(x, noise, amount)

scheduler = SimpleScheduler()
timesteps = torch.linspace(0, 999, 8).long()
noised_x = scheduler.add_noise(x, noise,
timesteps)
show_images(noised_x * 0.5 + 0.5)
```

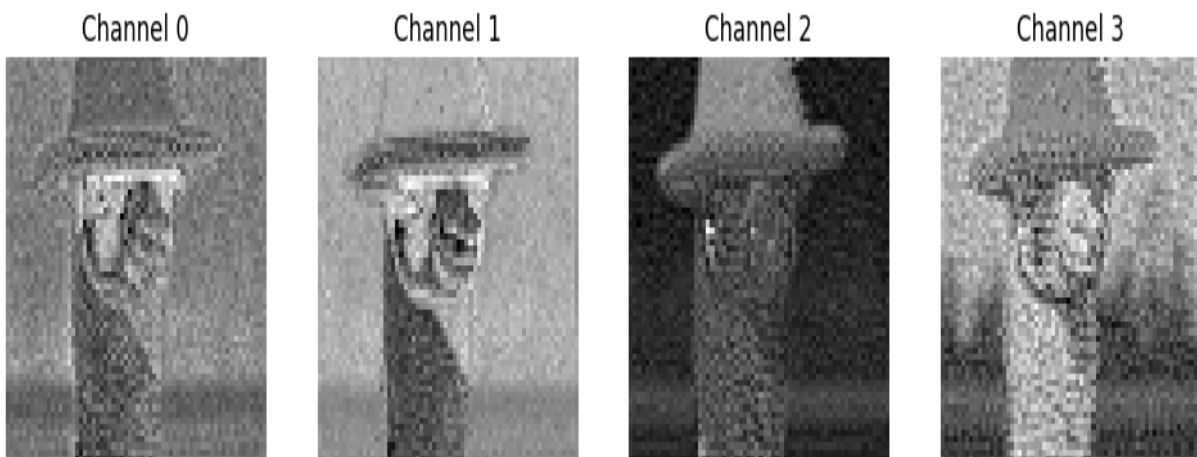


Figure 4-14: Demonstration of the `SimpleScheduler` class, which uses linear interpolation to add noise.

Now we have something we can directly compare to the schedulers used in the *diffusers* library, such as the `DDPMScheduler` we used during training. Let's see how it compares:

```
scheduler = DDPMScheduler(beta_end=0.01)
timesteps = torch.linspace(0, 999, 8).long()
noised_x = scheduler.add_noise(x, noise,
timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0,
1))
```



Figure 4-15: Results from the `DDPMScheduler` on the same batch shown in Figure 4-14.

If you compare Figure 4-14 (our scheduler) with Figure 4-15 (`DDPMScheduler`), the results are not exactly the same, but similar enough to explore training the model with our noise scheduler.

The Math

Let's dive into the underlying math that explains how noise is added to the original images. One thing to remember is that there are many notations and approaches in the literature. For

example, in some papers, the noise schedule is parametrized continuously, so t runs from 0 (no noise) to 1 (fully corrupted), as we did in our `corrupt` function. Other papers use a discrete time approach in which the timesteps are integers and run from 0 to some large number T , typically 1000. It is possible to convert between these two approaches the way we did with our `SimpleScheduler` class - make sure you're consistent when comparing different models. We'll stick with the discrete-time approach here.

A good place to start for going deeper into the math is the DDPM paper or the *Annotated Diffusion Model* blog post.¹² If you feel this section is too dense, it's okay to focus on the high-level concepts and come back to the math later on.

Let's kick things off by defining how to do a single noise step to go from timestep $t-1$ to timestep t . As mentioned earlier, the idea is to add Gaussian noise (ϵ). The noise has unit variance, which controls the spread of the noise values. By adding this noise to the previous step's image, we gradually corrupt the original image, which is a key part of the diffusion model's training process.

$$\mathbf{x}_t = \mathbf{x}_{t-1} + \epsilon$$

To control the amount of noise added at each step, let's introduce β_t . This parameter is defined for all timesteps t and specifies how much noise should be added at each step. In other words, $\mathbf{x}_t$ is a mix of $\mathbf{x}_{t-1}$ and some random noise scaled by β_t . This allows us to gradually increase the amount of noise added to the image as we move through the timesteps, which is a key part of the diffusion model's training process.

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon$$

We can further define the noise addition process as a *distribution*, where the noisy $\mathbf{x}_t$ has a mean $\sqrt{1 - \beta_t} \mathbf{x}_{t-1}$ and a variance of β_t . This distribution helps us model the noise addition process more accurately. This is what the formula looks like in distribution form:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}).$$

We've now defined a distribution to sample x conditioned on the previous value. To get the noisy input at timestep t , we could begin at $t=0$ and repeatedly apply this single step, which would be very inefficient. Instead, we can find a formula to move to any timestep t in one go by doing the *reparameterization trick*. The idea is to precompute the noise schedule, which is defined by the β_t values. We can then define $\alpha_t = 1 - \beta_t$ and α as the cumulative product of all the α

values up to the time t , which can be expressed as

$\alpha_t := \prod_{s=1}^t \alpha_s$. Using these tools and notation, we can redefine the distribution and how to sample at a particular time. The new distribution, $q(\mathbf{x}_t | \mathbf{x}_{t-1})$, has a mean of $\alpha_t \mathbf{x}_{t-1}$ and a variance of $(1 - \alpha_t) \mathbf{I}$.

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \alpha_t \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I}).$$

Exploring this reparameterization trick is part of the challenges at the end of the chapter. We can now sample a noisy image at timestep `t` by using the formula below.

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$$

The equation for x_t shows that the noisy input at timestep `t` is a combination of the original image x_0 (scaled by $\sqrt{\alpha_t}$) and ϵ (scaled by $\sqrt{1 - \alpha_t}$). Note that we can now calculate a sample directly without looping over all previous timesteps, making it much more efficient for training diffusion models.

In the *diffusers* library, the α values are stored in

`scheduler.alphas_cumprod`. Knowing this, we can plot the scaling factors for the original image x_0 and the noise ϵ across the different timesteps for a given scheduler. The *diffusers* library allows us to control the beta values by defining its initial value (`beta_start`), final value (`beta_end`), and how the values will step, for example, linearly

(`beta_schedule=" linear "`). Figure 4-16 shows that the noise is scaled up more as we have more timesteps, as expected.

```
from genaibook.core import plot_scheduler

plot_scheduler(
    DDPMScheduler(beta_start=0.001,
beta_end=0.02, beta_schedule="linear")
)
```

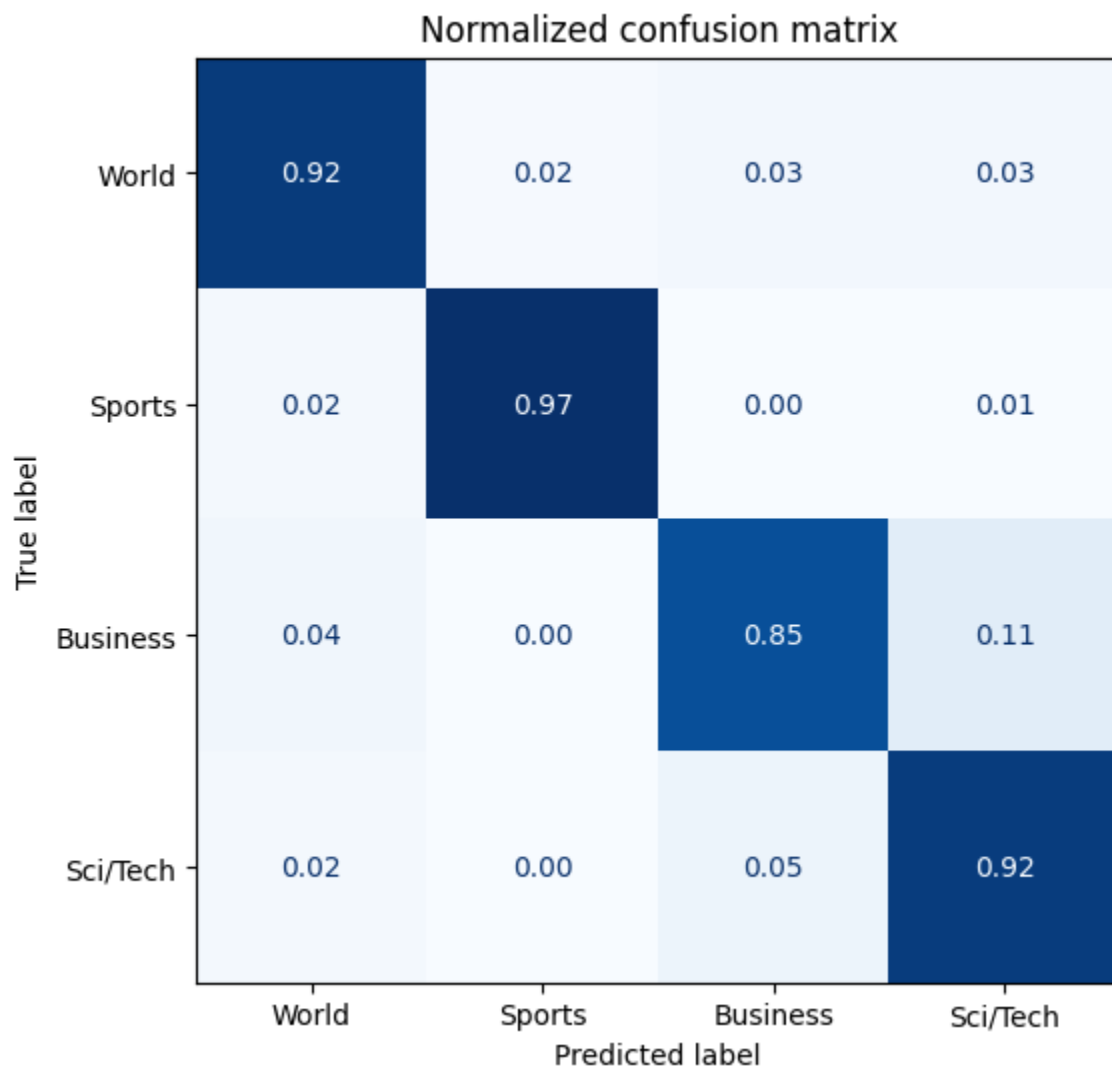



Figure 4-16: `DDPMScheduler`: Amount of noise (orange) added to input image (blue), per timestep).

Our `SimpleScheduler` just linearly mixes between the original image and noise, as we can see in Figure 4-17 if we plot the scaling factors (equivalent to $\sqrt{\alpha_t}$ and $\sqrt{(1 - \alpha_t)}$ in the DDPM case):

```
plot_scheduler(SimpleScheduler())
```



Figure 4-17. `SimpleScheduler`: Amount of noise added, note how it compares with `DDPMScheduler` in Figure 4-17.

A good noise schedule will ensure the model sees a mix of images at different noise levels. The best choice will differ based on the training data. Visualizing a few more options, note that: